MPI Course

Victor Eijkhout

2026 COE 379L / CSE 392

Materials

Textbooks and repositories:
`https://theartofhpc.com`

Justification

The MPI library is the main tool for parallel programming on a large scale.
(Compare: OpenMP and CUDA are limited to a single node)
This course introduces the main concepts through lecturing and exercises.

Table of Contents

1. The SPMD Model *link*
2. Collectives *link*
3. Point-to-point *link*
4. Derived datatypes *link*
5. Communicators *link*
6. MPI I/O *link*
7. One-sided communication *link*
8. Advanced collectives *link*
9. Shared memory *link*
10. Process management *link*
11. Process topologies *link*
12. Trace and performance *link*

1. Basics

1.1. Supercomputer clusters

Cluster setup

Typical cluster:

- Login nodes, where you ssh into; usually shared with 100 (or so) other people. You don't run your parallel program there!
- Compute nodes: where your job is run. They are often exclusive to you: no other users getting in the way of your program.

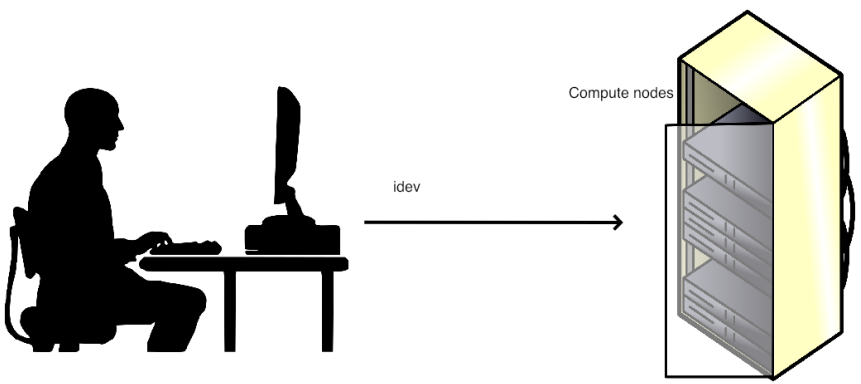
Login nodes

Shared between many users
(how many right now?)
You are allowed to do:

- Compilation
- Post-processing
- Run very short programs (but not MPI)
- Submit jobs for batch execution (`sbatch`)
- Connect for interactive job (`idev`)

Interactive run

- Do not run your programs on a login node.
- Acquire compute nodes with *idev*
- Caveat: only small short jobs; nodes may not be available.
  During training there will be a 'reservation':
  carefully look at messages from *idev*



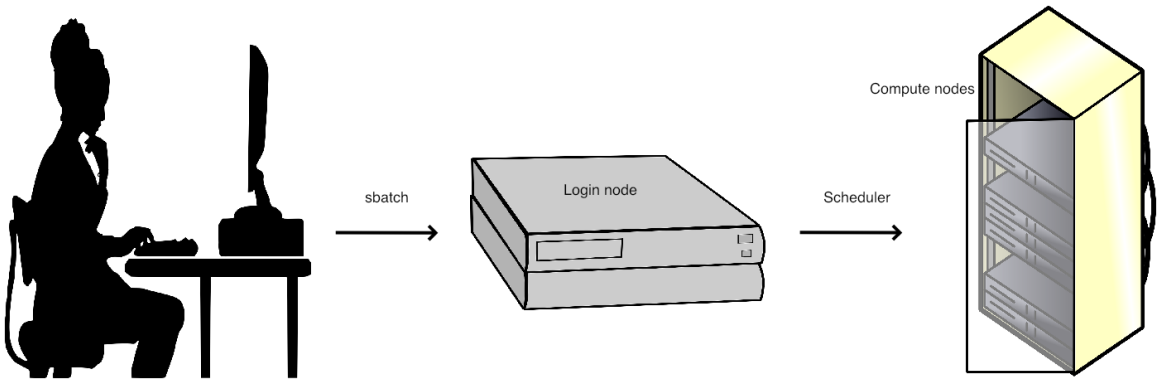
`idev` command

```
1 idev -t hh:mm:ss -N nodes -n cores -p queue
```

- `-t`: time
- `-N`: number of nodes
- `-n`: total number of cores
- `-p`: partition / queue

Batch run

- Submit batch job with *sbatch*
  (on other clusters: *qsub*)
- Your job will be executed . . . Real Soon Now.
-
  - *sbatch*: submit
  - *squeue*: job status
- See userguide for details about queues, sizes, runtimes, . . .



Exercise 1

- Connect to your cluster.
  What is the hostname? How many users are logged in?
- Start an interactive session with `idev`.
  What is the hostname? How many users are logged in?
- Run `ibrun hostname`, also `ibrun -n 3 hostname`.
- Same but use `idev -N 2` to get two nodes.
- Create a jobscript that will run on 10 nodes and run `ibrun hostname` again.
  Submit it with `sbatch`.

1.2. The SPMD model

Overview

In this section you will learn how to think about parallelism in MPI.

Commands learned:

- `MPI_Init`, `MPI_Finalize`,
- `MPI_Comm_size`, `MPI_Comm_rank`
- `MPI_Get_processor_name`,

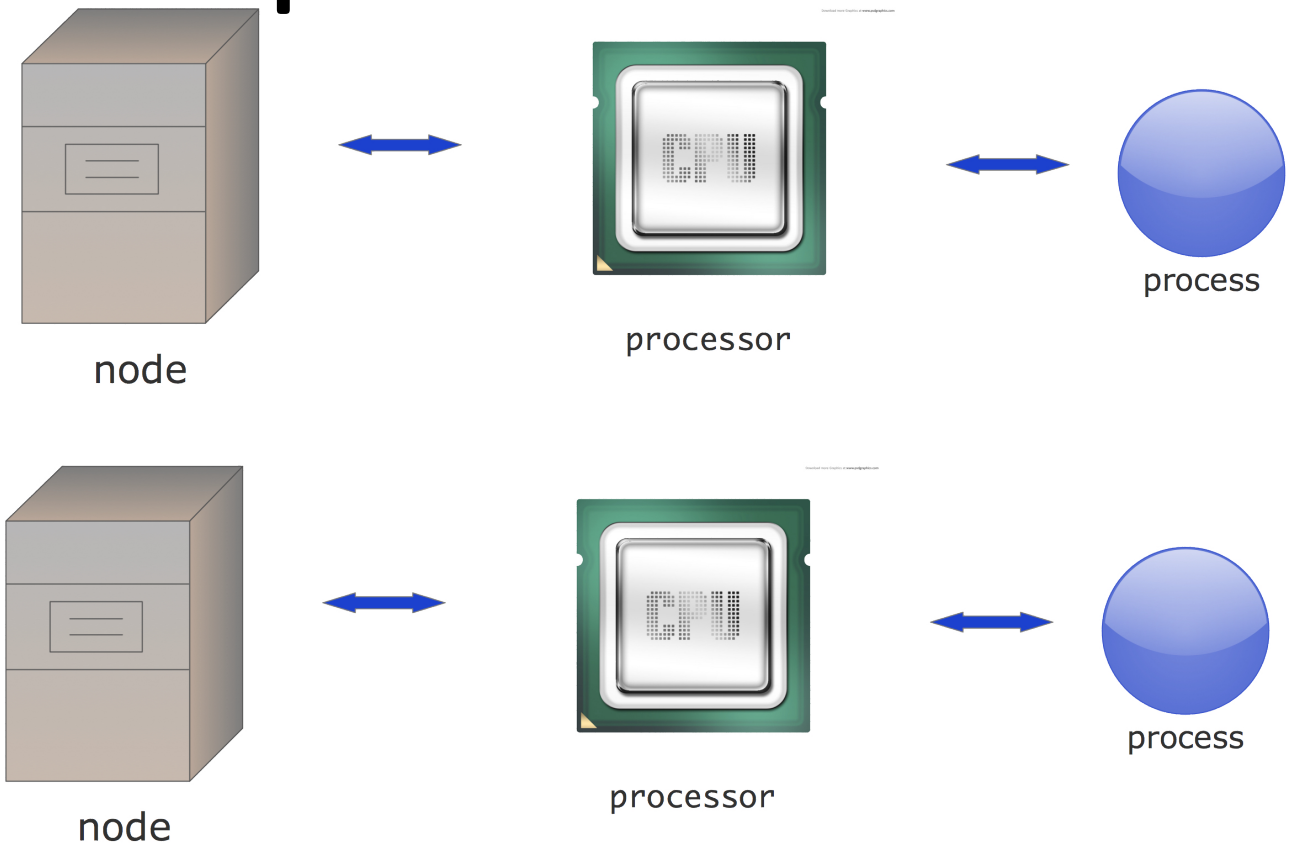
SPMD

The basic model of MPI is
'Single Program Multiple Data':
each process is an instance of the same program.

Symmetry: There is no 'master process', all processes are equal, start and end at the same time.

Communication calls do not see the cluster structure:
data sending/receiving is the same for all neighbors.

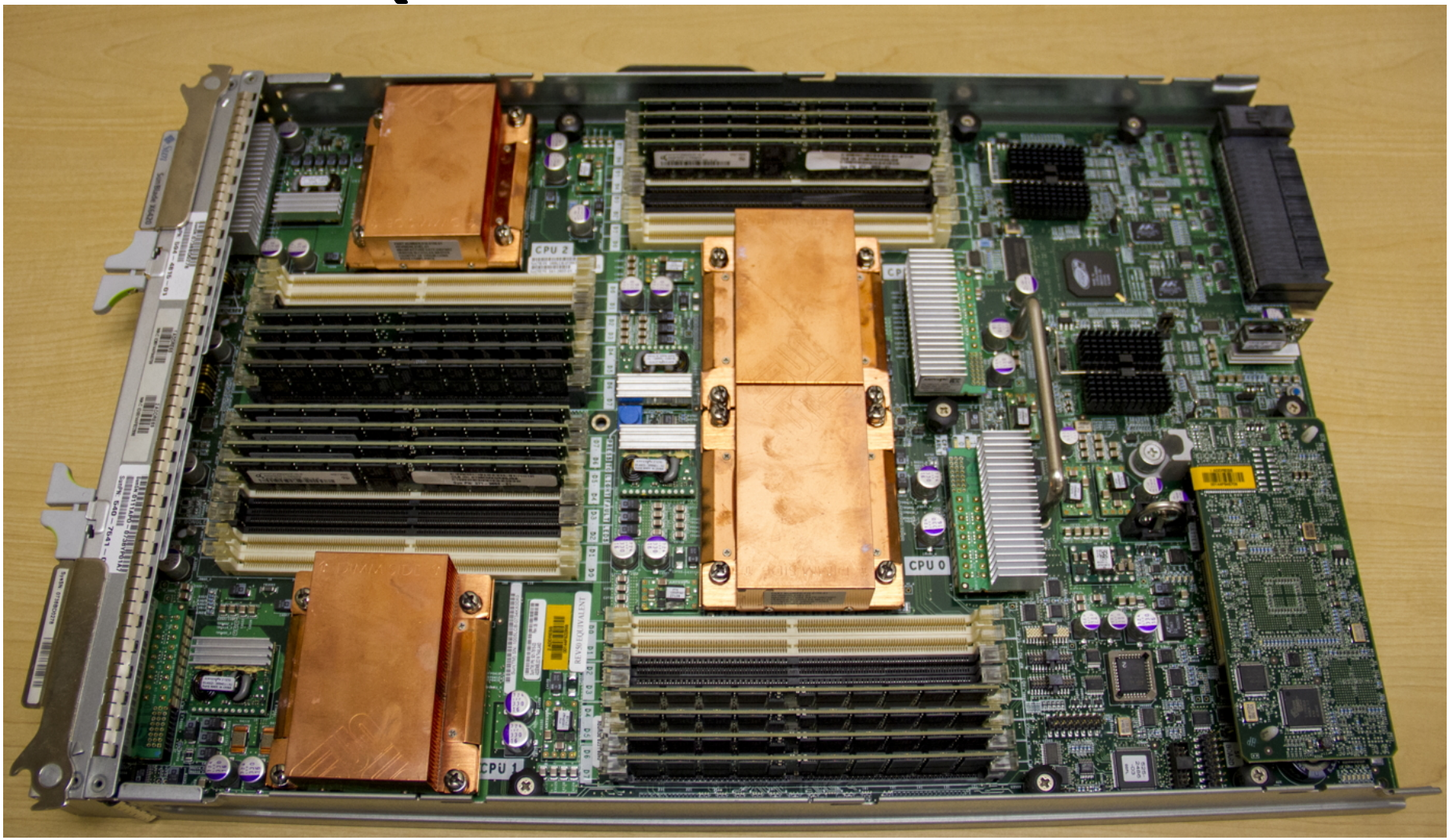
Computers when MPI was designed



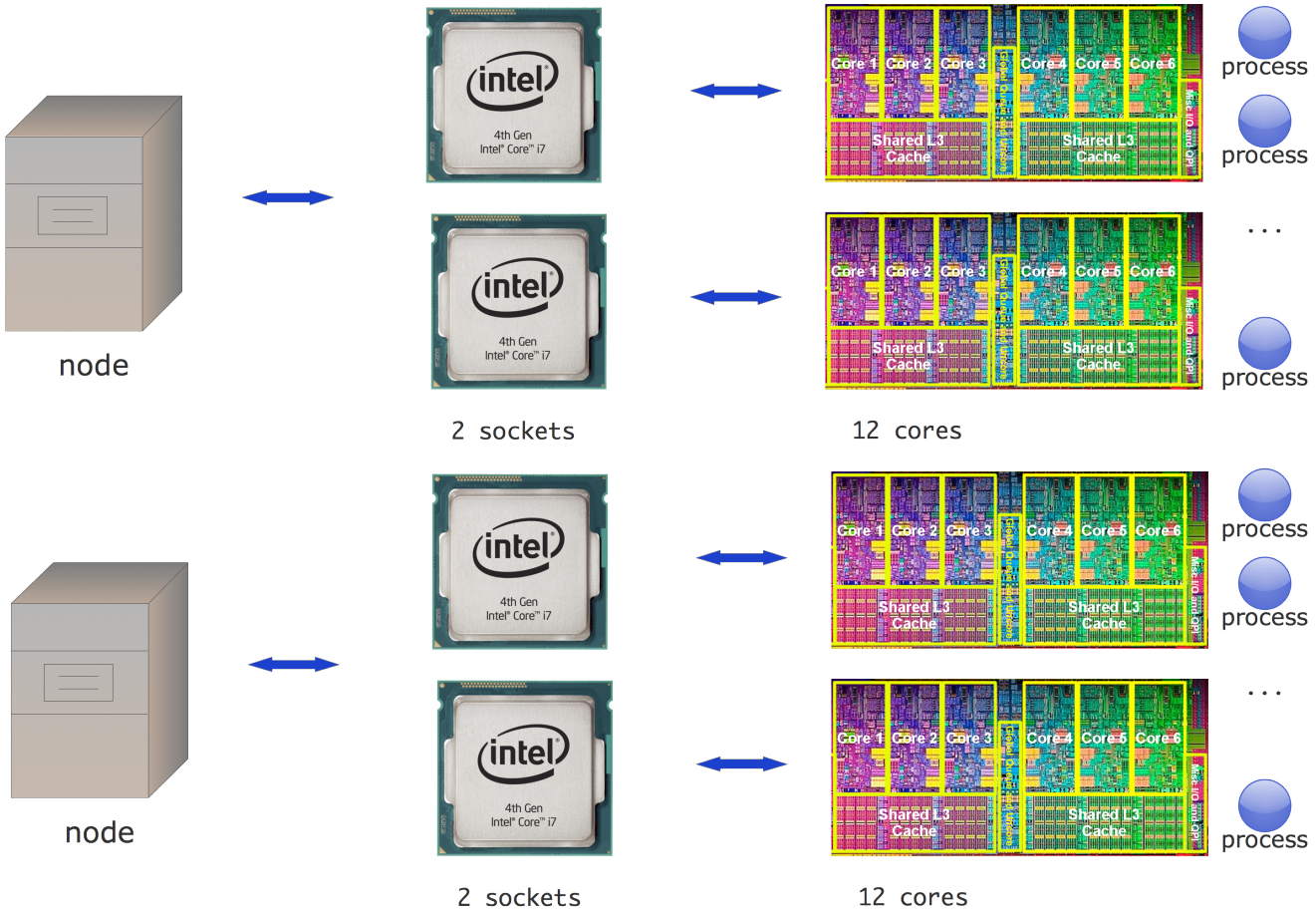
One processor and one process per node;
all communication goes through the network.

$\Rightarrow$ process model, no data sharing.

Quad socket node

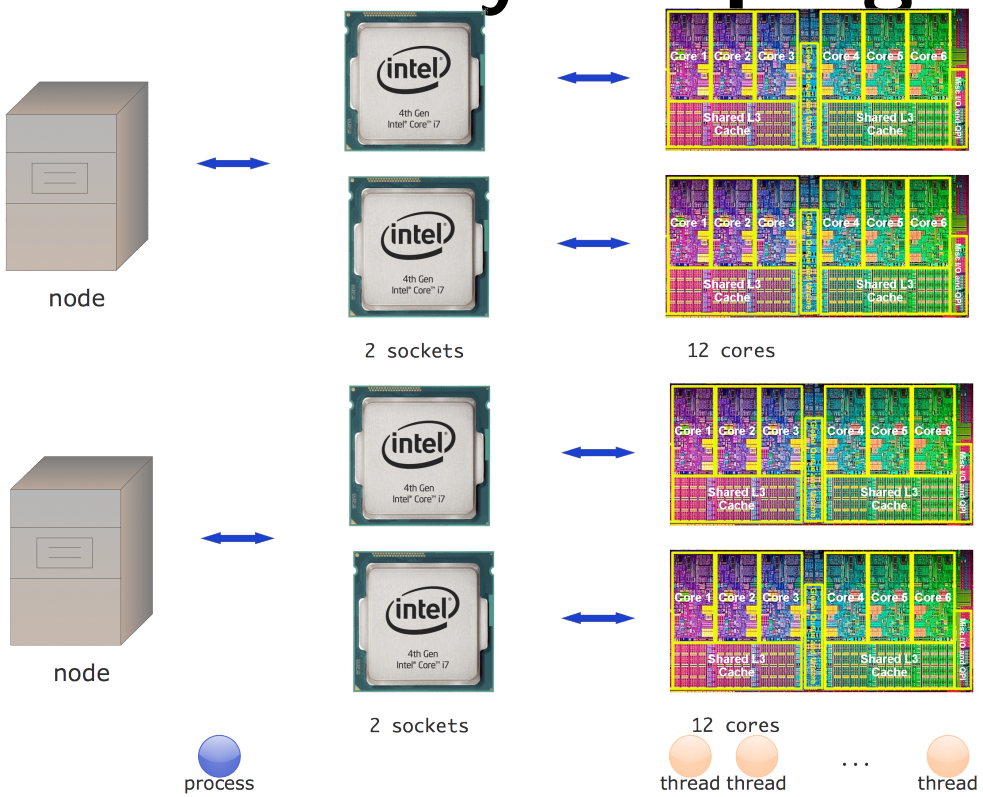


Pure MPI



A node has multiple sockets, each with multiple cores.
Pure MPI puts a process on each core: pretend shared memory doesn't exist.

Hybrid programming



Hybrid programming puts a process per node or per socket;
further parallelism comes from threading.
Not in this course. . .

MPI 'mindset'

MPI acts as if processes in their own grey box connected through a network cable.

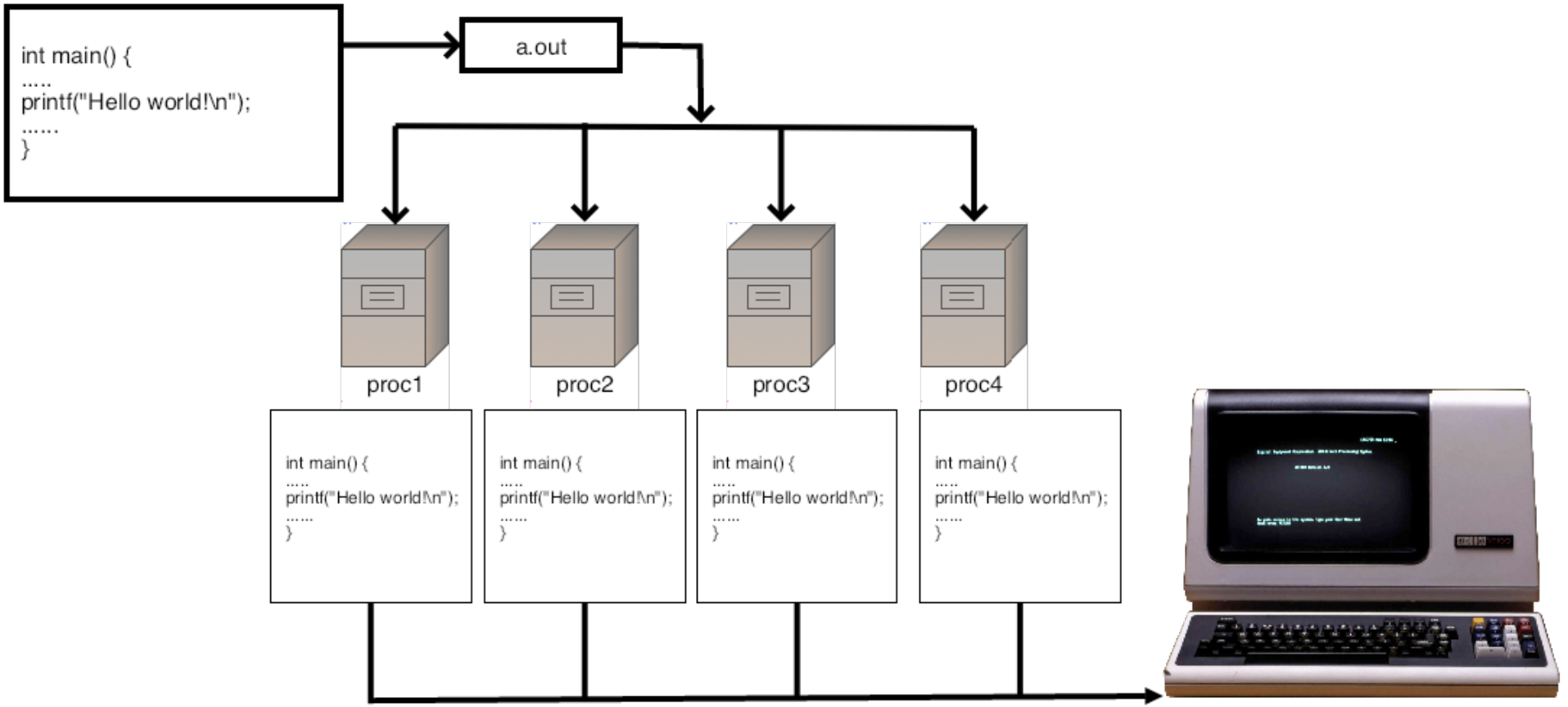
(Of course MPI implementations are optimized to use current architectures.)

Exercise 2 `hello`

Write a 'hello world' program, without any MPI in it, and run it in parallel with *`ibrun`* or equivalent. Explain the output.

1. In the directories `exercises-mpi-c/cxx/f08/p` do `make hello` to compile;
2. do `ibrun hello` to execute.

In a picture


int main() {
.....
printf("Hello world!\n");
.....
}
a.out
proc1
proc2
proc3
proc4
int main() {
.....
printf("Hello world!\n");
.....
}
int main() {
.....
printf("Hello world!\n");
.....
}
int main() {
.....
printf("Hello world!\n");
.....
}
int main() {
.....
printf("Hello world!\n");
.....
}

Terminology

'Processor' is ambiguous: is that a chip or one independent instruction processing unit?

- Node: a board with one or more sockets
- Socket: the processor chip
- Processor: we don't use that word
- Core: one instruction-stream processing unit
- Process: preferred terminology in talking about MPI.

Do I need a supercomputer?

- With `ibrun` and such, you start a bunch of processes that execute your MPI program.
- Does that mean that you need a cluster or a big multicore?
- No! You can start a large number of MPI processes, even on your laptop. The OS will use 'time slicing'.
- Of course it will not be very efficient...

Installing your own MPI

It is convenient to do MPI development on your laptop/desktop.

- Use a package manager
  Apple: brew or macports
  Linux: yum, aptget, ...
  Windows: I'll have to get back to you on that
- ... or download and compile from source `mpich.org`

1.3. We start learning MPI!

About library calls and bindings

Bindings

The standard defines interfaces to MPI from C and Fortran.
These look very similar; sometimes we will only show the C variant.

MPI can also be used from C++ and Python

MPI headers: C

You need an include file:

```
#include "mpi.h"
```

This defines all routines and constants.

MPI Init / Finalize

These calls need to be around the MPI part of your code:

```
MPI_Init(&argc,&argv); // zeros allowed
// your code
MPI_Finalize();
```

This is not a 'parallel region'.
Only internal library initialization:
allocate buffers, discover network, ...

Exercise 3 `hello`

Add the commands `MPI_Init` and `MPI_Finalize` to your code. Put three different print statements in your code: one before the init, one between init and finalize, and one after the finalize. Again explain the output.

Ranks

Process identification

Processes are organized in 'communicators':
groups of processes.

- For now only the 'world' communicator:
  all the processes you start out with.
- Processes can only interact if they are in the same communicator.
- Each process has a unique 'rank' wrt the communicator.

```
int MPI_Comm_size( MPI_Comm comm, int *nprocs )
int MPI_Comm_rank( MPI_Comm comm, int *procno )
```

Lowest number is always zero.

This is a logical view of parallelism: mapping to physical processors/cores is invisible here.

World communicator

For now, the communicator will be `MPI_COMM_WORLD`.

C:

```
MPI_Comm comm = MPI_COMM_WORLD;
```

Fortran:

```
Type(MPI_Comm) :: comm = MPI_COMM_WORLD
! legacy
Integer :: comm = MPI_COMM_WORLD
```

Python:

```
from mpi4py import MPI
comm = MPI.COMM_WORLD
```

MPL:

```
const mpl::communicator &comm_world =
    mpl::environment::comm_world();
```

MPI_Comm_size

Name	Param name	Explanation	C type	F type
MPI_Comm_size (				
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	size	number of processes in the group of comm	int*	INTEGER
	)			

MPI_Comm_rank

Name	Param name	Explanation	C type	F type
MPI_Comm_rank (				
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	rank	rank of the calling process in group of comm	int*	INTEGER
	)			

About routine signatures: C/C++

Signature:

```
int MPI_Comm_size(MPI_Comm comm,int *nprocs)
```

Use:

```
1 MPI_Comm comm = MPI_COMM_WORLD;
2 int nprocs;
3 int errorcode;
4 errorcode = MPI_Comm_size( comm,&nprocs );
```

But forget about that error code most of the time:

```
MPI_Comm_size( comm,&nprocs );
```

Exercise 4 `commrank`

Write a program where each process prints out a message reporting its number, and how many processes there are:

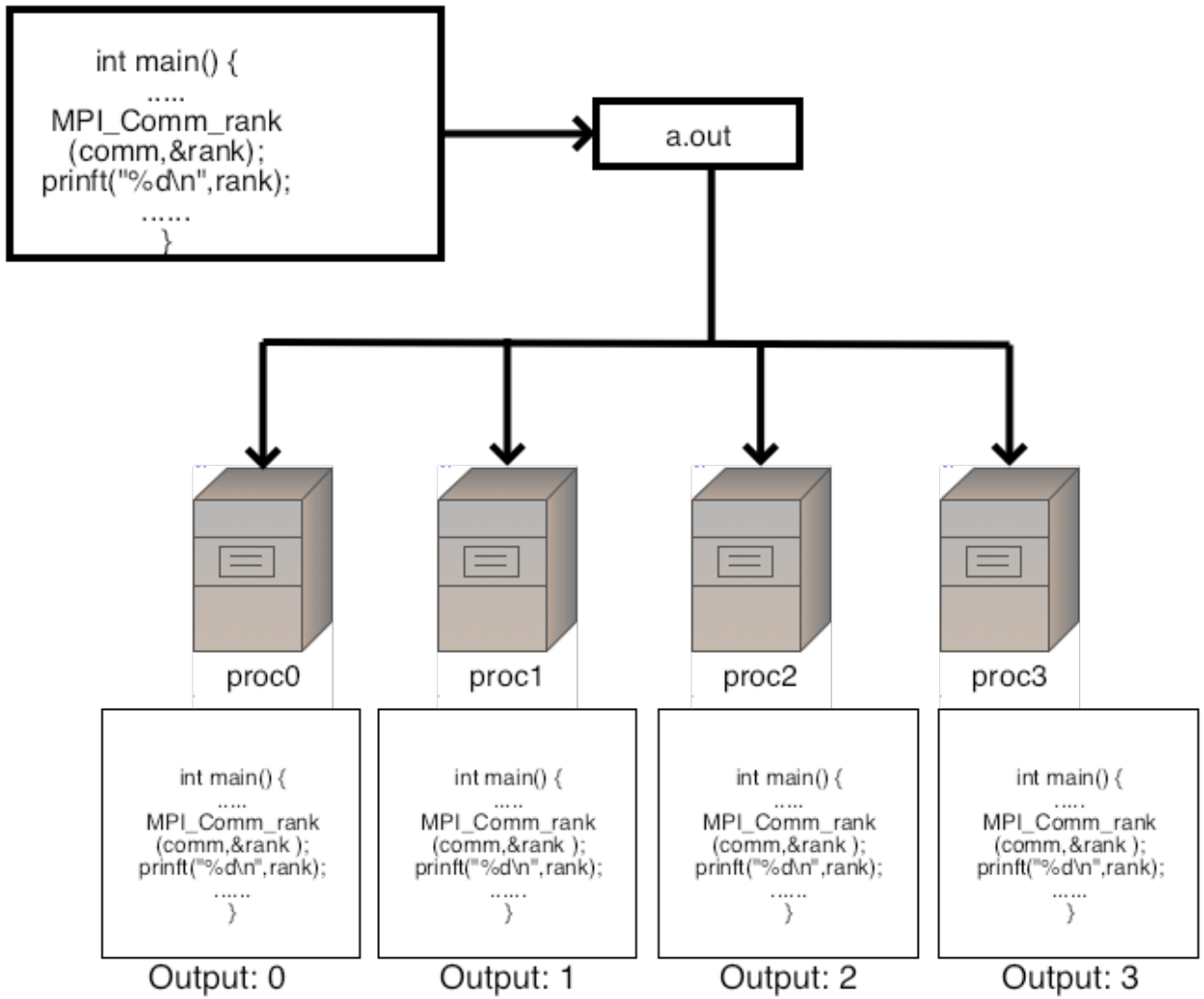
```
1 Hello from process 2 out of 5!
```

Write a second version of this program, where each process opens a unique file and writes to it.

Exercise 5 `commrank`

Write a program where only the process with number zero reports on how many processes there are in total.

Illustration



About errors

MPI routines invoke an error handler; (slide 392) Default action: abort

Every routine is defined as returning integer error code

- In C: function result.

  ```
  int ierr = MPI_Init(0,0);
  if (ierr!=MPI_SUCCESS) /* do something */
  ```

  But really: can often be ignored; is ignored in this course.

  ```
  MPI_Init(0,0);
  ```

- In Fortran: as optional (F08 only) parameter.
- MPL: throws exceptions
- In Python: throwing exception.

There's not a lot you can do with an error code:
very hard to recover from errors in parallel.
By default code bombs with (hopefully informative) message.

Processor name

MPI_Get_processor_name

Name	Param name	Explanation	C type	F type
MPI_Get_processor_name (				
	name	A unique specifier for the actual (as opposed to virtual) node.	char*	CHARACTER
	resultlen	Length (in printable characters) of the result returned in name	int*	INTEGER
	)			

Exercise 6

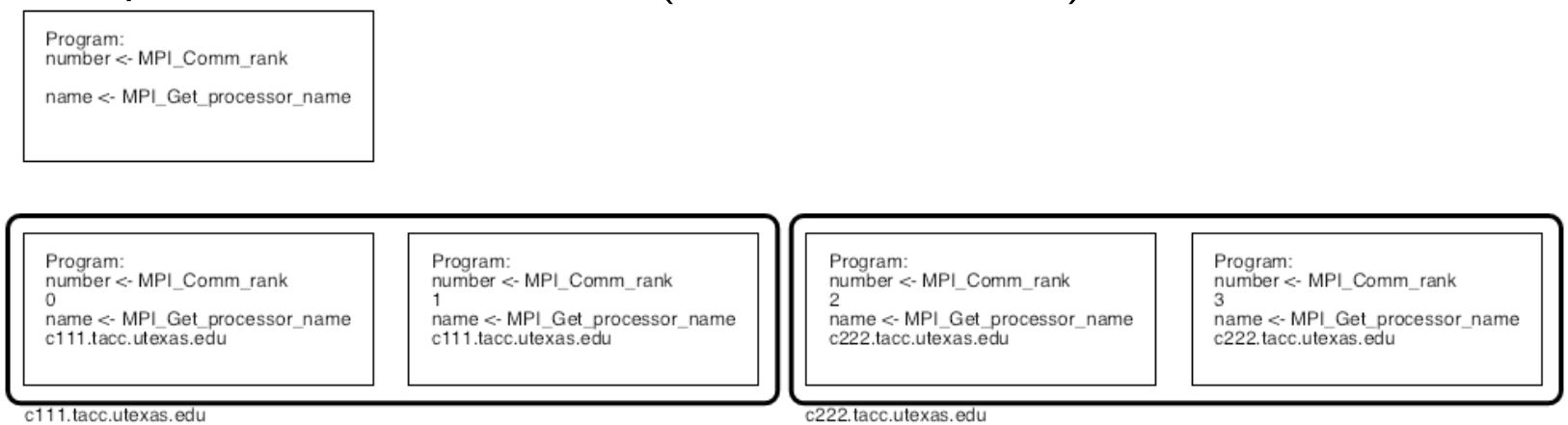
Use the command `MPI_Get_processor_name`. Confirm that you are able to run a program that uses two different nodes.

TACC nodes have a hostname cRRR-CNN, where RRR is the rack number, C is the chassis number in the rack, and NN is the node number within the chassis. Communication is faster inside a rack than between racks!

Go to `examples/mpi/xxx` and do `make procname`, then `ibrun procname`

In a picture

Four processes on two nodes (`idev -N 2 -n 4`)



Your first useful parallel program

Functional Parallelism

Parallelism by letting each process do a different thing.

Example: divide up a search space.

Each process knows its rank, so it can find its part of the search space.

Exercise 7 `prime`

Is the number $N = 2,000,000,111$ prime? Let each process test a disjoint set of integers, and print out any factor they find. You don't have to test all integers $< N$: any factor is at most $\sqrt{N} \approx 45,200$.

(Hint: `i%0` probably gives a runtime error.)

Can you find more than one solution strategy?

Exercise 8

Allocate on each process an array:

```
int my_ints[10];
```

and fill it so that process 0 has the integers $0 \cdots 9$, process 1 has $10 \cdots 19$, et cetera.

Let each process print out its array. It may be hard to print the output in a non-messy way.

1.4. Collectives

Overview

In this section you will learn 'collective' operations, that combine information from all processes.

Commands learned:

- `MPI_Bcast`, `MPI_Reduce`, `MPI_Gather`, `MPI_Scatter`
- *`MPI_All_...`* variants, *`MPI_....v`* variants
- `MPI_Barrier`, `MPI_Alltoall`, `MPI_Scan`

Technically

Routines can be 'collective on a communicator':

- They involve a communicator;
- if one process calls that routine, every process in that communicator needs to call it
- Mostly about combining data, but also opening shared files, declaring 'windows' for one-sided communication.

Collectives

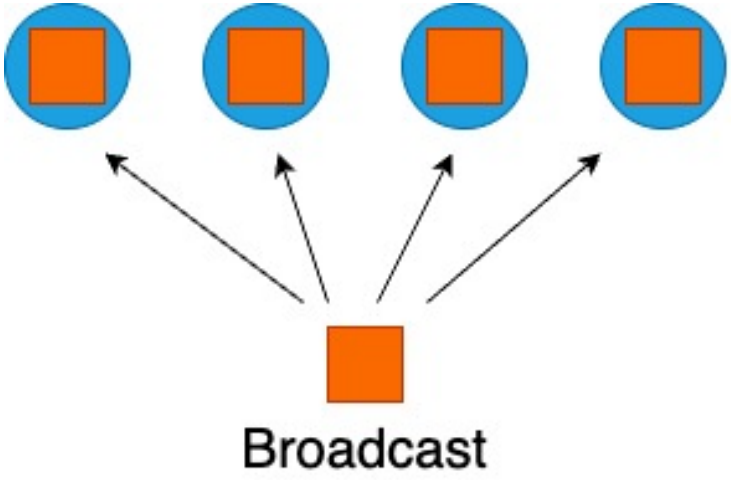
Gathering and spreading information:

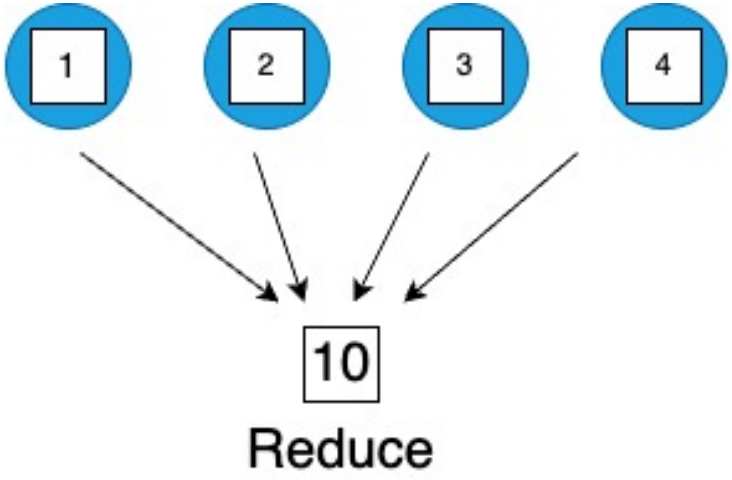
- One process has data, you want to spread it around.
- Every process has data, you want to bring it together;

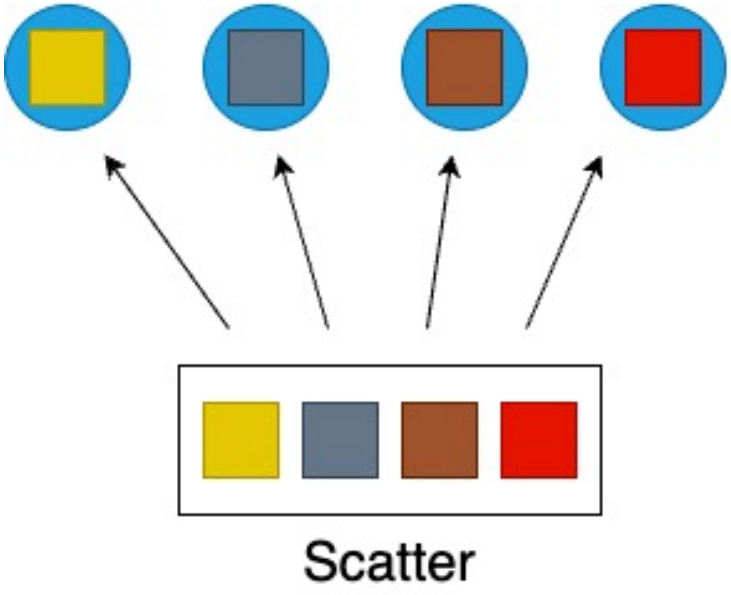
Root process: the one doing the collecting or disseminating.

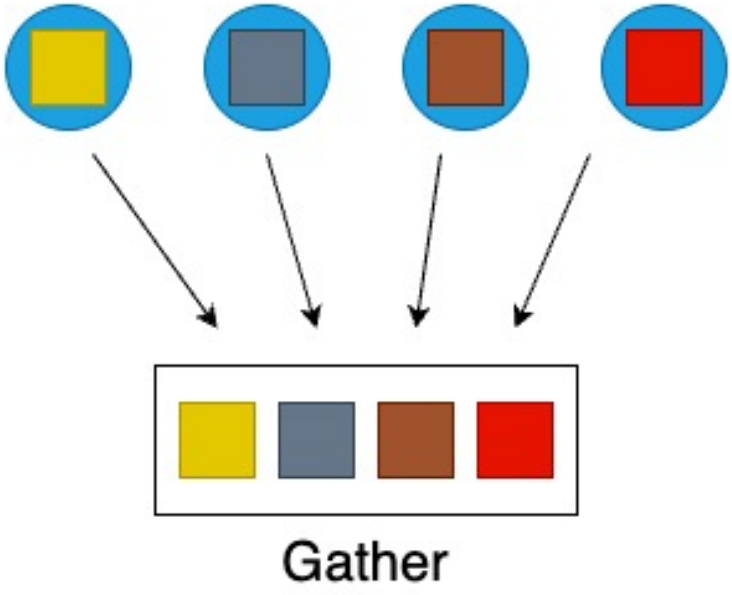
Basic cases:

- Send the same data to everyone: broadcast.
- Send individual data to each process: scatter.
- Collect data: gather.
- Collect data and compute some overall value (sum, max): reduction.


Broadcast


1
2
3
4
10
Reduce


Scatter


Gather

Exercise 9

How would you realize the following scenarios with MPI collectives?

1. Let each process compute a random number. You want to print the maximum of these numbers to your screen.
2. Each process computes a random number again. Each process wants to scale its number by the maximum computed in the previous question.
3. Let each process compute a random number. You want to print on what process the maximum value is computed.

Think about time and space complexity of your suggestions.

Allreduce: reduce-to-all

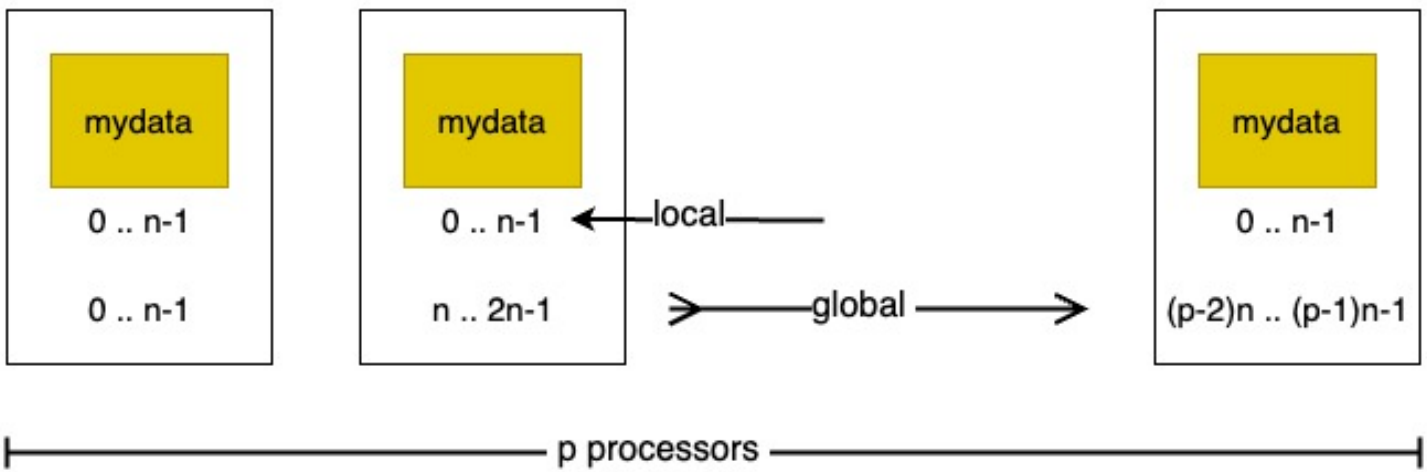
Case 2 in the exercise above contains a common case:
do a reduction, but everyone needs the result.

- `MPI_Allreduce` does the same as:
  `MPI_Reduce` (reduction) followed by `MPI_Bcast` (broadcast)
- Same running time as either, half of reduce-followed-by-broadcast
  (no proof given here)
- Common use case; see next

Motivation for allreduce

Example: normalizing a vector

$$y \leftarrow x/\|x\|$$



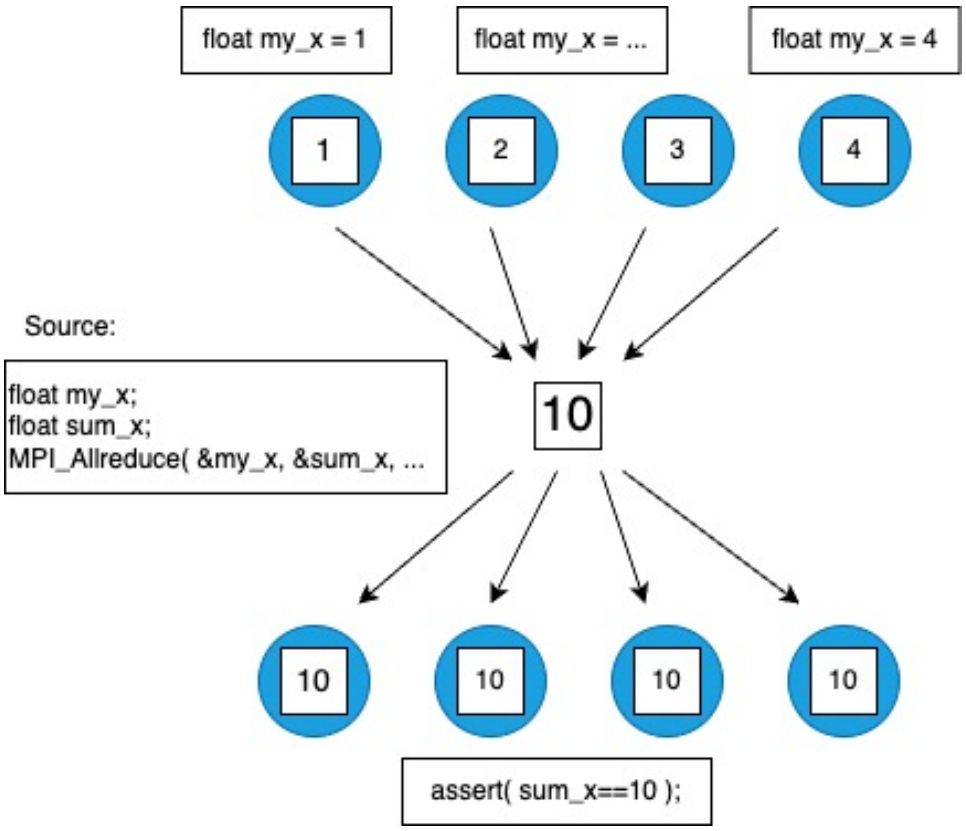
Norm needs to be known everywhere.
How do you compute it?

Structure of allreduce

- Vectors x,y are distributed: every process has certain elements
- The norm calculation is an all-reduce: every process gets same value
- Every process scales its part of the vector.
- Question: what kind of reduction do you use for an inf-norm? One-norm? Two-norm?

Conceptual picture

Recall SPMD: every process has the input and output variable



(What actually happens is a different story!)

Exercise 10

How many objections can you come up to this strategy:

1. Gather vector x on some root process;
2. Compute the reduction on that root;
3. Construct the scaled vector on the root;
4. Scatter the scaled vector.

Exercise 11

Standard deviation:

$$\sigma = \sqrt{\frac{1}{N}\sum_{i}^{N}(x_i - \mu)} \qquad \text{where} \qquad \mu = \frac{\sum_i^N x_i}{N}$$

and assume that every process stores just one x_i value;
the standard deviation is printed out, and not further used in computations.

Argue that there is one all-reduce and one reduce with a root.

MPI_Allreduce

Name	Param name	Explanation	C type	F type
MPI_Allreduce (				
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	recvbuf	starting address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in send buffer	int	INTEGER
	datatype	datatype of elements of send buffer	MPI_Datatype	TYPE(MPI_Datatype)
	op	operation	MPI_Op	TYPE(MPI_Op)
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Allreduce arguments

Scalar buffer

```
1 double in_x,out_x;
2 MPI_Allreduce( &in_x,&out_x,1,MPI_DOUBLE, ... )
```

Note: $\mathrm{count} = 1$ because reduction on single scalar.

General buffer:

```
1 double in_x[5],out_x[5];
2 MPI_Allreduce( in_x,out_x,5,MPI_DOUBLE, ... )
```

C++

```
1 vector<double> x(5);
2 MPI_Comm_allreduce( x.data(), ...,5,... )
```

- `MPI_Datatype` is `MPI_INT`, `MPI_FLOAT` (for C), `MPI_REAL8` (for Fortran) et cetera.
- `MPI_Op` is `MPI_SUM`, `MPI_MAX` et cetera.

Elementary datatypes

C	Fortran	Python	meaning
MPI_CHAR	MPI_CHARACTER		only for text
MPI_SHORT	MPI_BYTE		8 bits
MPI_INT	MPI_INTEGER		like the C/F types
MPI_FLOAT	MPI_REAL		
MPI_DOUBLE	MPI_DOUBLE_PRECISION	*MPI.DOUBLE*	
	MPI_COMPLEX		
	MPI_LOGICAL		
unsigned	extensions		
			MPI_Aint
			MPI_Offset

A bunch more.

Reduction operators

MPI type	Python	meaning	applies to
	MPI.Op		
MPI_MAX	*MPI.MAX*	maximum	integer, floating point
MPI_MIN	*MPI.MIN*	minimum	
MPI_SUM	*MPI.SUM*	sum	integer, floating point, complex, multilanguage types
MPI_PROD	*MPI.PROC*	product	
MPI_REPLACE	*MPI.REPLACE*	overwrite	
MPI_NO_OP	*MPI.OP_NULL*	no change	
MPI_LAND	*MPI.LAND*	logical and	C integer, logical
MPI_LOR	*MPI.LOR*	logical or	
MPI_LXOR	*MPI.LXOR*	logical xor	
MPI_BAND	*MPI.BAND*	bitwise and	integer, byte, multilanguage types
MPI_BOR	*MPI.BOR*	bitwise or	
MPI_BXOR	*MPI.BXOR*	bitwise xor	
MPI_MAXLOC	*MPI.MAXLOC*	max value and location	MPI_DOUBLE_INT and such
MPI_MINLOC	*MPI.MINLOC*	min value and location	

Exercise 12 `randommax`

Let each process compute a random number, and compute the sum of these numbers using the `MPI_Allreduce` routine.

$$\xi = \sum_i x_i$$

Each process then scales its value by this sum.

$$x_i' \leftarrow x_i / \xi$$

Compute the sum of the scaled numbers

$$\xi' = \sum_i x_i'$$

and check that it is 1.

Buffers

Buffers in C

General principle: buffer argument is address in memory of the data.

- Buffer is void pointer:
- write `&x` or `(void*)&x` for scalar
- write `x` or `(void*)x` for array

```
1 // scalar argument
2 double x;
3 MPI_Bcast( &x, .... );
4 // array argument
5 double x[5];
6 MPI_Bcast( x, .... );
```

Large buffers

MPI buffersize is specified as `int`. In 1993 that was enough.

As of MPI-4 a buffer can be longer than 2^{31} elements.

- Use `MPI_Count` for count
- In C: use `MPI_Reduce_c`
- in Fortran: polymorphism means no change to the call.
- MPL: `long int` and `size_t` supported for layouts.

```
size_t ibuffersize = (size_t)(1<<30);
double *indata,*outdata;
indata = (double*) malloc( ibuffersize*sizeof(double) );
outdata = (double*) malloc( ibuffersize*sizeof(double) );
MPI_Count buffersize = ibuffersize;
MPI_Allreduce_c(indata,outdata,buffersize,
                MPI_DOUBLE,MPI_SUM,MPI_COMM_WORLD);
```

Exercise 13

Extend exercise 12 to letting each process have an array.

Reduction to single process

Reduce with a single root process: great for printing out summary information at the end of your job.

Can you think of a case were a rooted reduce is appropriate or unavoidable? (Hint: tree)

MPI_Reduce

Name	Param name	Explanation	C type	F type
MPI_Reduce	(			
	sendbuf	address of send buffer	const void*	TYPE(*), DIMENSION(..)
	recvbuf	address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in send buffer	int	INTEGER
	datatype	datatype of elements of send buffer	MPI_Datatype	TYPE(MPI_Datatype)
	op	reduce operation	MPI_Op	TYPE(MPI_Op)
	root	rank of root process	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Reduction to root

- Buffers: *sendbuf*, *recvbuf* are ordinary variables/arrays.
- Every process has data in its *sendbuf*,
  Root combines it in *recvbuf* (ignored on non-root processes).
- `count` is number of items in the buffer: 1 for scalar.
- `MPI_Op` is `MPI_SUM`, `MPI_MAX` et cetera.

In-place operations

```
1 // allreduceinplace.c
2 for (int irand=0; irand<nrandoms; irand++)
3         myrandoms[irand] = (float) rand()/(float)RAND_MAX;
4 // add all the random variables together
5 MPI_Allreduce(MPI_IN_PLACE,myrandoms,
6                     nrandoms,MPI_FLOAT,MPI_SUM,comm);
```

MPI_Bcast

Name	Param name	Explanation	C type	F type
MPI_Bcast (				
	buffer	starting address of buffer	void*	TYPE(*), DIMENSION(..)
	count	number of entries in buffer	int	INTEGER
	datatype	datatype of buffer	MPI_Datatype	TYPE(MPI_Datatype)
	root	rank of broadcast root	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Broadcast

- All processes call with the same argument list
- *root* is the rank of the process doing the broadcast
- Each process allocates buffer space;
  root explicitly fills in values,
  all others receive values through broadcast call.
- Datatype is `MPI_FLOAT`, `MPI_INT` et cetera, different between C/Fortran.
- *comm* is usually `MPI_COMM_WORLD`

Gauss-Jordan elimination

https://youtu.be/aQYuwatlWME

Exercise 14 `jordan`

The *Gauss-Jordan algorithm* for solving a linear system with a matrix A (or computing its inverse) runs as follows:

for pivot $k = 1, \ldots, n$
    let the vector of scalings $\ell_i^{(k)} = A_{ik}/A_{kk}$
    for row $r \neq k$
        for column $c = 1, \ldots, n$
            $A_{rc} \leftarrow A_{rc} - \ell_r^{(k)} A_{kc}$

where we ignore the update of the righthand side, or the formation of the inverse.

Let a matrix be distributed with each process storing one column. Implement the Gauss-Jordan algorithm as a series of broadcasts: in iteration k process k computes and broadcasts the scaling vector $\{\ell_i^{(k)}\}_i$. Replicate the right-hand side on all processors.

Exercise (optional) 15

Bonus exercise: can you extend your program to have multiple columns per process?

Scan

Scan

Scan or 'parallel prefix': reduction with partial results

- Useful for indexing operations:
- Each process has an array of n_p elements;
- My first element has global number $\sum_{q<p} n_q$.
- Two variants: `MPI_Scan` inclusive, and `MPI_Exscan` exclusive.

In vs Exclusive

process :	0	1	2	$\cdots$	$p-1$
data :	x_0	x_1	x_2	$\cdots$	x_{p-1}
inclusive :	x_0	$x_0 \oplus x_1$	$x_0 \oplus x_1 \oplus x_2$	$\cdots$	$\oplus_{i=0}^{p-1} x_i$
exclusive :	unchanged	x_0	$x_0 \oplus x_1$	$\cdots$	$\oplus_{i=0}^{p-2} x_i$

MPI_Scan

Name	Param name	Explanation	C type	F type
MPI_Scan (				
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	recvbuf	starting address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in input buffer	int	INTEGER
	datatype	datatype of elements of input buffer	MPI_Datatype	TYPE(MPI_Datatype)
	op	operation	MPI_Op	TYPE(MPI_Op)
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

MPI_Exscan

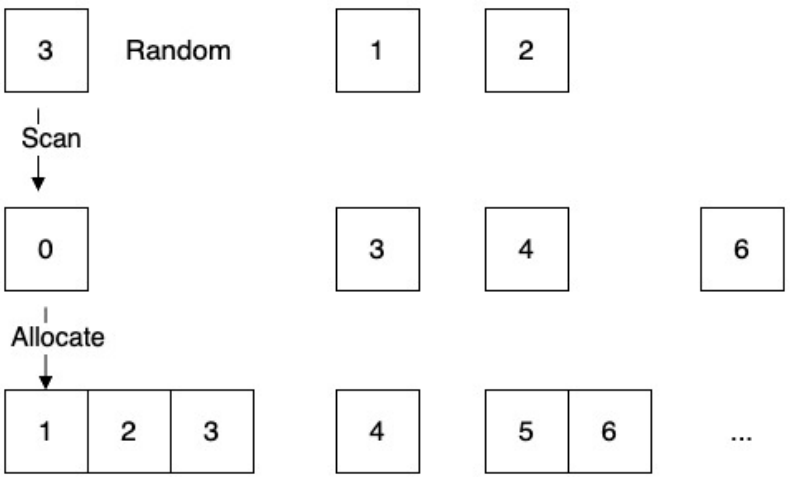
Name	Param name	Explanation	C type	F type
MPI_Exscan	(			
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	recvbuf	starting address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in input buffer	int	INTEGER
	datatype	datatype of elements of input buffer	MPI_Datatype	TYPE(MPI_Datatype)
	op	operation	MPI_Op	TYPE(MPI_Op)
	comm	intra-communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Exercise 16 `scangather`

- Let each process compute a random value n_{local}, and allocate an array of that length. Define

$$N = \sum n_{\text{local}}$$

- Fill the array with consecutive integers, so that all local arrays, laid end-to-end, contain the numbers $0 \cdots N-1$.



Gather/Scatter, Barrier, and others

MPI_Gather

Name	Param name	Explanation	C type	F type
MPI_Gather	(			
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	sendcount	number of elements in send buffer	int	INTEGER
	sendtype	datatype of send buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	recvbuf	address of receive buffer	void*	TYPE(*), DIMENSION(..)
	recvcount	number of elements for any single receive	int	INTEGER
	recvtype	datatype of recv buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	root	rank of receiving process	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

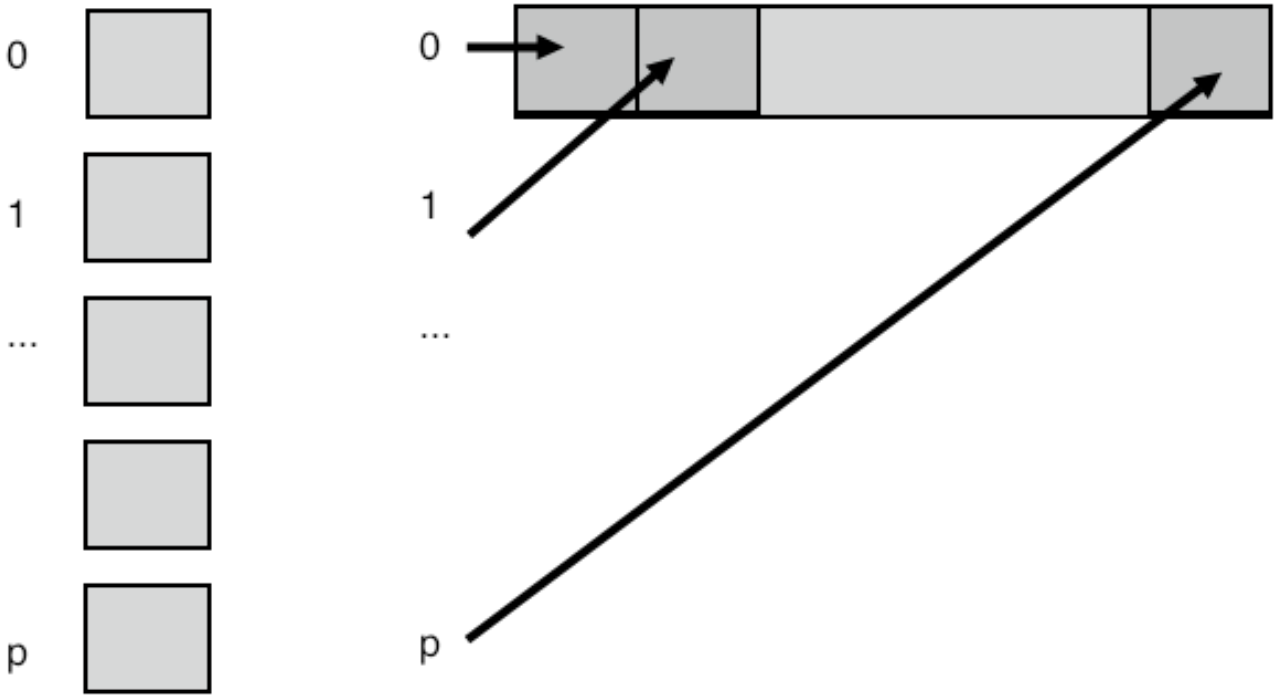
MPI_Scatter

Name	Param name	Explanation	C type	F type
MPI_Scatter (				
	sendbuf	address of send buffer	const void*	TYPE(*), DIMENSION(..)
	sendcount	number of elements sent to each process	int	INTEGER
	sendtype	datatype of send buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	recvbuf	address of receive buffer	void*	TYPE(*), DIMENSION(..)
	recvcount	number of elements in receive buffer	int	INTEGER
	recvtype	datatype of receive buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	root	rank of sending process	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Gather/Scatter

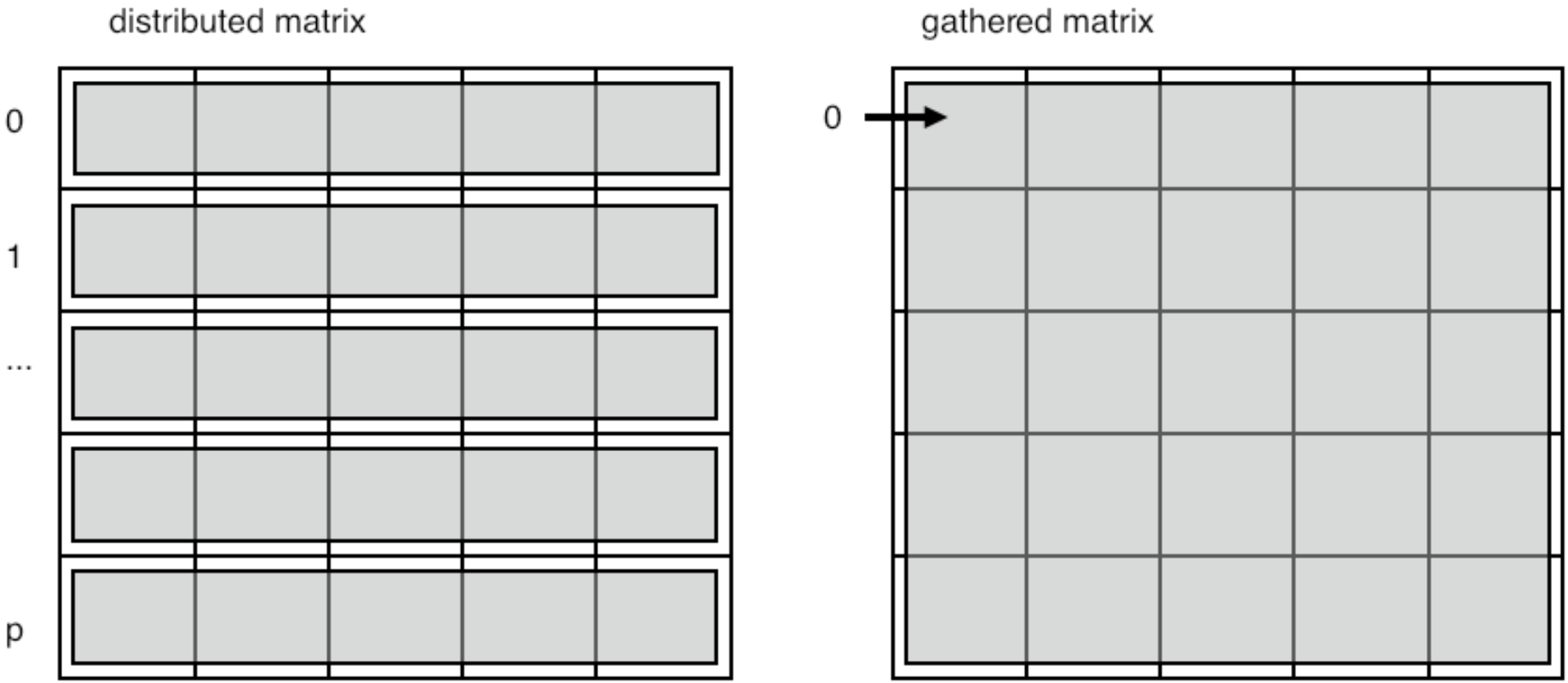
- Compare buffers to reduce
- Scatter: the `sendcount` / Gather: the `recvcount`:
  this is not, as you might expect, the total length of the buffer;
  instead, it is the amount of data to/from each process.

Gather pictured



Popular application of gather

Matrix is constructed distributed, but needs to be brought to one process:

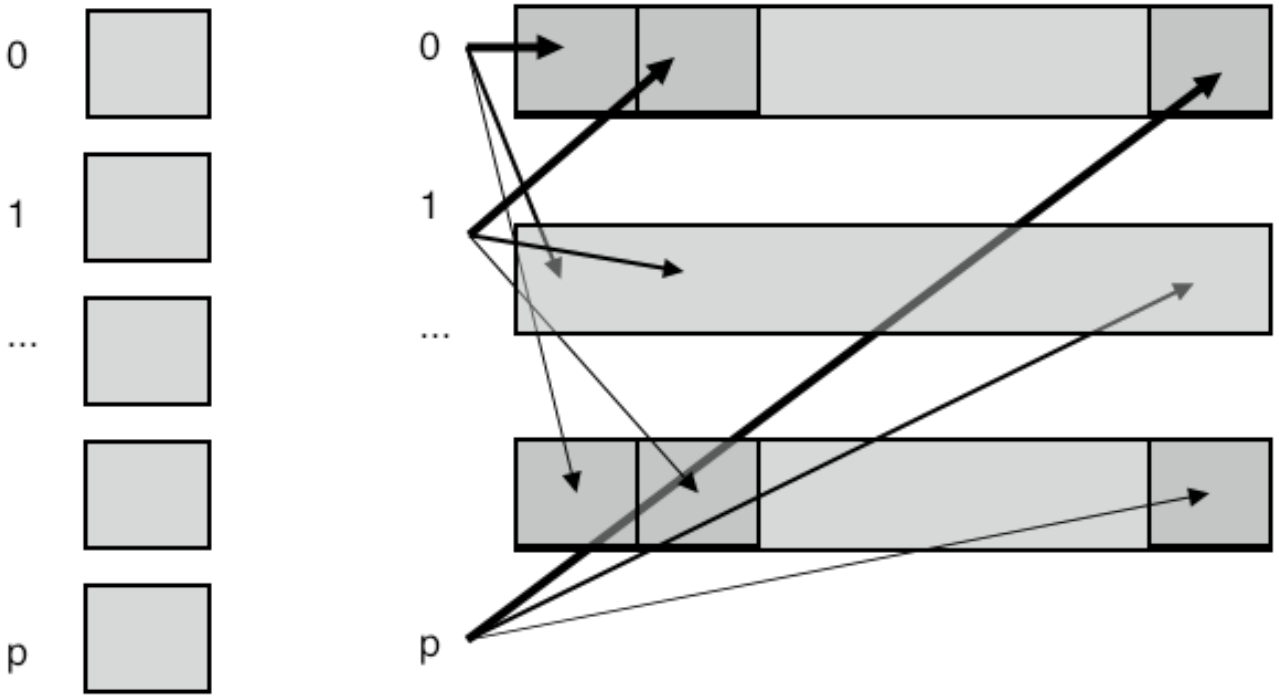


This is not efficient in time or space. Do this only when strictly necessary.
Remember SPMD: try to keep everything symmetrically parallel.

MPI_Allgather

Name	Param name	Explanation	C type	F type
MPI_Allgather (				
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	sendcount	number of elements in send buffer	int	INTEGER
	sendtype	datatype of send buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	recvbuf	address of receive buffer	void*	TYPE(*), DIMENSION(..)
	recvcount	number of elements received from any process	int	INTEGER
	recvtype	datatype of receive buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Allgather pictured



V-type collectives

- Gather/scatter but with individual sizes
- Requires displacement in the gather/scatter buffer

MPI_Gatherv

Name	Param name	Explanation	C type	F type
MPI_Gatherv (				
	sendbuf	starting address of send buffer	const void*	TYPE(*), DIMENSION(..)
	sendcount	number of elements in send buffer	int	INTEGER
	sendtype	datatype of send buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	recvbuf	address of receive buffer	void*	TYPE(*), DIMENSION(..)
	recvcounts	non-negative integer array (of length group size) containing the number of elements that are received from each process	const int[]	INTEGER(*)
	displs	integer array (of length group size). Entry i specifies the displacement relative to recvbuf at which to place the incoming data from process i	const int[]	INTEGER(*)
	recvtype	datatype of recv buffer elements	MPI_Datatype	TYPE(MPI_Datatype)
	root	rank of receiving process	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Exercise 17 `scangather`

Take the code from exercise 16 and extend it to gather all local buffers onto rank zero. Since the local arrays are of differing lengths, this requires `MPI_Gatherv`.

How do you construct the lengths and displacements arrays?

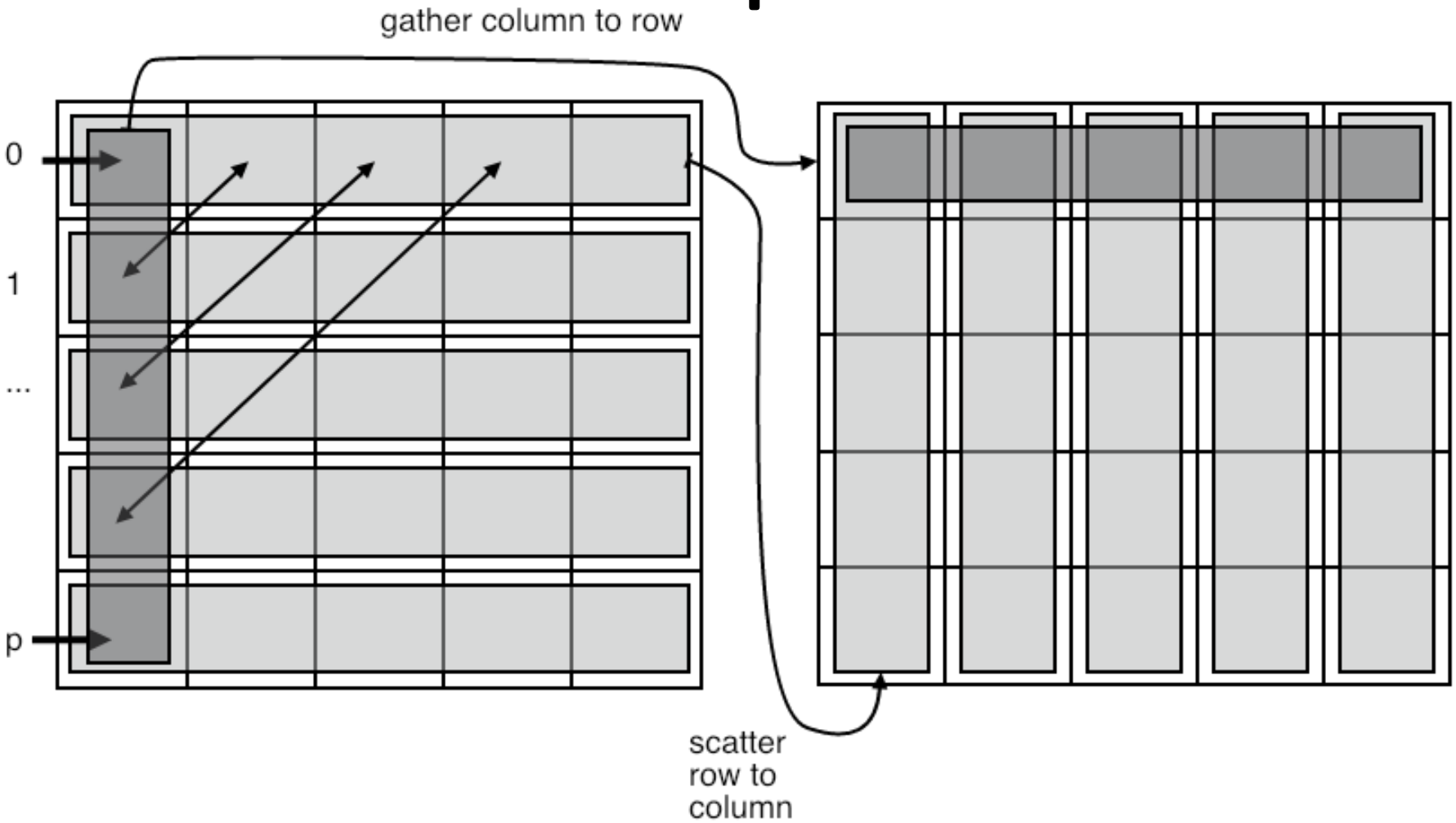
Review 1

- An `MPI_Scatter` call puts the same data on each process
- A call to `MPI_Scatter` only updates on the non-root processes; it does not alter any array on the root process.

All-to-all

- Every process does a scatter;
- (equivalently: every process gather)
- each individual data, but amounts are identical
- Example: data transposition in FFT

Data transposition



All-to-allv

- Every process does a scatter or gather;
- each individual data and individual amounts.
- Example: radix sort by least-significant digit.

Radix sort

Sort 4 numbers on two processes:

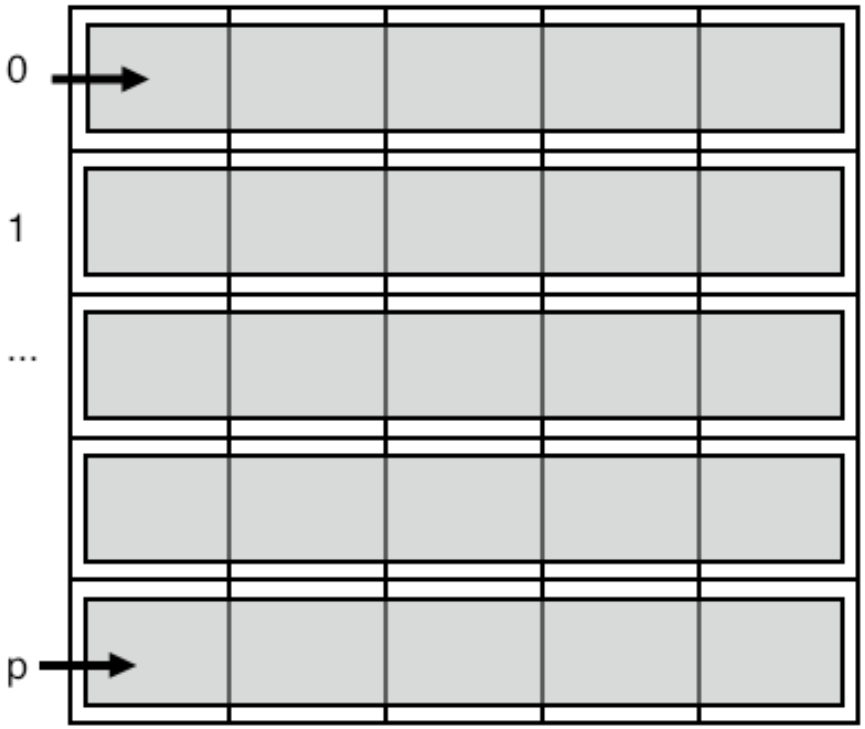
	proc0		proc1		
array	2	5	7	1	
binary	010	101	111	001	
stage 1					
last digit	0	1	1	1	
	(this serves as bin number)				
sorted	010		101	111	001
stage 2					
next digit	1		0	1	0
	(this serves as bin number)				
sorted	101	001	010	111	
stage 3					
next digit	1	0	0	1	
	(this serves as bin number)				
sorted	001	010	101	111	
decimal	1	2	5	7	

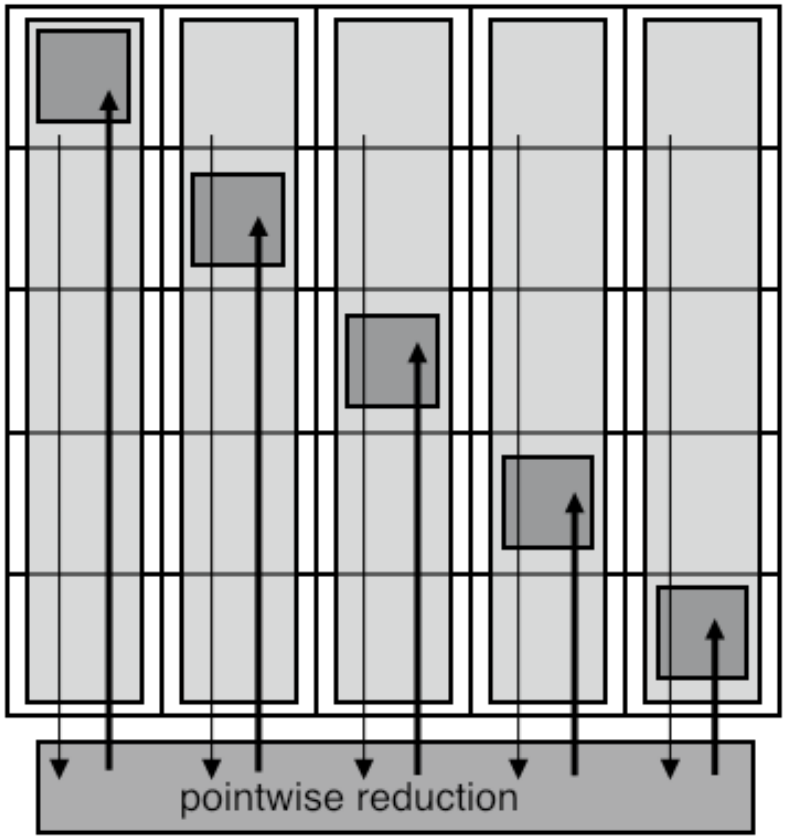
Reduce-scatter

- Pointwise reduction (one element per process) followed by scatter
- Somewhat related to all-to-all: data transpose but reduced information, rather than gathered.
- Applications in both sparse and dense matrix-vector product.

Example: sparse matrix setup

Example: each process knows who to send to,
all-to-all gives information how many messages to expect
reduce-scatter leaves only relevant information





Barrier

```
int MPI_Barrier( MPI_Comm comm )
```

- Synchronize processes:
- each process waits at the barrier until all processes have reached the barrier
- **This routine is almost never needed**:
  collectives are already a barrier of sorts, two-sided communication is a local synchronization
- One conceivable use: timing

User-defined operators

MPI Operators

Define your own reduction operator

- Define operator between partial result and new operand

```
typedef void MPI_User_function
    ( void *invec, void *inoutvec, int *len,
      MPI_Datatype *datatype);
```

- Don't forget to free:

```
int MPI_Op_free(MPI_Op *op)
```

- Make your own reduction scheme `MPI_Reduce_local`

MPI_Op_create

Name	Param name	Explanation	C type	F type
MPI_Op_create (				
	user_fn	user defined function	MPI_User_function*	PROCEDURE (MPI_User_function)
	commute	true if commutative; false otherwise.	int	LOGICAL
	op	operation	MPI_Op*	TYPE(MPI_Op)
	)			

Example

Smallest nonzero:

```
1  // reductpositive.c
2  void reduce_without_zero(void *in,void *inout,int *len,MPI_Datatype *type) {
3  // r is the already reduced value, n is the new value
4    int n = *(int*)in, r = *(int*)inout;
5    int m;
6    if (n==0) { // new value is zero: keep r
7      m = r;
8    } else if (r==0) {
9      m = n;
10   } else if (n<r) { // new value is less but not zero: use n
11     m = n;
12   } else { // new value is more: use r
13     m = r;
14   };
15 // return the new value
16   *(int*)inout = m;
17 }
```

Review 2

The $\|\cdot\|_2$ norm (sum of squares) needs a custom operator.

Exercise 18 `onenorm`

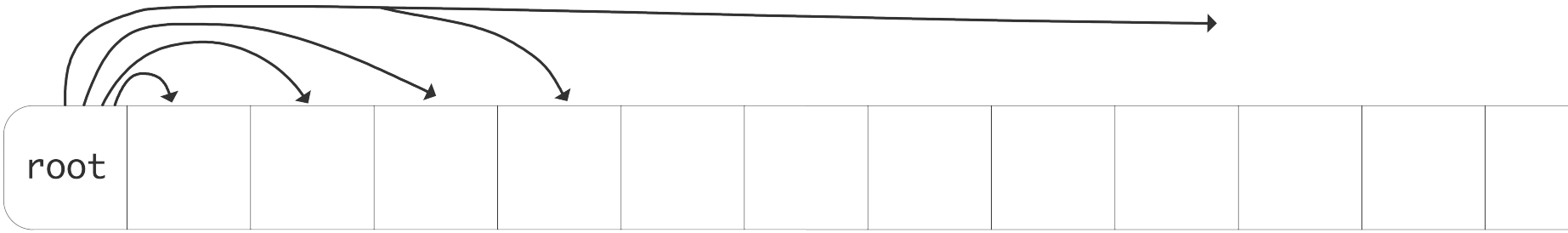
Write the reduction function to implement the *one-norm* of a vector:

$$\|x\|_1 \equiv \sum_i |x_i|.$$

Performance of collectives

Naive realization of collectives

Broadcast:



Single message:

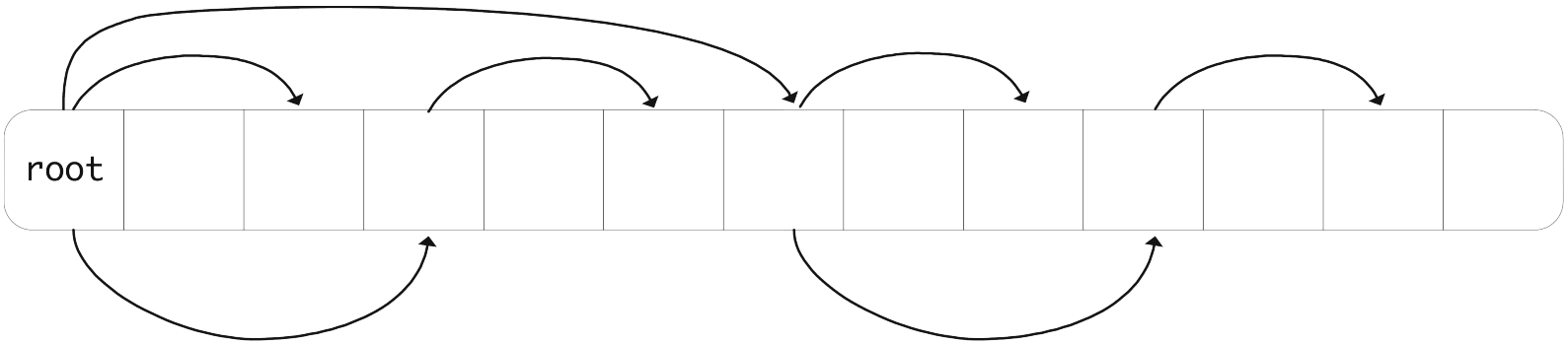
$$\alpha = \text{message startup} \approx 10^{-6}s, \qquad \beta = \text{time per word} \approx 10^{-9}s$$

- Time for message of n words:

$$\alpha + \beta n$$

- Time for collective? Can you improve on that?

Better implementation of collective



- What is the running time now?
- Can you come up with lower bounds on the α, β terms? Are these achieved here?

Implementation of Reduce

	$t = 1$	$t = 2$	$t = 3$
p_0	$x_0^{(0)}, x_1^{(0)}, x_2^{(0)}, x_3^{(0)}$	$x_0^{(0:1)}, x_1^{(0:1)}, x_2^{(0:1)}, x_3^{(0:1)}$	$x_0^{(0:3)}, x_1^{(0:3)}, x_2^{(0:3)}, x_3^{(0:3)}$
p_1	$x_0^{(1)} \uparrow, x_1^{(1)} \uparrow, x_2^{(1)} \uparrow, x_3^{(1)} \uparrow$		
p_2	$x_0^{(2)}, x_1^{(2)}, x_2^{(2)}, x_3^{(2)}$	$x_0^{(2:3)} \uparrow, x_1^{(2:3)} \uparrow, x_2^{(2:3)} \uparrow, x_3^{(2:3)} \uparrow$	
p_3	$x_0^{(3)} \uparrow, x_1^{(3)} \uparrow, x_2^{(3)} \uparrow, x_3^{(3)} \uparrow$		

Implementation of Allreduce

	$t = 1$	$t = 2$	$t = 3$
p_0	$x_0^{(0)} \downarrow, x_1^{(0)} \downarrow, x_2^{(0)} \downarrow, x_3^{(0)} \downarrow$	$x_0^{(0:1)} \downarrow\downarrow, x_1^{(0:1)} \downarrow\downarrow, x_2^{(0:1)} \downarrow\downarrow, x_3^{(0:1)} \downarrow\downarrow$	$x_0^{(0:3)}, x_1^{(0:3)}, x_2^{(0:3)}, x_3^{(}$
p_1	$x_0^{(1)} \uparrow, x_1^{(1)} \uparrow, x_2^{(1)} \uparrow, x_3^{(1)} \uparrow$	$x_0^{(0:1)} \downarrow\downarrow, x_1^{(0:1)} \downarrow\downarrow, x_2^{(0:1)} \downarrow\downarrow, x_3^{(0:1)} \downarrow\downarrow$	$x_0^{(0:3)}, x_1^{(0:3)}, x_2^{(0:3)}, x_3^{(}$
p_2	$x_0^{(2)} \downarrow, x_1^{(2)} \downarrow, x_2^{(2)} \downarrow, x_3^{(2)} \downarrow$	$x_0^{(2:3)} \uparrow\uparrow, x_1^{(2:3)} \uparrow\uparrow, x_2^{(2:3)} \uparrow\uparrow, x_3^{(2:3)} \uparrow\uparrow$	$x_0^{(0:3)}, x_1^{(0:3)}, x_2^{(0:3)}, x_3^{(}$
p_3	$x_0^{(3)} \uparrow, x_1^{(3)} \uparrow, x_2^{(3)} \uparrow, x_3^{(3)} \uparrow$	$x_0^{(2:3)} \uparrow\uparrow, x_1^{(2:3)} \uparrow\uparrow, x_2^{(2:3)} \uparrow\uparrow, x_3^{(2:3)} \uparrow\uparrow$	$x_0^{(0:3)}, x_1^{(0:3)}, x_2^{(0:3)}, x_3^{(}$

1.5. Point-to-point communication

Overview

This section concerns direct communication between two processes. Discussion of distributed work, deadlock and other parallel phenomena.

Commands learned:

- `MPI_Send`, `MPI_Recv`, `MPI_Sendrecv`, `MPI_Isend`, `MPI_Irecv`
- `MPI_Wait`...
- Mention of `MPI_Test`, `MPI_Bsend`/*Ssend*/*Rsend*.

MPI point-to-point mechanism

- Two-sided communication
- Matched send and receive calls
- One process sends to a specific other process
- Other process does a specific receive.

Ping-pong

A sends to B, B sends back to A

What is the code for A? For B?

Ping-pong in MPI

Remember SPMD:

```
1 if ( /* I am process A */ ) {
2   MPI_Send( /* to: */ B ..... );
3   MPI_Recv( /* from: */ B ... );
4 } else if ( /* I am process B */ ) {
5   MPI_Recv( /* from: */ A ... );
6   MPI_Send( /* to: */ A ..... );
7 }
```

Sample send and recv calls

```
1 double x[10],y[10];
2 MPI_Send( x,10,MPI_DOUBLE, tgt,0,comm );
3 MPI_Status status;
4 MPI_Recv( y,10,MPI_DOUBLE, src,0,comm,&status );
```

MPI_Send

Name	Param name	Explanation	C type	F type
MPI_Send (				
	buf	initial address of send buffer	const void*	TYPE(*), DIMENSION(..)
	count	number of elements in send buffer	int	INTEGER
	datatype	datatype of each send buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	dest	rank of destination	int	INTEGER
	tag	message tag	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

MPI_Recv

Name	Param name	Explanation	C type	F type
MPI_Recv (				
	buf	initial address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in receive buffer	int	INTEGER
	datatype	datatype of each receive buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	source	rank of source or MPI_ANY_SOURCE	int	INTEGER
	tag	message tag or MPI_ANY_TAG	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	status	status object	MPI_Status*	TYPE(MPI_Status)
	)			

Status object

Use `MPI_STATUS_IGNORE` unless . . .

- Receive call can have various wildcards:
  `MPI_ANY_SOURCE`, `MPI_ANY_TAG`
- Receive buffer size is actually upper bound, not exact
  $\Rightarrow$ Use status object to retrieve actual description of the message

```
1 int s = status.MPI_SOURCE;
2 MPI_Status_get_source( &status,&s ); // mpi-4
3 int t = status.MPI_TAG;
4 MPI_Status_get_tag( &status,&t ); // mpi-4
5 MPI_Status_get_error( &status,&e ); // mpi-4
6 MPI_Get_count(status,MPI_FLOAT,&c);
```

Exercise 19 `pingpong`

Implement the ping-pong program. Add a timer using `MPI_Wtime`. For the `status` argument of the receive call, use `MPI_STATUS_IGNORE`.

- Run multiple ping-pongs (say a thousand) and put the timer around the loop. The first run may take longer; try to discard it.
- Run your code with the two communicating processes first on the same node, then on different nodes. Do you see a difference?
- Then modify the program to use longer messages. How does the timing increase with message size?

For bonus points, can you do a regression to determine α, β?

MPI_Wtime

Name	Param name	Explanation	C type	F type
`MPI_Wtime (` `)`				

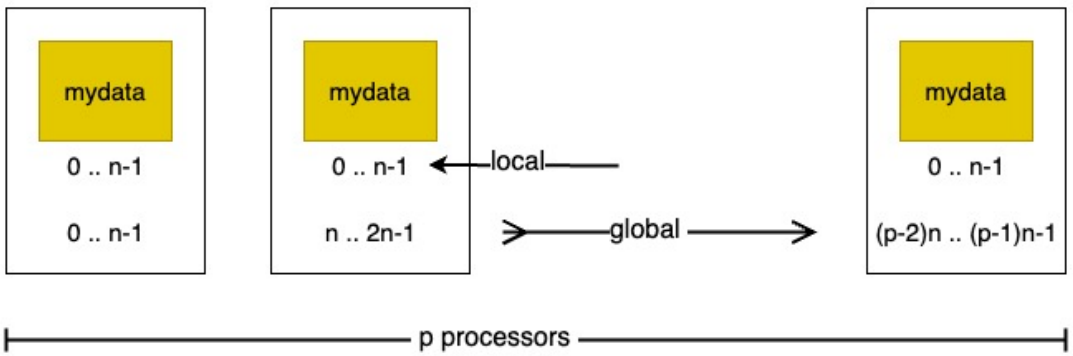
MPI_Wtick

Name	Param name	Explanation	C type	F type
`MPI_Wtick (` `)`				

Distributed data

Distributed data

Distributed array: each process stores disjoint local part



Local numbering $0, \ldots, n_{\text{local}}$;
global numbering is 'in your mind'.

Local and global indexing

Every local array starts at 0 (Fortran: 1);
you have to translate that yourself to global numbering:

```
1 int myfirst = .....;
2 for (int ilocal=0; ilocal<nlocal; ilocal++) {
3    int iglobal = myfirst+ilocal;
4    array[ilocal] = f(iglobal);
5 }
```

Exercise (optional) 20

Implement a (very simple-minded) Fourier transform: if f is a function on the interval $[0, 1]$, then the n-th Fourier coefficient is

$$f_n \hat{=} \int_0^1 f(t) e^{-2\pi x}\, dx$$

which we approximate by

$$f_n \hat{=} \sum_{i=0}^{N-1} f(ih) e^{-in\pi/N}$$

- Make one distributed array for the e^{-inh} coefficients,
- make one distributed array for the $f(ih)$ values
- calculate a couple of coefficients

Load balancing

If the distributed array is not perfectly divisible:

```
int Nglobal, // is something large
    Nlocal = Nglobal/nprocs,
    excess = Nglobal%nprocs;
if (procno==nprocs-1)
  Nlocal += excess;
```

This gives a load balancing problem. Better solution?

(for future reference)

Let

$$f(i) = \lfloor iN/p \rfloor$$

and give process i the points $f(i)$ up to $f(i+1)$.
Result:

$$\lfloor N/p \rfloor \leq f(i+1) - f(i) \leq \lceil N/p \rceil$$

Local information exchange

Motivation

Partial differential equations:

$$-\Delta u = -u_{xx}(\bar{x}) - u_{yy}(\bar{x}) = f(\bar{x}) \text{ for } \bar{x} \in \Omega = [0,1]^2 \text{ with } u(\bar{x}) = u_0 \text{ on } \delta\Omega.$$

Simple case:

$$-u_{xx} = f(x).$$

Finite difference approximation:

$$\frac{2u(x) - u(x+h) - u(x-h)}{h^2} = f(x, u(x), u'(x)) + O(h^2),$$

Finite dimensional: $u_i \equiv u(ih)$.

Motivation (continued)

Equations

$$\begin{cases} -u_{i-1} + 2u_i - u_{i+1} &= h^2 f(x_i) & 1 < i < n \\ 2u_1 - u_2 &= h^2 f(x_1) + u_0 \\ 2u_n - u_{n-1} &= h^2 f(x_n) + u_{n+1}. \end{cases}$$

$$\begin{pmatrix} 2 & -1 & & \emptyset \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ \emptyset & & & \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \end{pmatrix} = \begin{pmatrix} h^2 f_1 + u_0 \\ h^2 f_2 \\ \vdots \end{pmatrix} \tag{1}$$

So we are interested in sparse/banded matrices.

Matrix-vector product

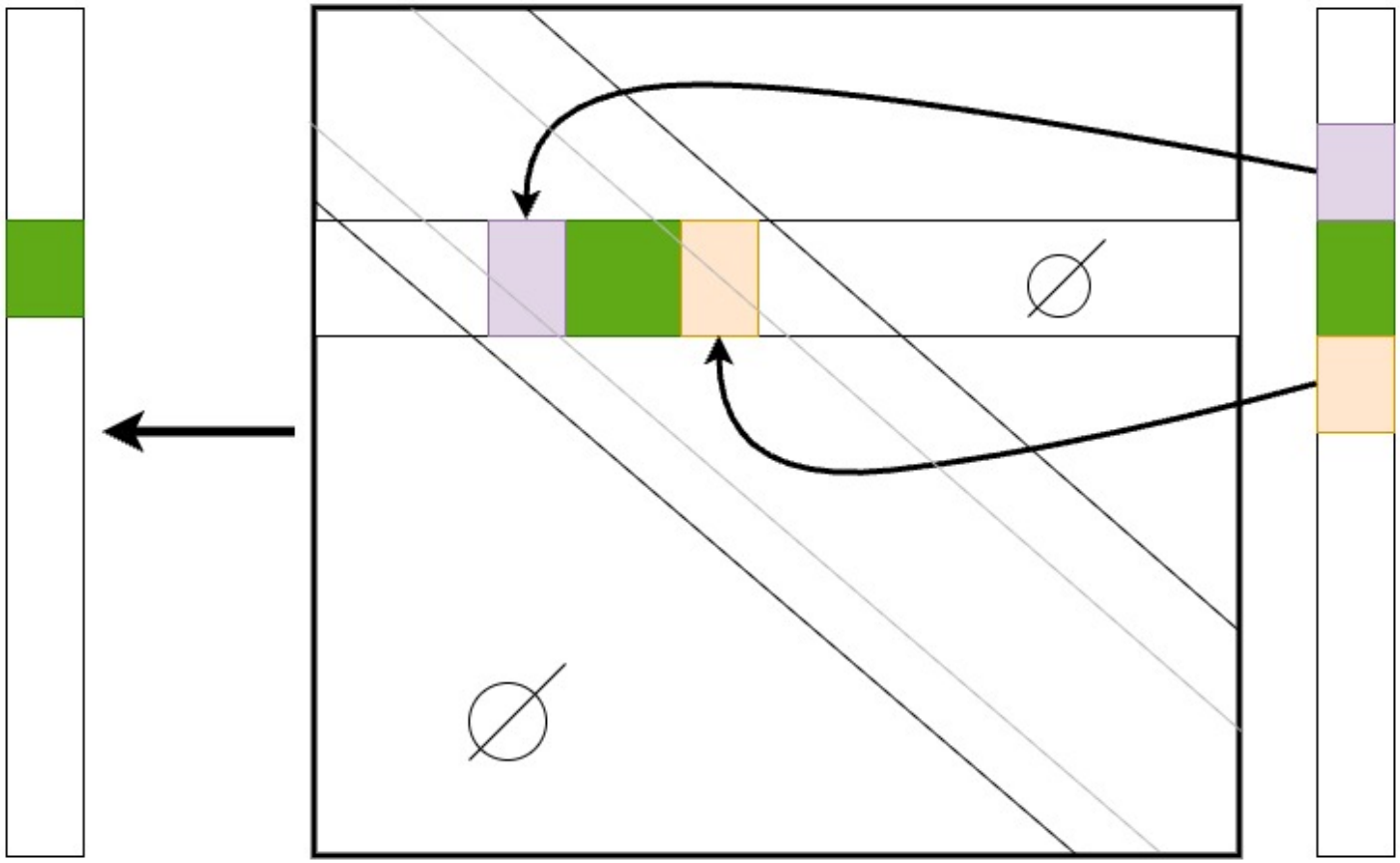
Most common operation: matrix-vector product

$$y \leftarrow Ax, \qquad A = \begin{pmatrix} 2 & -1 & & \emptyset \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ \emptyset & & & \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \\ \vdots \end{pmatrix}$$

- Component operation: $y_i = 2x_i - x_{i-1} - x_{i+1}$
- Parallel execution: each process has range of i-coordinates
- $\Rightarrow$ segment of vector, block row of matrix

Partitioned matrix-vector product

We need a point-to-point mechanism:



each process with ones before/after it.

Operating on distributed data

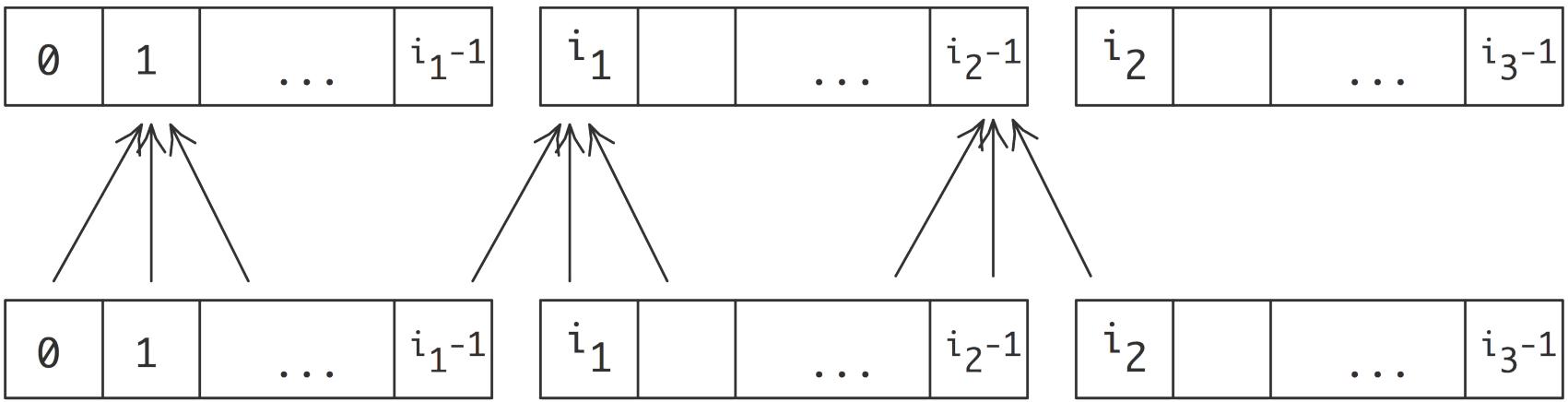
Array of numbers $x_i : i = 0, \ldots, N$

compute

$$y_i = -x_{i-1} + 2x_i - x_{i+1} : i = 1, \ldots, N-1$$

'owner computes'

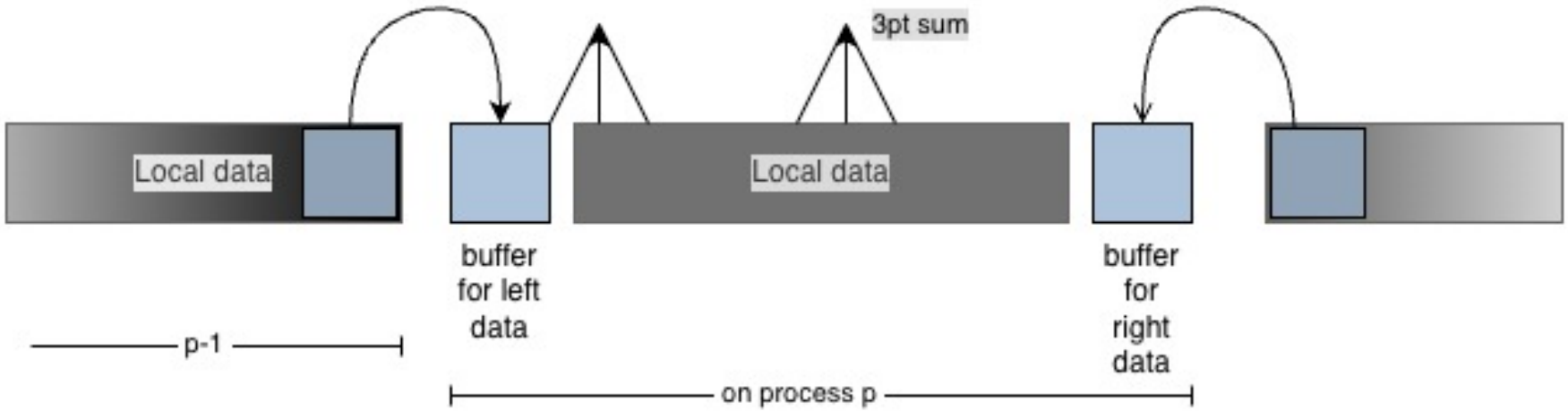
This leads to communication:



so we need a point-to-point mechanism.

Halo regions

Each process needs buffers for receiving data from the left and right:

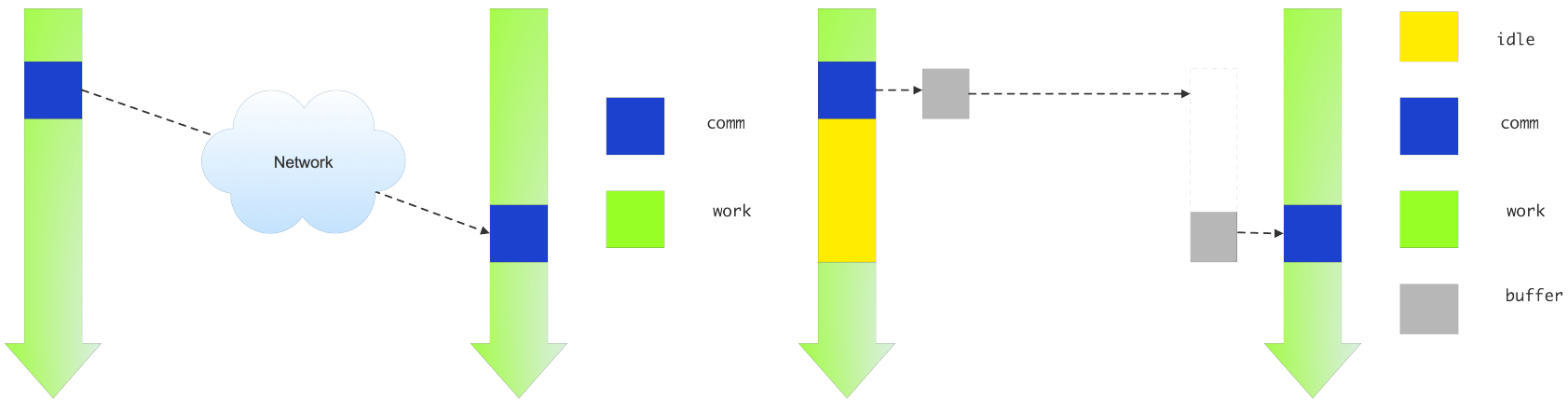


Blocking communication

Blocking send/recv

`MPI_Send` and `MPI_Recv` are *blocking* operations:

- The process waits ('blocks') until the operation is concluded.
- A send can not complete until the receive executes.



Ideal vs actual send/recv behaviour.

Deadlock

Exchange between processes 0/1:

```
1 other = 1-procno; /* if I am 0, other is 1; and vice versa */
2 receive(source=other);
3 send(target=other);
```

A subtlety.
This code may actually work:

```
1 other = 1-procno; /* if I am 0, other is 1; and vice versa */
2 send(target=other);
3 receive(source=other);
```

Small messages get sent even if there is no corresponding receive.
(Often a system parameter, see next)

Rendezvous vs Eager Protocol

Communication is a ‘rendezvous’ or ‘hand-shake’ protocol:

- Sender: ‘I have data for you’
- Receiver: ‘I have a buffer ready, send it over’
- Sender: ‘Ok, here it comes’
- Receiver: ‘Got it.’

Small messages bypass this: ‘eager’ send.
Definition of ‘small message’ controlled by environment variables:

I_MPI_EAGER_THRESHOLD MV2_IBA_EAGER_THRESHOLD

Exercise 21

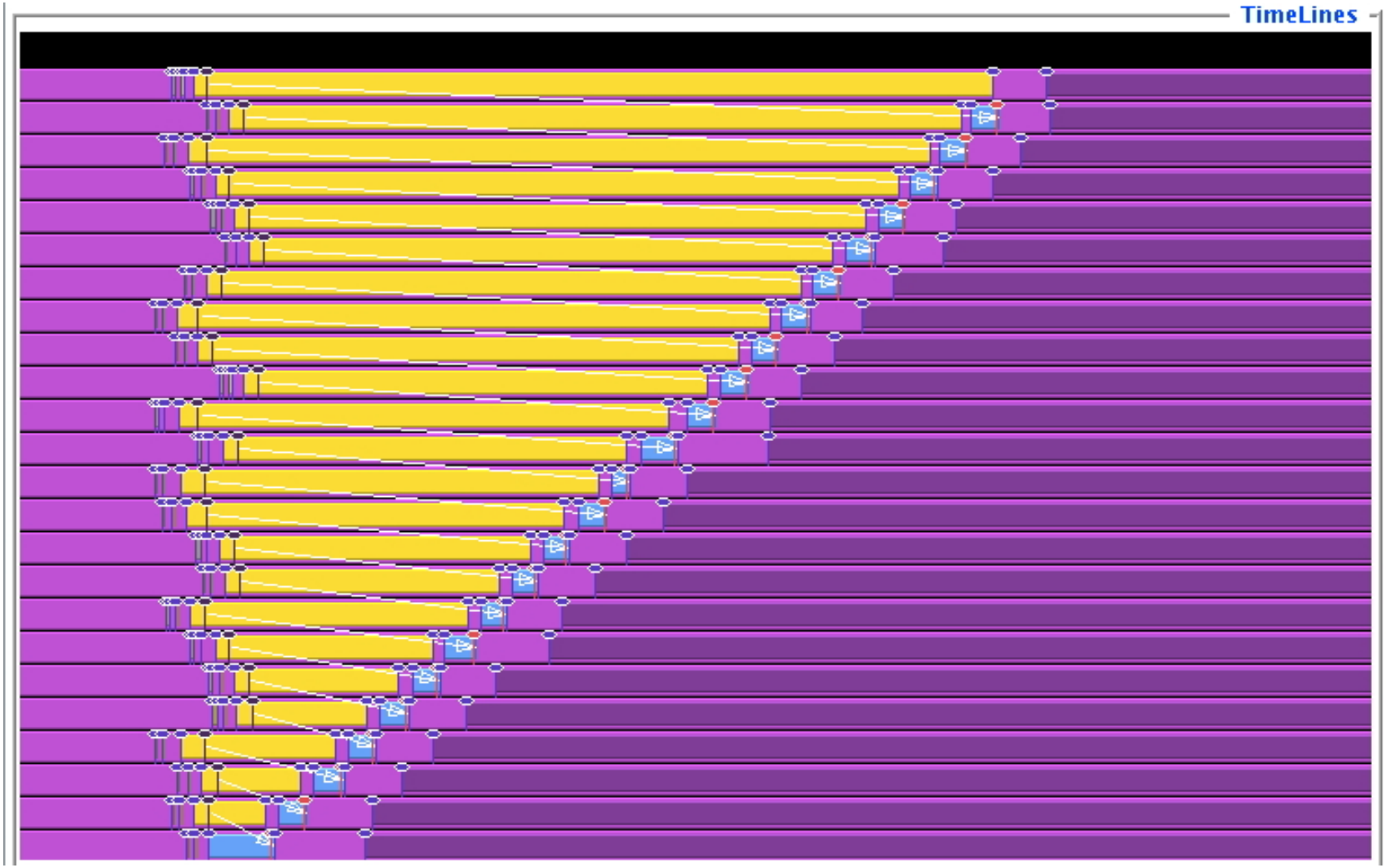
(Classroom exercise) Each student holds a piece of paper in the right hand – keep your left hand behind your back – and we want to execute:

1. Give the paper to your right neighbor;
2. Accept the paper from your left neighbor.

Including boundary conditions for first and last process, that becomes the following program:

1. If you are not the rightmost student, turn to the right and give the paper to your right neighbor.
2. If you are not the leftmost student, turn to your left and accept the paper from your left neighbor.

TAU trace: serialization



The problem here...

Here you have a case of a program that computes the right output, just way too slow.

Conclusion: blocking sends/receives can be trouble.
(How would you solve this particular case?)

Food for thought: what happens if you flip the send and receive call?

Exercise 22 `rightsend`

Implement the above algorithm using `MPI_Send` and `MPI_Recv` calls. Run the code, and use TAU to reproduce the trace output of figure 146. If you don't have TAU, can you show this serialization behavior using timings, for instance running it on an increasing number of processes?

(This is half of the three-point stencil computation;
see slide 140)

Synchronous send

Synchronous send:

- sender and receiver synchronize
- No 'eager' sends
- $\Rightarrow$ enforce always blocking behavior
  $\Rightarrow$ reproducible behavior

MPI_Ssend

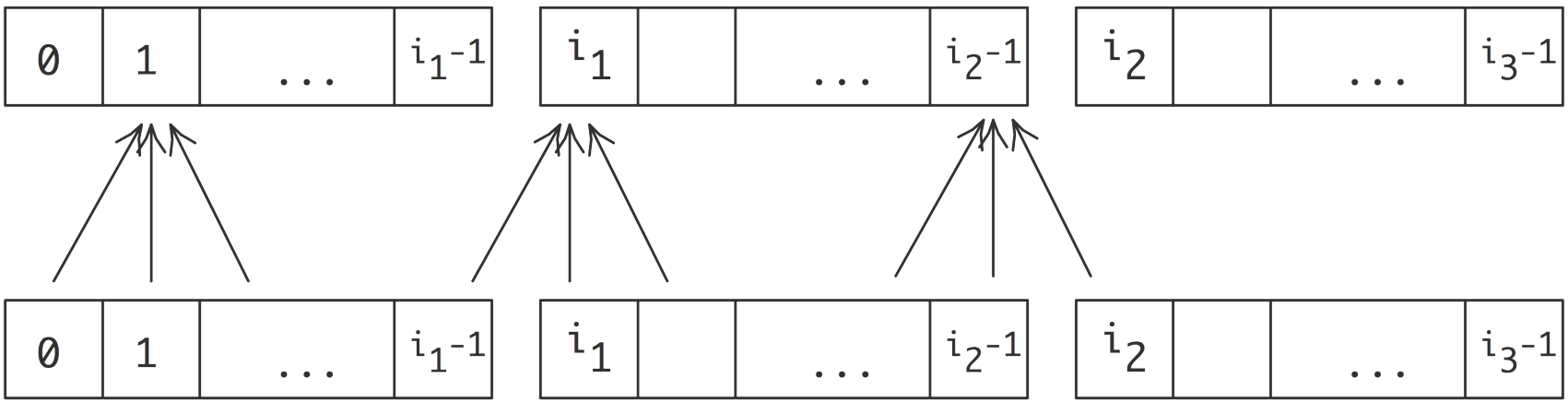
Name	Param name	Explanation	C type	F type
MPI_Ssend (				
	buf	initial address of send buffer	const void*	TYPE(*), DIMENSION(..)
	count	number of elements in send buffer	int	INTEGER
	datatype	datatype of each send buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	dest	rank of destination	int	INTEGER
	tag	message tag	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	)			

Pairwise exchange

Operating on distributed data

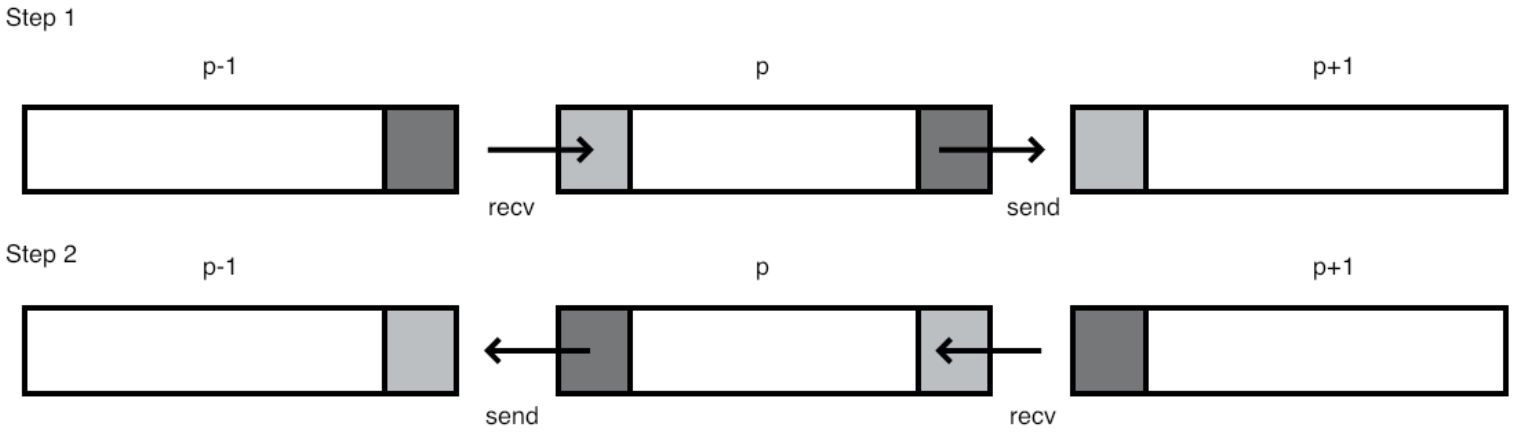
Take another look:

$$y_i = x_{i-1} + x_i + x_{i+1} : i = 1, \ldots, N-1$$



- One-dimensional data and linear process numbering;
- Operation between neighboring indices: communication between neighboring processes.

Two steps



First do all the data movement to the right, later to the left.

- Each process does a send and receive
- So everyone does the send, then the receive? We just saw the problem with that.
- Better solution coming up!

Sendrecv

Instead of separate send and receive: use

`MPI_Sendrecv`

To be used if every* process does exactly one send and receive.

Combined calling sequence of send and receive;
execute such that no deadlock or sequentialization.

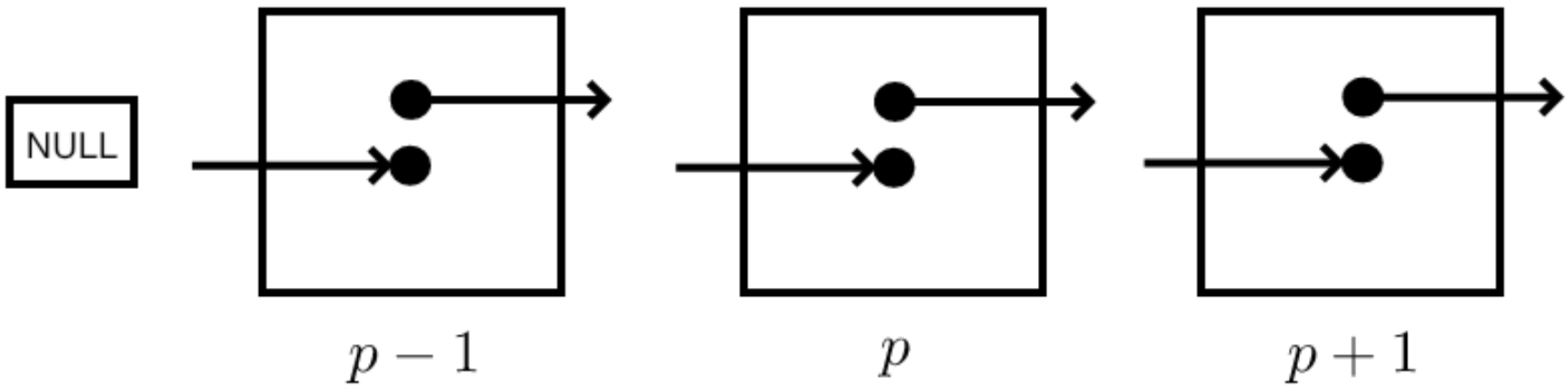
(Also: `MPI_Sendrecv_replace` with single buffer.)

MPI_Sendrecv

Name	Param name	Explanation	C type	F type
MPI_Sendrecv (				
	sendbuf	initial address of send buffer	const void*	TYPE(*), DIMENSION(..)
	sendcount	number of elements in send buffer	int	INTEGER
	sendtype	type of elements in send buffer	MPI_Datatype	TYPE(MPI_Datatype)
	dest	rank of destination	int	INTEGER
	sendtag	send tag	int	INTEGER
	recvbuf	initial address of receive buffer	void*	TYPE(*), DIMENSION(..)
	recvcount	number of elements in receive buffer	int	INTEGER
	recvtype	type of elements receive buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	source	rank of source or MPI_ANY_SOURCE	int	INTEGER
	recvtag	receive tag or MPI_ANY_TAG	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	status	status object	MPI_Status*	TYPE(MPI_Status)
	)			

SPMD picture

What does process p do?



Sendrecv with incomplete pairs

Not all processes do both send and receive:

```
1  MPI_Comm_rank( .... &procno );
2  if ( /* I am not the first process */ )
3    predecessor = procno-1;
4  else
5    predecessor = MPI_PROC_NULL;
6
7  if ( /* I am not the last process */ )
8    successor = procno+1;
9  else
10   successor = MPI_PROC_NULL;
11
12 sendrecv(from=predecessor,to=successor);
```

(Receive from `MPI_PROC_NULL` succeeds without altering the receive buffer.)

A point of programming style

The previous slide had:

- a conditional for computing the sender and receiver rank;
- a single Sendrecv call.

Also possible:

Three versions of the sendrecv call:

```
1 if ( /* i am first */ )
2   Sendrecv( to=right, from=NULL );
3 else if ( /* i am last */
4   Sendrecv( to=NULL,  from=left );
5 else
6   Sendrecv( to=right, from=left );
```

Combine sendrecv and send/recv:

```
1 if ( /* i am first */ )
2   Send( to=right );
3 else if ( /* i am last */
4   Recv( from=left );
5 else
6   Sendrecv( to=right, from=left );
```

But:
Code duplication is error-prone, also
chance of deadlock by missing a case

Exercise (optional) 23 (`rightsend`)

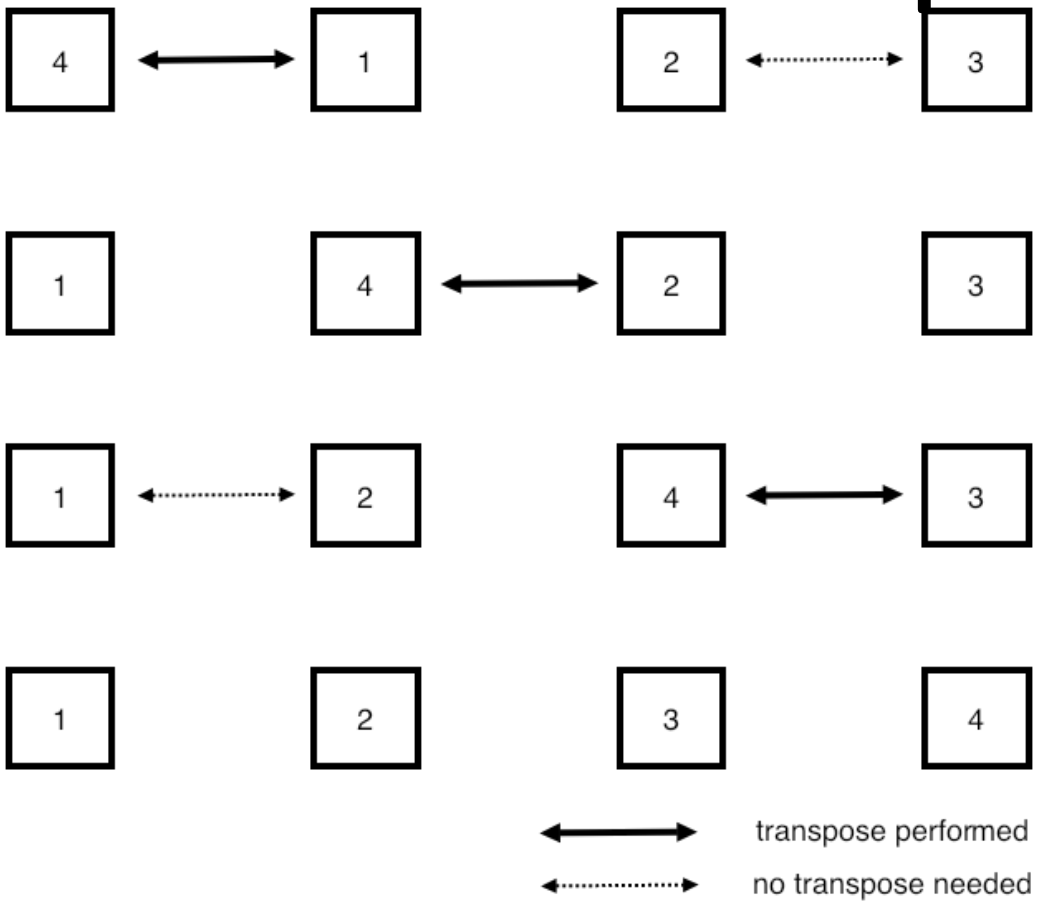
Revisit exercise 22 and solve it using `MPI_Sendrecv`.

If you have TAU installed, make a trace. Does it look different from the serialized send/recv code? If you don't have TAU, run your code with different numbers of processes and show that the runtime is essentially constant.

Exercise 24 `sendrecv`

Implement the above three-point combination scheme using `MPI_Sendrecv`; every processor only has a single number to send to its neighbor.

Odd-even transposition sort



Odd-even transposition sort on 4 elements.

Exercise (optional) 25

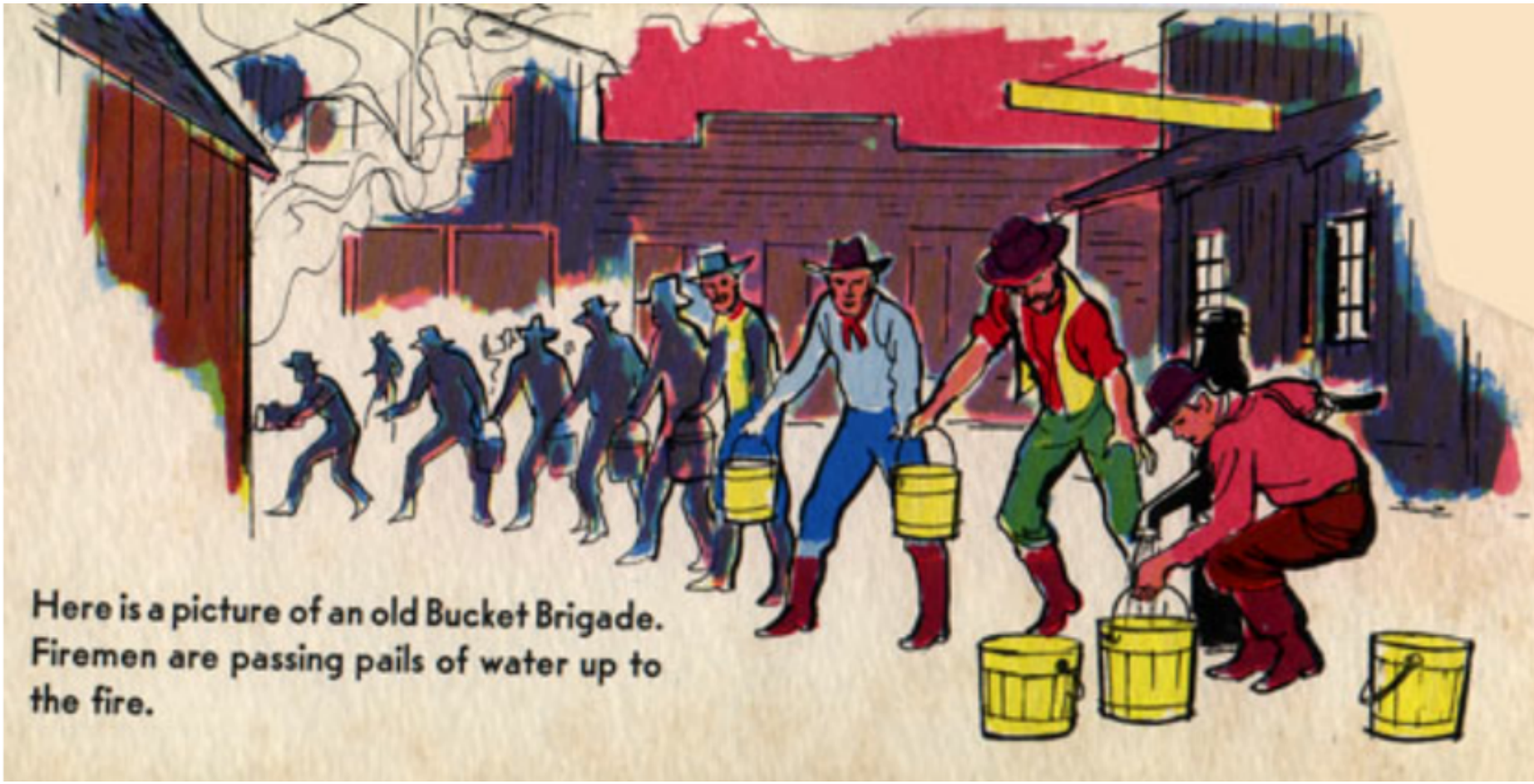
A very simple sorting algorithm is *swap sort* or *odd-even transposition sort*: pairs of processors compare data, and if necessary exchange. The elementary step is called a *compare-and-swap*: in a pair of processors each sends their data to the other; one keeps the minimum values, and the other the maximum. For simplicity, in this exercise we give each processor just a single number.

The transposition sort algorithm is split in even and odd stages, where in the even stage processors $2i$ and $2i+1$ compare and swap data, and in the odd stage processors $2i+1$ and $2i+2$ compare and swap. You need to repeat this $P/2$ times, where P is the number of processors; see figure 161.

Implement this algorithm using `MPI_Sendrecv`. (Use `MPI_PROC_NULL` for the edge cases if needed.) Use a gather call to print the global state of the distributed array at the beginning and end of the sorting process.

Bucket brigade

Sometimes you really want to pass information from one process to the next: 'bucket brigade'



Exercise 26 `bucketblock`

Take the code of exercise 22 and modify it so that the data from process zero gets propagated to every process. Specifically, compute all partial sums $\sum_{i=0}^{p} i^2$:

$$\begin{cases} x_0 = 1 & \text{on process zero} \\ x_p = x_{p-1} + (p+1)^2 & \text{on process } p \end{cases}$$

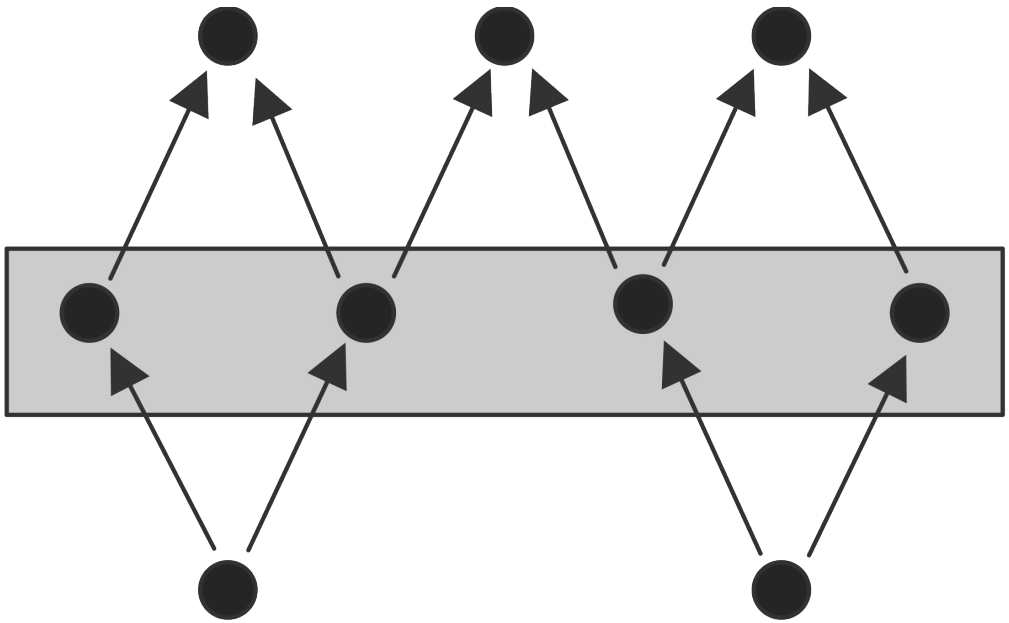
Use `MPI_Send` and `MPI_Recv`; make sure to get the order right.

Food for thought: all quantities involved here are integers. Is it a good idea to use the integer datatype here?

Irregular exchanges: non-blocking communication

Sending with irregular connections

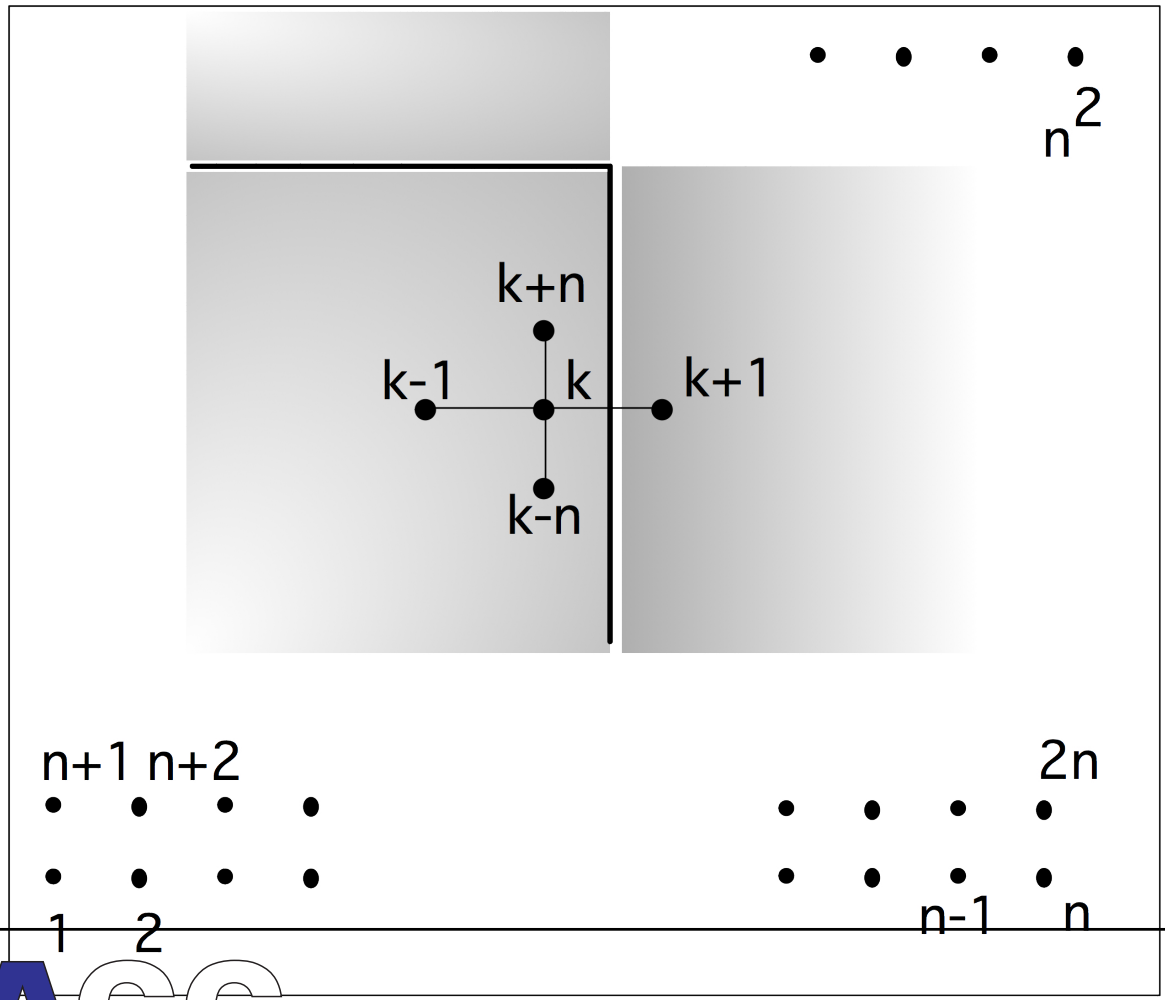
Graph operations:



Communicating other than in pairs

PDE, 2D case

A difference stencil applied to a two-dimensional square domain, distributed over processes. A cross-process connection is indicated ⇒ complicated to express pairwise

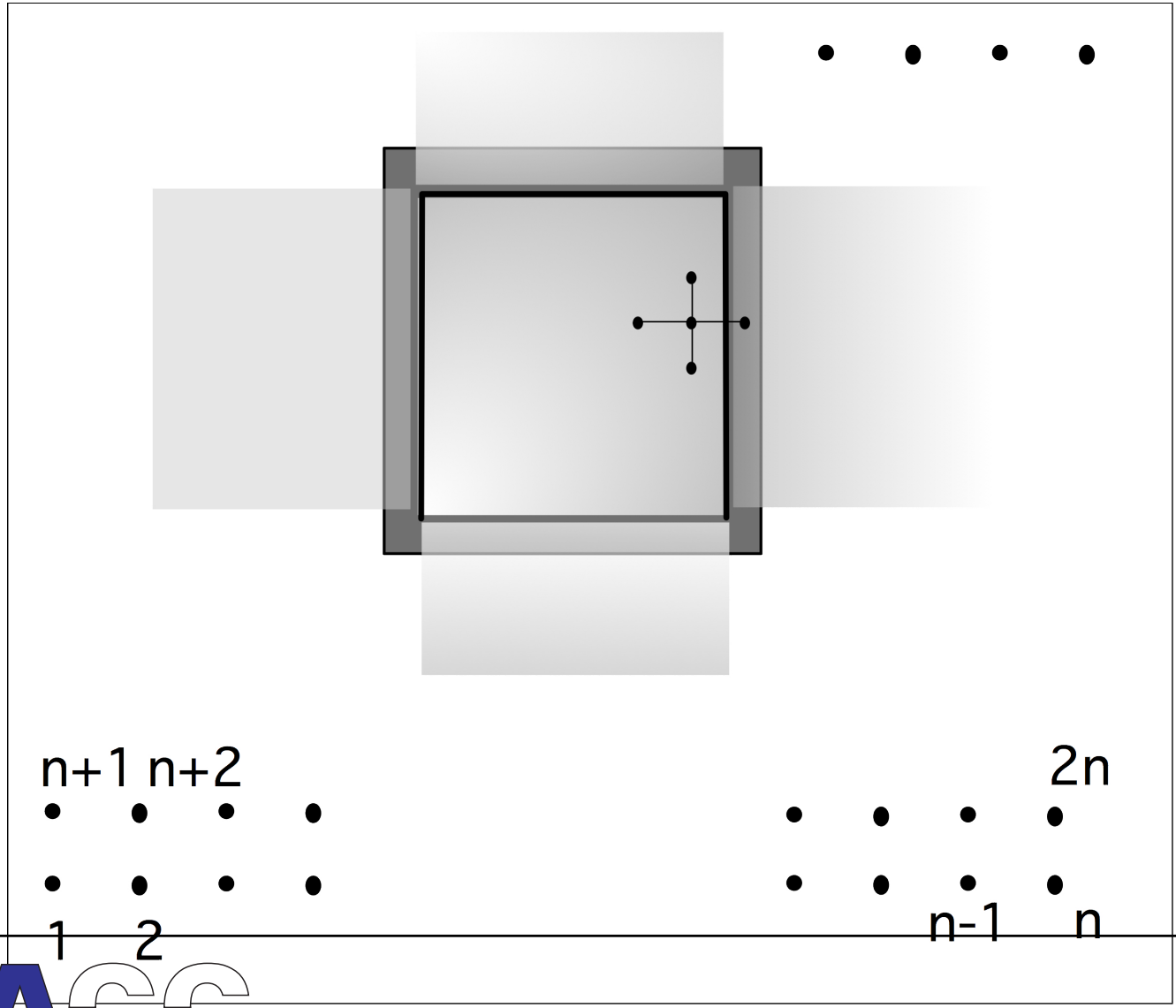


PDE matrix

$$
A = \left(\begin{array}{cccccc|cccccc|cc}
4 & -1 & & & \emptyset & & -1 & & & & \emptyset & & & \\
-1 & 4 & -1 & & & & & -1 & & & & & & \\
 & \ddots & \ddots & \ddots & & & & & \ddots & & & & & \\
 & & \ddots & \ddots & -1 & & & & & \ddots & & & & \\
\emptyset & & & -1 & 4 & & \emptyset & & & & -1 & & & \\
\hline
-1 & & & & \emptyset & & 4 & -1 & & & & & -1 & \\
 & -1 & & & & & -1 & 4 & -1 & & & & & -1 \\
 & \uparrow & \ddots & & & & \uparrow & \uparrow & \uparrow & & & & \uparrow & \\
 & k-n & & & & & k-1 & k & k+1 & & -1 & & k+n & \\
 & & & & -1 & & & & & -1 & 4 & & & \\
\hline
 & & & & & & \ddots & & & & & & \ddots &
\end{array}\right)
$$

Halo region

The halo region of a process, induced by a stencil



How do you approach this?

- It is very hard to figure out a send/receive sequence that does not deadlock or serialize
- Even if you manage that, you may have process idle time.

Instead:

- Declare 'this data needs to be sent' or 'these messages are expected', and
- then wait for them collectively.

Non-blocking send/recv

- `MPI_Isend` / `MPI_Irecv` does not send/receive:
- They declare a buffer.
- The buffer contents are there after a wait call.
- In between the `MPI_Isend` and `MPI_Wait` the data may not have been sent.
- In between the `MPI_Irecv` and `MPI_Wait` the data may not have arrived.

```
1 // start non-blocking communication
2 MPI_Isend( ... ); MPI_Irecv( ... );
3 // wait for the Isend/Irecv calls to finish in any order
4 MPI_Wait( ... );
```

MPI_Isend

Name	Param name	Explanation	C type	F type
MPI_Isend (				
	buf	initial address of send buffer	const void*	TYPE(*), DIMENSION(..)
	count	number of elements in send buffer	int	INTEGER
	datatype	datatype of each send buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	dest	rank of destination	int	INTEGER
	tag	message tag	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	request	communication request	MPI_Request*	TYPE(MPI_Request)
	)			

MPI_Irecv

Name	Param name	Explanation	C type	F type
MPI_Irecv (				
	buf	initial address of receive buffer	void*	TYPE(*), DIMENSION(..)
	count	number of elements in receive buffer	int	INTEGER
	datatype	datatype of each receive buffer element	MPI_Datatype	TYPE(MPI_Datatype)
	source	rank of source or MPI_ANY_SOURCE	int	INTEGER
	tag	message tag or MPI_ANY_TAG	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	request	communication request	MPI_Request*	TYPE(MPI_Request)
	)			

Request waiting

Basic wait:

```
MPI_Wait( MPI_Request*, MPI_Status* );
```

- Status object or `MPI_STATUS_IGNORE`;
- Request gets deallocated.

Most common way of waiting for completion:

```
int MPI_Waitall(
    int count, MPI_Request array_of_requests[],
    MPI_Status array_of_statuses[])
```

- Ignore status array: `MPI_STATUSES_IGNORE`;
- Other wait calls: `MPI_Wait`, `MPI_Waitany`, `MPI_Waitsome`.

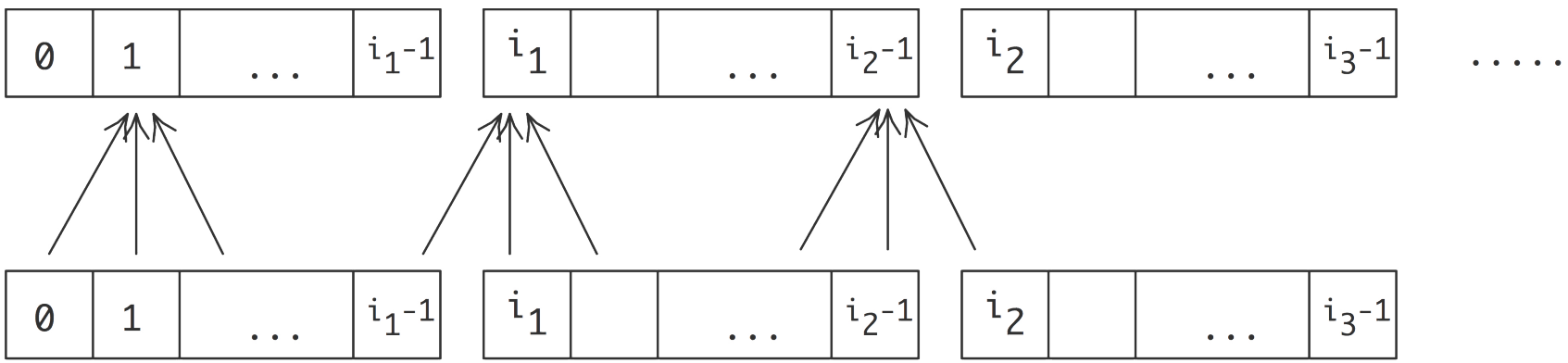
Exercise 27 `isendirecv`

Now use nonblocking send/receive routines to implement the three-point averaging operation

$$y_i = \big(x_{i-1} + x_i + x_{i+1}\big)/3 \colon i = 1, \ldots, N-1$$

on a distributed array. There are two approaches to the first and last process:

1. you can use `MPI_PROC_NULL` for the 'missing' communications;
2. you can skip these communications altogether, but now you have to count the requests carefully.



(Can you think of a different way of handling the end points?)

Comparison

- Obvious: blocking vs non-blocking behaviour.
- Buffer reuse: when a blocking call returns, the buffer is safe for reuse or free;
- A buffer in a non-blocking call can only be reused/freed after the wait call.

Buffer use in blocking/non-blocking case

Blocking:

```
1 double *buffer;
2 // allocate the buffer
3 for ( ... p ... ) {
4    buffer = // fill in the data
5    MPI_Send( buffer, ... /* to: */ p );
```

Non-blocking:

```
1 double **buffers;
2 // allocate the buffers
3 for ( ... p ... ) {
4    buffers[p] = // fill in the data
5    MPI_Isend( buffers[p], ... /* to: */ p );
6 MPI_Waitall(.....)
```

Pitfalls

- Strictly one request/wait per `isend`/`irecv`:
  can not use one request for multiple simultaneous `isend`/`irecvs`
- Some people argue:

  > *Wait for the send is not necessary: if you wait for the receive, the message has arrived safely*

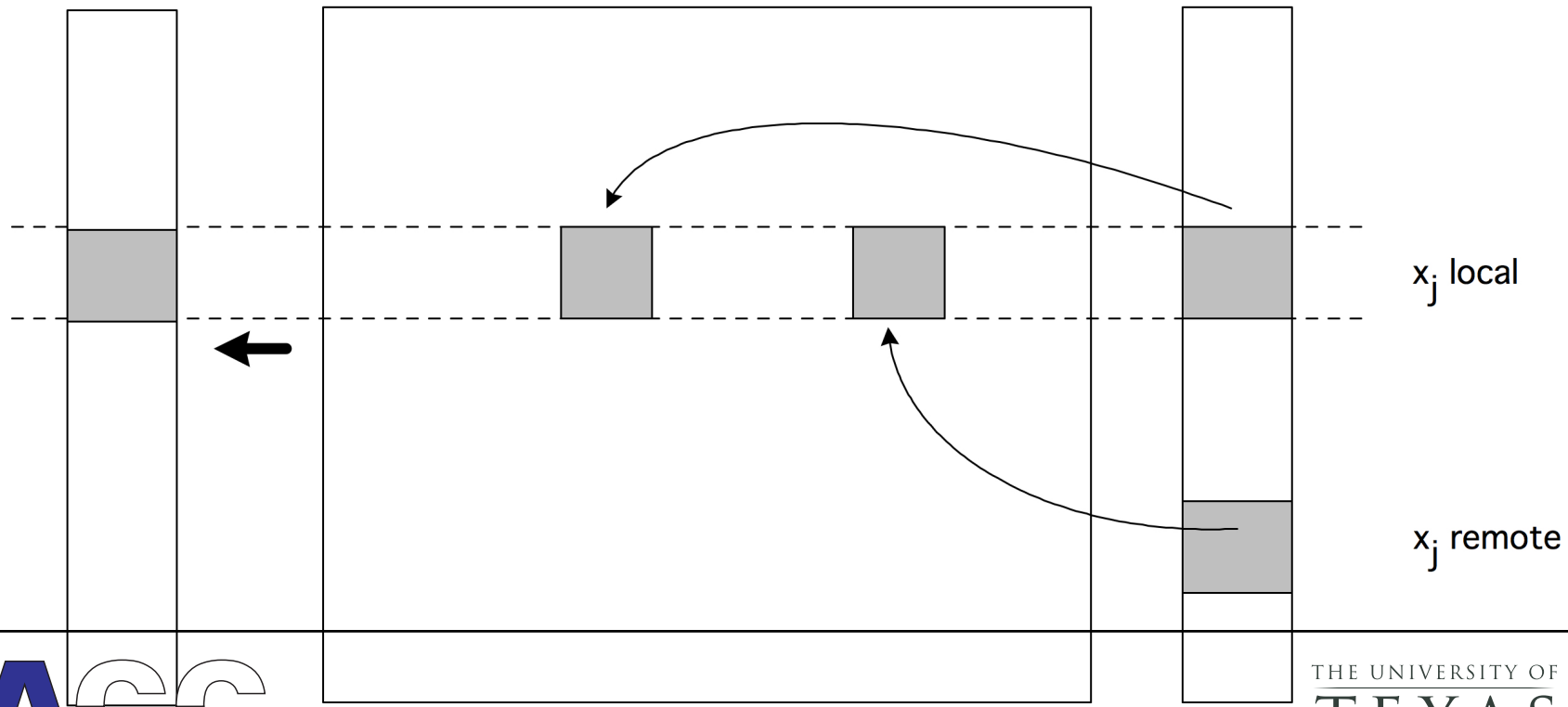
  This leads to memory leaks! The `wait` call deallocates the request object.

Matrices in parallel

$$y \leftarrow Ax$$

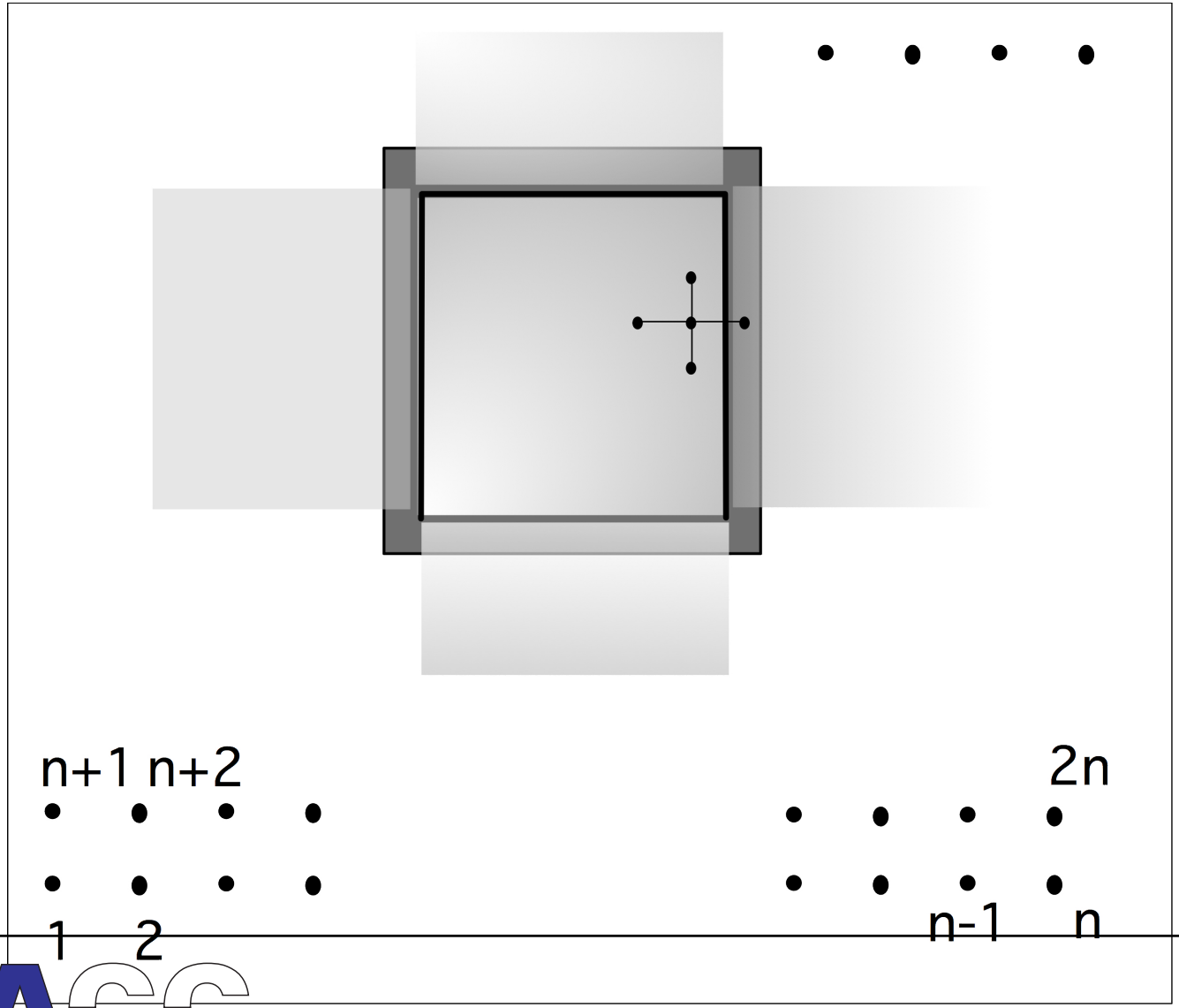
and A, x, y all distributed:

$$y_p \leftarrow A_{pp}x_p + \sum_{j \neq p} A_{pj}x_j$$



Hiding the halo

Interior of a process domain can overlap with halo transfer:



Latency hiding

Other motivation for non-blocking calls:
overlap of computation and communication, provided hardware support.

Also known as 'latency hiding'.

Example: three-point combination operation (see above):

1. Start communication for edge points,
2. Do local operations while communication goes on,
3. Wait for edge points from neighbor processes
4. Incorporate incoming data.

Exercise 28 `isendirecvarray`

Take your code of exercise 27 and modify it to use latency hiding. Operations that can be performed without needing data from neighbors should be performed in between the `MPI_Isend` / `MPI_Irecv` calls and the corresponding `MPI_Wait` calls.

Write your code so that it can achieve latency hiding.

Mix and match

You can match blocking and non-blocking:

```
1 if ( /* I am Process A */ ) {
2   MPI_Isend( /* to B */, &req2 );
3   MPI_Irecv( /* from B */, &req1 );
4   MPI_Waitall( /* requests 1 and 2 */ );
5 } else if ( /* I am Process B */ ) {
6   MPI_Recv( /* from A */ );
7   MPI_Send( /* to A */ );
8 }
```

- Can you flip the Isend/Irecv?
- Can you change the Recv/Send for process B to Irecv/Isend?

Test: non-blocking wait

- Post non-blocking receives
- test on the request(s) for incoming messages
- if nothing comes in, do local work

```
1 while (1) {
2   MPI_Test( some_request, &flag );
3   if (flag)
4     // do something with incoming message
5   else
6     // do local work
7 }
```

Local operation.
Also *MPI_Testall* et cetera.

Probe for message

Is there a message?

```
1  // probe.c
2  if (procno==receiver) {
3    MPI_Status status;
4    MPI_Probe(sender,0,comm,&status);
5    int count;
6    MPI_Get_count(&status,MPI_FLOAT,&count);
7    float recv_buffer[count];
8    MPI_Recv(recv_buffer,count,MPI_FLOAT, sender,0,comm,MPI_STATUS_IGNORE);
9  } else if (procno==sender) {
10   float buffer[buffer_size];
11   ierr = MPI_Send(buffer,buffer_size,MPI_FLOAT, receiver,0,comm); CHK(ierr);
12 }
```

(Also non-blocking `MPI_Iprobe`.)
These commands force global progress.

The Pipeline Pattern

- Remember the bucket brigade: data propagating through processes
- If you have many buckets being passed: pipeline
- This is very parallel: only filling and draining the pipeline is not completely parallel
- Application to long-vector broadcast: pipelining gives overlap

Exercise (optional) 29 (`bucketpipenonblock`)

Implement a pipelined broadcast for long vectors:
use non-blocking communication to send the vector in parts.

Exercise 30 `setdiff`

Create two distributed arrays of positive integers. Take the set difference of the two: the first array needs to be transformed to remove from it those numbers that are in the second array.

How could you solve this with an `MPI_Allgather` call? Why is it not a good idea to do so? Solve this exercise instead with a circular bucket brigade algorithm.

Consider: `MPI_Send` and `MPI_Recv` vs `MPI_Sendrecv` vs `MPI_Sendrecv_replace` vs `MPI_Isend` and `MPI_Irecv`

The wheel of reinvention

The circular bucket brigade is the idea behind the 'Horovod' library, which is the key to efficient parallel Deep Learning.

More sends and receive

- `MPI_Bsend`, `MPI_Ibsend`: buffered send
- `MPI_Ssend`, `MPI_Issend`: synchronous send
- `MPI_Rsend`, `MPI_Irsend`: ready send
- Persistent communication: repeated instance of same proc/data description.

  MPI-4:

  - *Partitioned sends.*

too obscure to go into.

Review 3

Does this code deadlock?

```
1 for (int p=0; p<nprocs; p++)
2   if (p!=procid)
3     MPI_Send(sbuffer,buflen,MPI_INT,p,0,comm);
4 for (int p=0; p<nprocs; p++)
5   if (p!=procid)
6     MPI_Recv(rbuffer,buflen,MPI_INT,p,0,comm,MPI_STATUS_IGNORE);
```

Review 4

Does this code deadlock?

```
1 int ireq = 0;
2 for (int p=0; p<nprocs; p++)
3   if (p!=procid)
4     MPI_Isend(sbuffers[p],buflen,MPI_INT,p,0,comm,&(requests[ireq++]));
5 for (int p=0; p<nprocs; p++)
6   if (p!=procid)
7     MPI_Recv(rbuffer,buflen,MPI_INT,p,0,comm,MPI_STATUS_IGNORE);
8 MPI_Waitall(nprocs-1,requests,MPI_STATUSES_IGNORE);
```

Review 5

Does this code deadlock?

```
1 int ireq = 0;
2 for (int p=0; p<nprocs; p++)
3   if (p!=procid)
4     MPI_Irecv(rbuffers[p],buflen,MPI_INT,p,0,comm,&(requests[ireq++]));
5 MPI_Waitall(nprocs-1,requests,MPI_STATUSES_IGNORE);
6 for (int p=0; p<nprocs; p++)
7   if (p!=procid)
8     MPI_Send(sbuffer,buflen,MPI_INT,p,0,comm);
```

Review 6

Does this code deadlock?

```
1 int ireq = 0;
2 for (int p=0; p<nprocs; p++)
3   if (p!=procid)
4     MPI_Irecv(rbuffers[p],buflen,MPI_INT,p,0,comm,&(requests[ireq++]));
5 for (int p=0; p<nprocs; p++)
6   if (p!=procid)
7     MPI_Send(sbuffer,buflen,MPI_INT,p,0,comm);
8 MPI_Waitall(nprocs-1,requests,MPI_STATUSES_IGNORE);
```

Where to go from here...

- Derived data types: send strided/irregular/inhomogeneous data
- Sub-communicators: work with subsets of `MPI_COMM_WORLD`
- I/O: efficient file operations
- One-sided communication: 'just' put/get the data somewhere
- Process management
- Non-blocking collectives
- Graph topology and neighborhood collectives
- Shared memory

2. Intermediate topics

Justification

MPI basic concepts suffice for many applications. The Intermediate Topics section deals with more complicated data, process groups, file I/O, and the basics of one-sided communication.

2.1. Derived Datatypes

Overview

In this section you will learn about derived data types.

Commands learned:

- `MPI_Type_contiguous`/`vector`/`indexed`/`struct` `MPI_Type_create_subarray`
- `MPI_Pack` / `MPI_Unpack`

Motivation: datatypes in MPI

All examples so far:

- contiguous buffer
- elements of single type

We need data structures with gaps, or heterogeneous types.

- Send real or imaginary parts out of complex array.
- Gather/scatter cyclicly.
- Send `struct` data.

MPI allows for recursive construction of data types.

Datatype topics

- Elementary types: built-in.
- Derived types: user-defined.
- Packed data: not really a datatype.

Elementary datatypes

C/C++	Fortran
MPI_CHAR	MPI_CHARACTER
MPI_UNSIGNED_CHAR	
MPI_SIGNED_CHAR	
	MPI_LOGICAL
MPI_SHORT	
MPI_UNSIGNED_SHORT	
MPI_INT	MPI_INTEGER
MPI_UNSIGNED	
MPI_LONG	
MPI_UNSIGNED_LONG	
MPI_FLOAT	MPI_REAL
MPI_DOUBLE	MPI_DOUBLE_PRECISION
MPI_LONG_DOUBLE	
	MPI_COMPLEX
	MPI_DOUBLE_COMPLEX

How to use derived types

Create, commit, use, free:

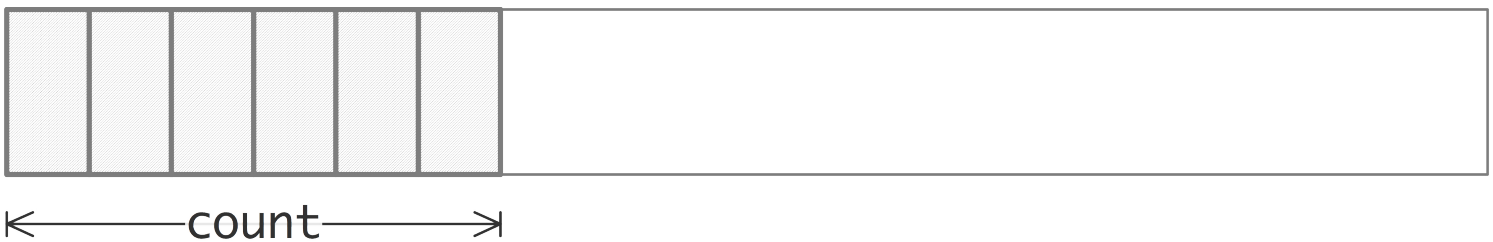
```
MPI_Datatype newtype;
MPI_Type_xxx( ... oldtype ... &newtype);
MPI_Type_commit ( &newtype );

  // code using the new type

MPI_Type_free ( &newtype );
```

```
Type(MPI_Datatype) :: newtype ! F2008
Integer            :: newtype ! F90
```

The *oldtype* can be elementary or derived.
Recursively constructed types.

Contiguous type

```
1 int MPI_Type_contiguous(
2   int count, MPI_Datatype old_type, MPI_Datatype *new_type_p)
```



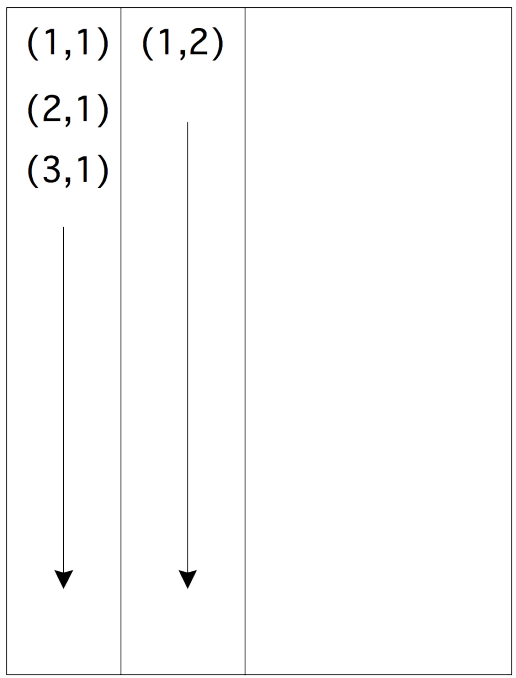
This one is indistinguishable from sending `count` instances of the *`old_type`*.

Example: non-contiguous data

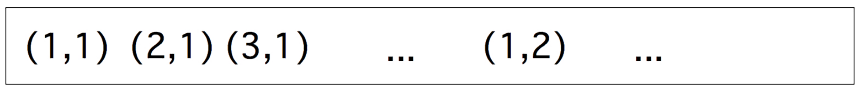
Matrix in column storage:

- Columns are contiguous
- Rows are not contiguous

Logical:

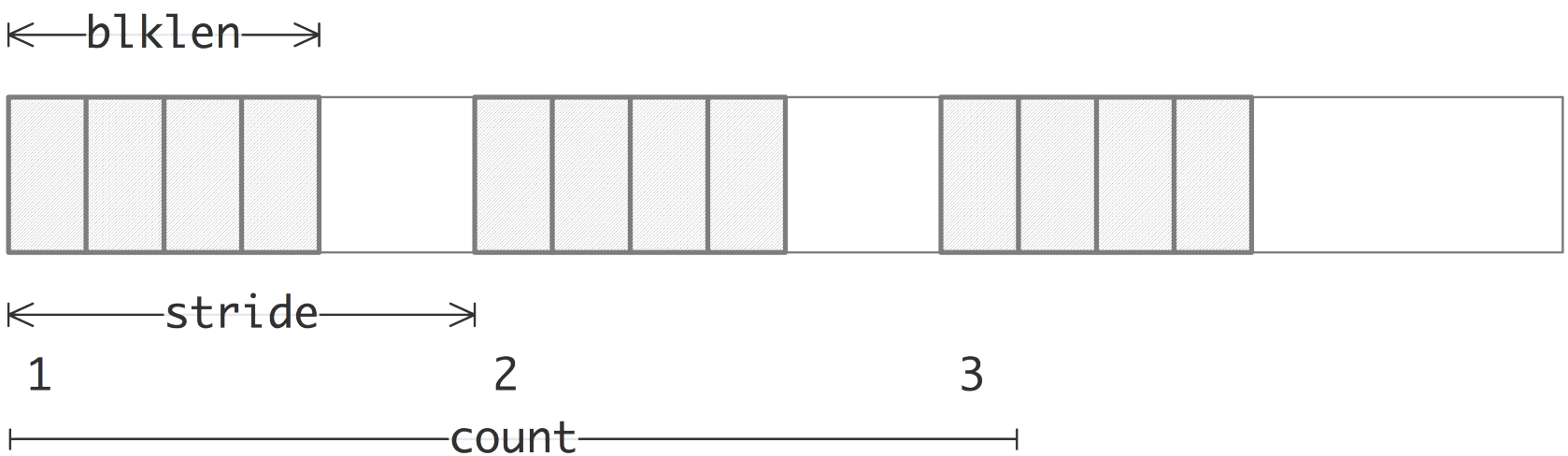


Physical:



Vector type

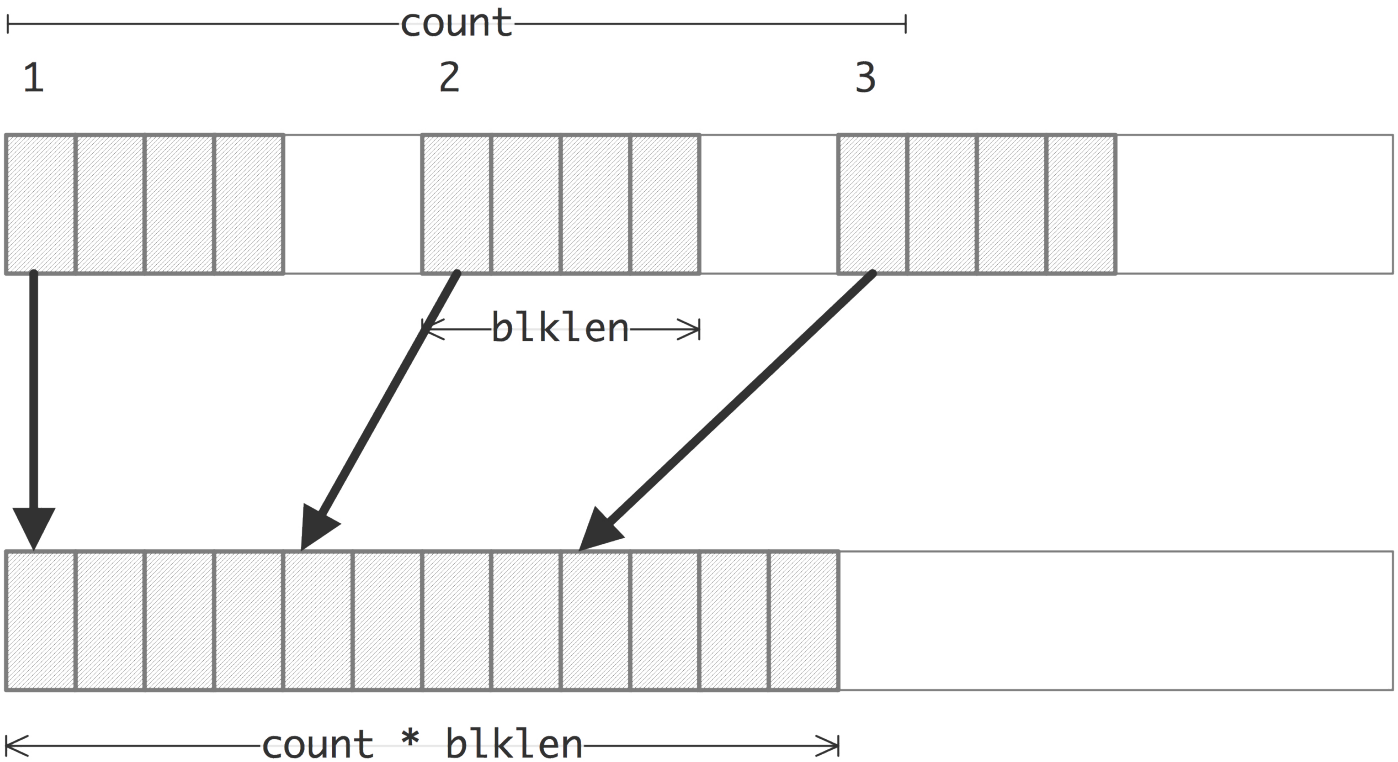
```
1 int MPI_Type_vector(
2   int count, int blocklength, int stride,
3   MPI_Datatype old_type, MPI_Datatype *newtype_p
4 );
```



Used to pick a regular subset of elements from an array.

Different send and receive types

Send and receive type can differ. Example:
sender type: vector
receiver type: contiguous or elementary

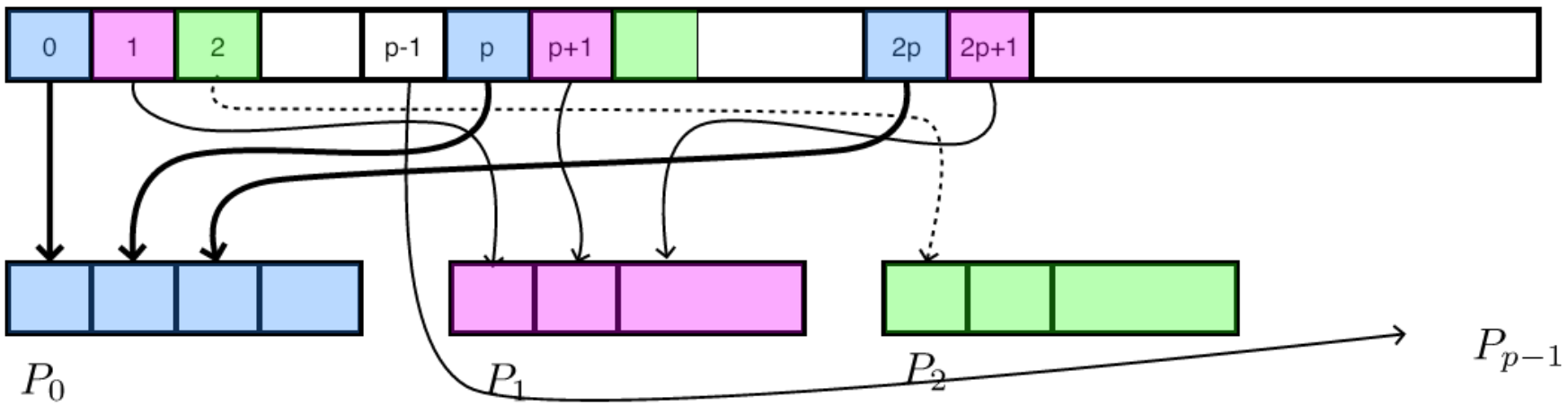


Receiver has no knowledge of the stride of the sender.

Send vs recv type

```
1  // vector.c
2  source = (double*) malloc(stride*count*sizeof(double));
3  target = (double*) malloc(count*sizeof(double));
4  MPI_Datatype newvectortype;
5  if (procno==sender) {
6    MPI_Type_vector(count,1,stride,MPI_DOUBLE,&newvectortype);
7    MPI_Type_commit(&newvectortype);
8    MPI_Send(source,1,newvectortype,the_other,0,comm);
9    MPI_Type_free(&newvectortype);
10 } else if (procno==receiver) {
11   MPI_Status recv_status;
12   int recv_count;
13   MPI_Recv(target,count,MPI_DOUBLE,the_other,0,comm,
14     &recv_status);
15   MPI_Get_count(&recv_status,MPI_DOUBLE,&recv_count);
16   ASSERT(recv_count==count);
17 }
```

Illustration of the next exercise



Sending strided data from process zero to all others

Exercise 31 `stridesend`

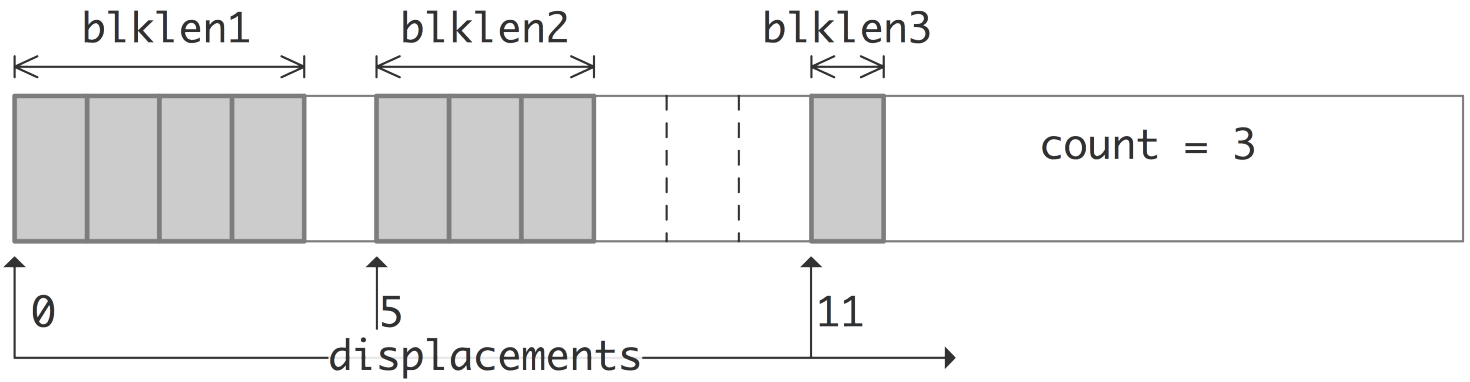
Let processor 0 have an array x of length $10P$, where P is the number of processors. Elements $0, P, 2P, \ldots, 9P$ should go to processor zero, $1, P+1, 2P+1, \ldots$ to processor 1, et cetera.

- Code this as a sequence of send/recv calls, using a vector datatype for the send, and a contiguous buffer for the receive.
- For simplicity, skip the send to/from zero. What is the most elegant solution if you want to include that case?
- For testing, define the array as $x[i] = i$.

Exercise (optional) 32

Allocate a matrix on processor zero, using Fortran column-major storage. Using P sendrecv calls, distribute the rows of this matrix among the processors.

Indexed type




```
1 int MPI_Type_indexed(
2   int count, int blocklens[], int displacements[],
3   MPI_Datatype old_type, MPI_Datatype *newtype);
```

Hindexed type

Similar to indexed but using byte offsets:
explicit memory address.

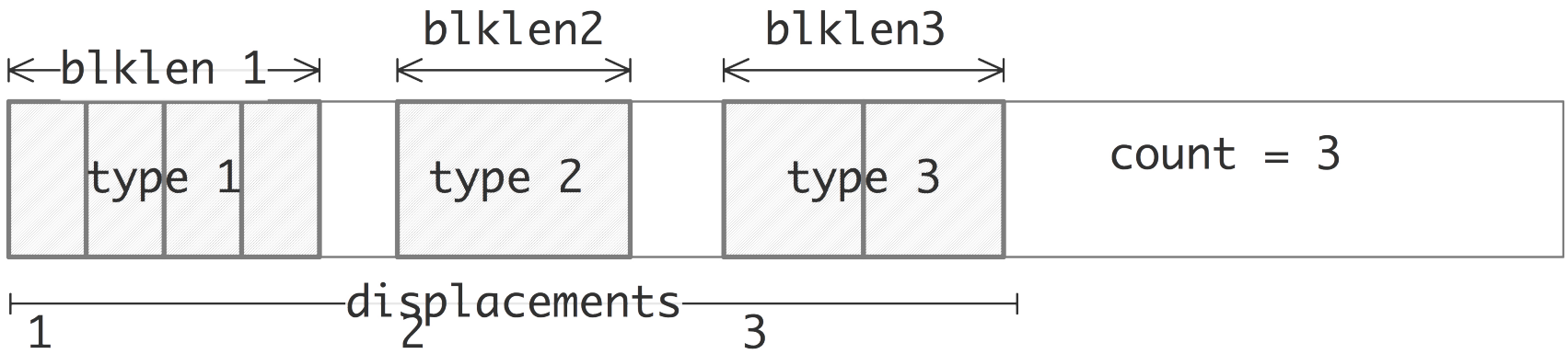
Example usage scenario: send linked list.
Use `MPI_Get_address`

MPI_Get_address

Name	Param name	Explanation	C type	F type
MPI_Get_address (				
	location	location in caller memory	const void*	TYPE(*), DIMENSION(..)
	address	address of location	MPI_Aint*	INTEGER (KIND=MPI_ADDRESS_KIND)
	)			

Heterogeneous: Structure type

```
1 int MPI_Type_create_struct(
2   int count, int blocklengths[], MPI_Aint displacements[],
3   MPI_Datatype types[], MPI_Datatype *newtype);
```



This gets very tedious. . .

Packed data

Packing into buffer

- Construct a buffer with `MPI_Pack`
- Send with `MPI_Send` and such
- Receive
- Unpack buffer elements with `MPI_Unpack`

MPI_Pack

Name	Param name	Explanation	C type	F type
MPI_Pack (				
MPI_Pack_c (				
	inbuf	input buffer start	const void*	TYPE(*), DIMENSION(..)
	incount	number of input data items	int MPI_Count	INTEGER
	datatype	datatype of each input data item	MPI_Datatype	TYPE(MPI_Datatype)
	outbuf	output buffer start	void*	TYPE(*), DIMENSION(..)
	outsize	output buffer size, in bytes	int MPI_Count	INTEGER
	position	current position in buffer, in bytes	int* MPI_Count*	INTEGER
	comm	communicator for packed message	MPI_Comm	TYPE(MPI_Comm)
	)			

MPI_Unpack

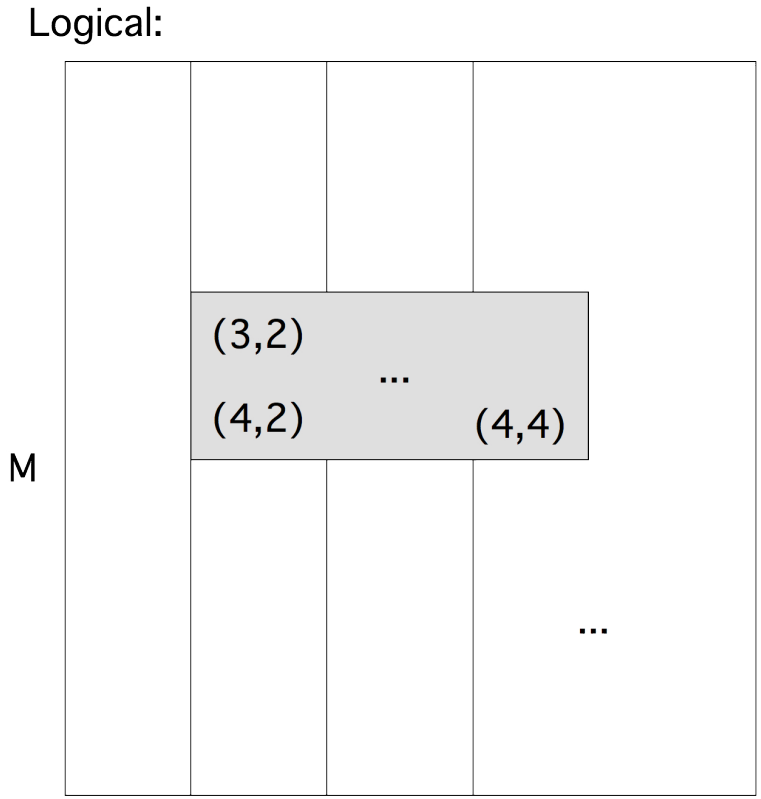
Name	Param name	Explanation	C type	F type
MPI_Unpack (				
MPI_Unpack_c (				
	inbuf	input buffer start	const void*	TYPE(*), DIMENSION(..)
	insize	size of input buffer, in bytes	int MPI_Count	INTEGER
	position	current position in bytes	int* MPI_Count*	INTEGER
	outbuf	output buffer start	void*	TYPE(*), DIMENSION(..)
	outcount	number of items to be unpacked	int MPI_Count	INTEGER
	datatype	datatype of each output data item	MPI_Datatype	TYPE(MPI_Datatype)
	comm	communicator for packed message	MPI_Comm	TYPE(MPI_Comm)
	)			

Example

```
1  if (procno==sender) {
2    position = 0;
3    MPI_Pack(&nsends,1,MPI_INT,
4             buffer,buflen,&position,comm);
5    for (int i=0; i<nsends; i++) {
6      double value = rand()/(double)RAND_MAX;
7      printf("[%d] pack %e\n",procno,value);
8      MPI_Pack(&value,1,MPI_DOUBLE,
9               buffer,buflen,&position,comm);
10   }
11   MPI_Pack(&nsends,1,MPI_INT,
12            buffer,buflen,&position,comm);
13   MPI_Send(buffer,position,MPI_PACKED,other,0,comm);
14 } else if (procno==receiver) {
15   int irecv_value;
16   double xrecv_value;
17   MPI_Recv(buffer,buflen,MPI_PACKED,other,0,
18            comm,MPI_STATUS_IGNORE);
19   position = 0;
20   MPI_Unpack(buffer,buflen,&position,
21              &nsends,1,MPI_INT,comm);
22   for (int i=0; i<nsends; i++) {
23     MPI_Unpack(buffer,buflen,
24                &position,&xrecv_value,1,MPI_DOUBLE,comm);
25     printf("[%d] unpack %e\n",procno,xrecv_value);
26   }
27   MPI_Unpack(buffer,buflen,&position,
28              &irecv_value,1,MPI_INT,comm);
29   ASSERT(irecv_value==nsends);
30 }
```

Subarray type

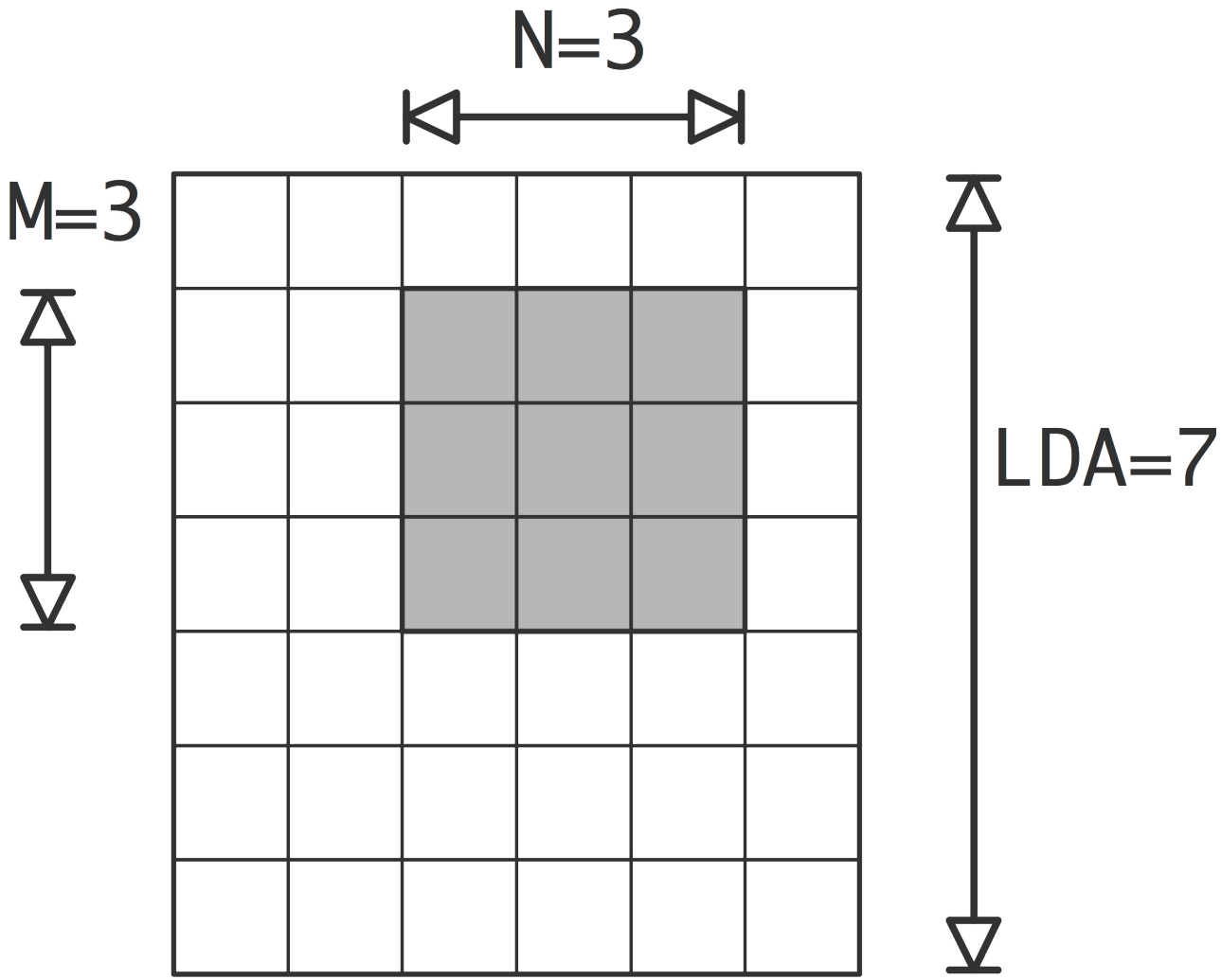
Submatrix storage



- Location of first element
- Stride, blocksize

BLAS/Lapack storage

Three parameter description:



How about as a 'block within a block'?

TACC
THE UNIVERSITY OF TEXAS AT AUSTIN

Subarray type

- Vector type is convenient for 2D subarrays,
- it gets tedious in higher dimensions.
- Better solution: `MPI_Type_create_subarray`

```
1 MPI_Type_create_subarray(
2     ndims, array_of_sizes, array_of_subsizes,
3     array_of_starts, order, oldtype, newtype)
```

Subtle: data does not start at the buffer start

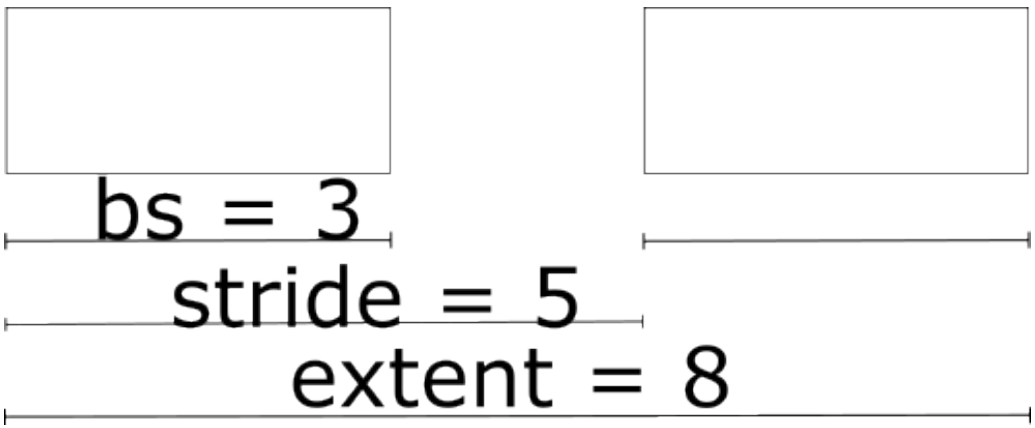
Exercise 33 `cubegather`

Assume that your number of processors is $P = Q^3$, and that each process has an array of identical size. Use `MPI_Type_create_subarray` to gather all data onto a root process. Use a sequence of send and receive calls; `MPI_Gather` does not work here.

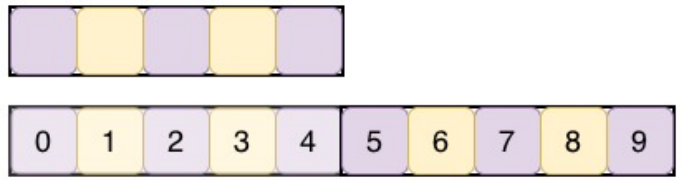
Extent and resizing

Extent

Extent: 'size' of a type,
especially useful for derived types.



Naive code



Send multiple derived types:

```
1 // vectorpadsend.c
2 int count = 3, stride = 2, blocklength = 1, ntypes = 2;
3   for (int i=0; i<max_elements; i++) sendbuffer[i] = i;
4   MPI_Type_vector(count,blocklength,stride,MPI_INT,&stridetype);
5   MPI_Type_commit(&stridetype);
6   MPI_Send( sendbuffer,ntypes,stridetype, receiver,0, comm );
```

Receive as single block:

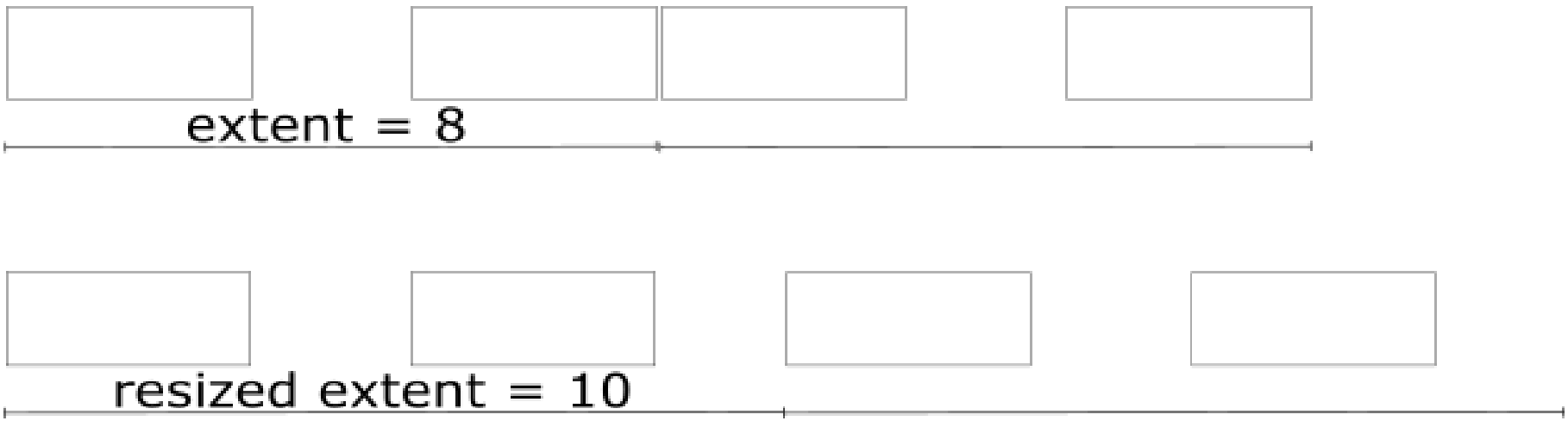
```
1 MPI_Recv( recvbuffer,max_elements,MPI_INT, sender,0, comm,&status );
2 int count; MPI_Get_count(&status,MPI_INT,&count);
3 printf("Receive %d elements:",count);
4 for (int i=0; i<count; i++) printf(" %d",recvbuffer[i]);
5 printf("\n");
```

giving an output of:

```
Receive 6 elements: 0 2 4 5 7 9
```

Extent resizing: enlarging

Multiple derived types may not be what you intended
extent resizing makes it artificually larger:



Extent computation

Use `MPI_Type_get_extent` to query extent

note: parameters are measured in bytes.

```
1 MPI_Aint lb,asize;
2 MPI_Type_vector(count,bs,stride,MPI_DOUBLE,&newtype);
3 MPI_Type_commit(&newtype);
4 MPI_Type_get_extent(newtype,&lb,&asize);
5 ASSERT( lb==0 );
6 ASSERT( asize==((count-1)*stride+bs)*sizeof(double) );
7 MPI_Type_free(&newtype);
```

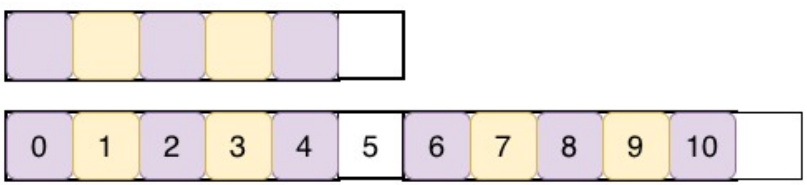
Resizing code

Extend the vector type with padding:

```
1 MPI_Type_get_extent(stridetype,&l,&e);
2 printf("Stride type l=%ld e=%ld\n",l,e);
3 e += ( stride-blocklength) * sizeof(int);
4 MPI_Type_create_resized(stridetype,l,e,&paddedtype);
5 MPI_Type_get_extent(paddedtype,&l,&e);
6 printf("Padded type l=%ld e=%ld\n",l,e);
7 MPI_Type_commit(&paddedtype);
8 MPI_Send( sendbuffer,ntypes,paddedtype, receiver,0, comm );
```

giving:

```
Strided type l=0 e=20
Padded  type l=0 e=24
Receive 6 elements: 0 2 4 6 8 10
```

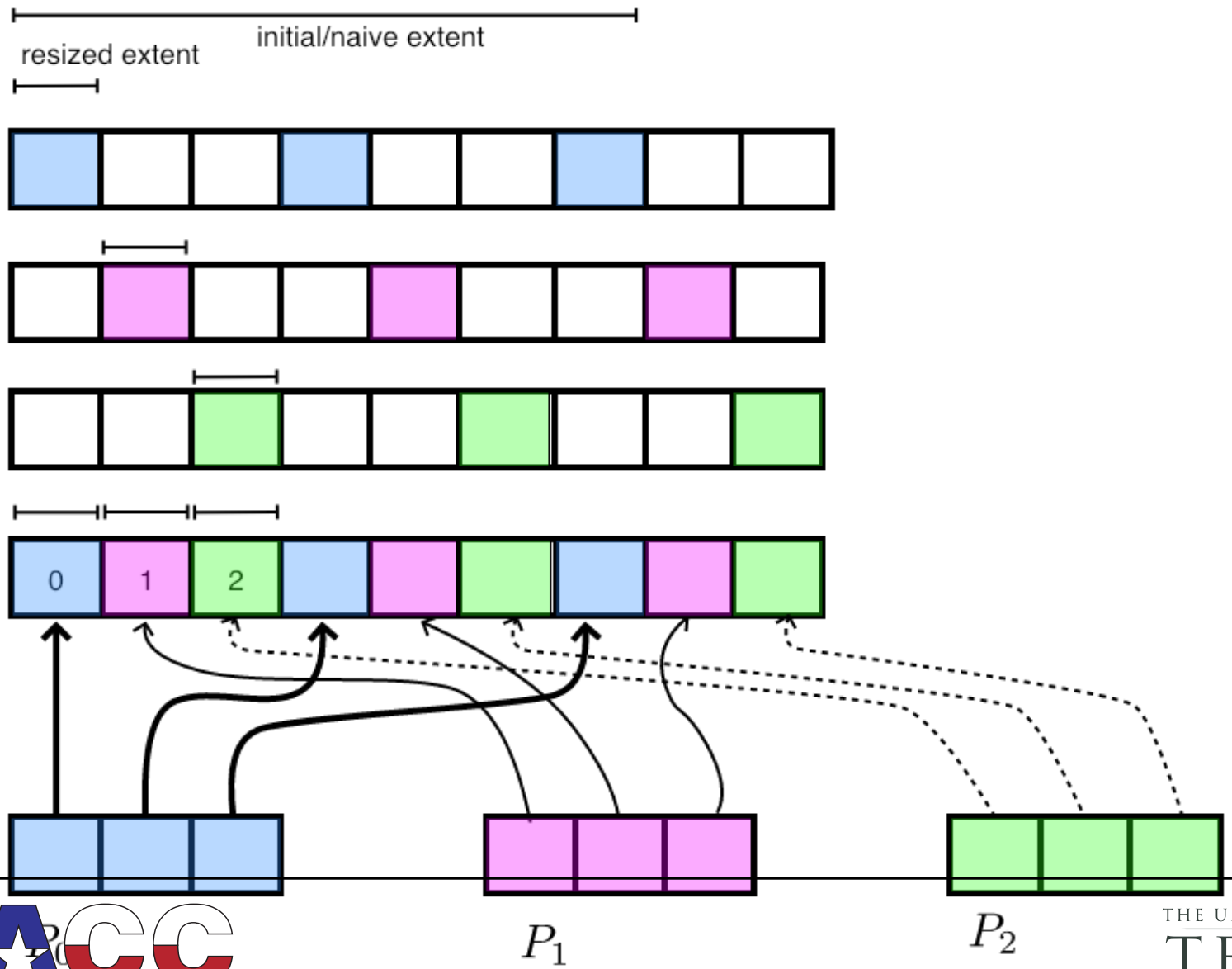


MPI_Type_create_resized

Name	Param name	Explanation	C type	F type
MPI_Type_create_resized (				
MPI_Type_create_resized_c (				
	oldtype	input datatype	MPI_Datatype	TYPE(MPI_Datatype)
	lb	new lower bound of datatype	[MPI_Aint MPI_Count	INTEGER (KIND=MPI_ADDRESS_KIND)
	extent	new extent of datatype	[MPI_Aint MPI_Count	INTEGER (KIND=MPI_ADDRESS_KIND)
	newtype	output datatype	MPI_Datatype*	TYPE(MPI_Datatype)
	)			

Extent resizing: shrinking

Elements are placed at distance equal to extent:



Exercise 34 `stridescatter`

Change the `stridesend` code to use a scatter call, rather than a sequence of sends.

Extent of subarray type

The 'subarray' type:
data does not start at the start of the type.

`MPI_Type_get_true_extent` (figure 1)

Figure 1 `MPI_Type_get_true_extent`

Name	Param name	Explanation	C type	F type
`MPI_Type_get_true_extent (`				
`MPI_Type_get_true_extent_c (`				
	`datatype`	`datatype to get information on`	`MPI_Datatype`	`TYPE (MPI_Dataty`
	`true_lb`	`true lower bound of datatype`	`MPI_Aint*` or `MPI_Count*`	`INTEGER (KIND=MPI_A`
	`true_extent`	`true extent of datatype`	`MPI_Aint*` or `MPI_Count*`	`INTEGER (KIND=MPI_A`
	`)`			

returns non-zero lower bound.

Transposition

- Basic block of FFT
- Before: Each process stores a block column,
- After: Each process stores a block row

2.2. Communicator manipulations

Overview

In this section you will learn about various subcommunicators.

Commands learned:

- `MPI_Comm_dup`, discussion of library design
- `MPI_Comm_split`
- discussion of groups
- discussion of inter/intra communicators.

Sub-computations

Simultaneous groups of processes, doing different tasks, but loosely interacting:

- Simulation pipeline: produce input data, run simulation, post-process.
- Climate model: separate groups for air, ocean, land, ice.
- Quicksort: split data in two, run quicksort independently on the halves.
- Process grid: do broadcast in each column.

New communicators are formed recursively from `MPI_COMM_WORLD`.

Communicator duplication

Simplest new communicator: identical to a previous one.

```
1 int MPI_Comm_dup(MPI_Comm comm, MPI_Comm *newcomm)
```

This is useful for library writers:

```
1 MPI_Isend(...); MPI_Irecv(...);
2 // library call
3 MPI_Waitall(...);
```

- Naively, the library can 'catch' the user messages.
- With a duplicate communicator there is no confusion:
  user and library both have their own 'context' for their messages.

Interleaved library and user code

```
1 library my_library(comm);
2 MPI_Isend(&sdata,1,MPI_INT,other,1,comm,&(request[0]));
3 my_library.communication_start();
4 MPI_Irecv(&rdata,1,MPI_INT,other,MPI_ANY_TAG,
5             comm,&(request[1]));
6 MPI_Waitall(2,request,status);
7 my_library.communication_end();
```

Library internally has messages

```
1  int library::communication_start() {
2    int sdata=6,rdata;
3    MPI_Isend(&sdata,1,MPI_INT,other,2,comm,&(request[0]));
4    MPI_Irecv(&rdata,1,MPI_INT,other,MPI_ANY_TAG,
5              comm,&(request[1]));
6    return 0;
7  }
8
9  int library::communication_end() {
10   MPI_Status status[2];
11   MPI_Waitall(2,request,status);
12   return 0;
13 }
```

Wrong way of setting up the library

```
1  // commdupwrong.cxx
2  class library {
3  private:
4    MPI_Comm comm;
5    int procno,nprocs,other;
6    MPI_Request request[2];
7  public:
8    library(MPI_Comm incomm) {
9      comm = incomm;
10     MPI_Comm_rank(comm,&procno);
11     other = 1-procno;
12   };
13   int communication_start();
14   int communication_end();
15 };
```

Right way of setting up the library

```
1  // commdupright.cxx
2  class library {
3  private:
4    MPI_Comm comm;
5    int procno,nprocs,other;
6    MPI_Request request[2];
7  public:
8    library(MPI_Comm incomm) {
9      MPI_Comm_dup(incomm,&comm);
10     MPI_Comm_rank(comm,&procno);
11     other = 1-procno;
12   };
13   ~library() {
14     MPI_Comm_free(&comm);
15   }
16   int communication_start();
17   int communication_end();
18 };
```

Disjoint splitting

Split a communicator in multiple disjoint others.

- Give each process a 'color':
  color parameter
- For each color: a new communicator of those processes
- Rank ordering determined by *key* parameter
- Usually you want rank ordering to stay intact:
  use *key*=rank or *key*=0 (tie-breaking)
  key parameter can be used for special effects

MPI_Comm_split

Name	Param name	Explanation	C type	F type
MPI_Comm_split (				
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	color	control of subset assignment	int	INTEGER
	key	control of rank assignment	int	INTEGER
	newcomm	new communicator	MPI_Comm*	TYPE(MPI_Comm)
	)			

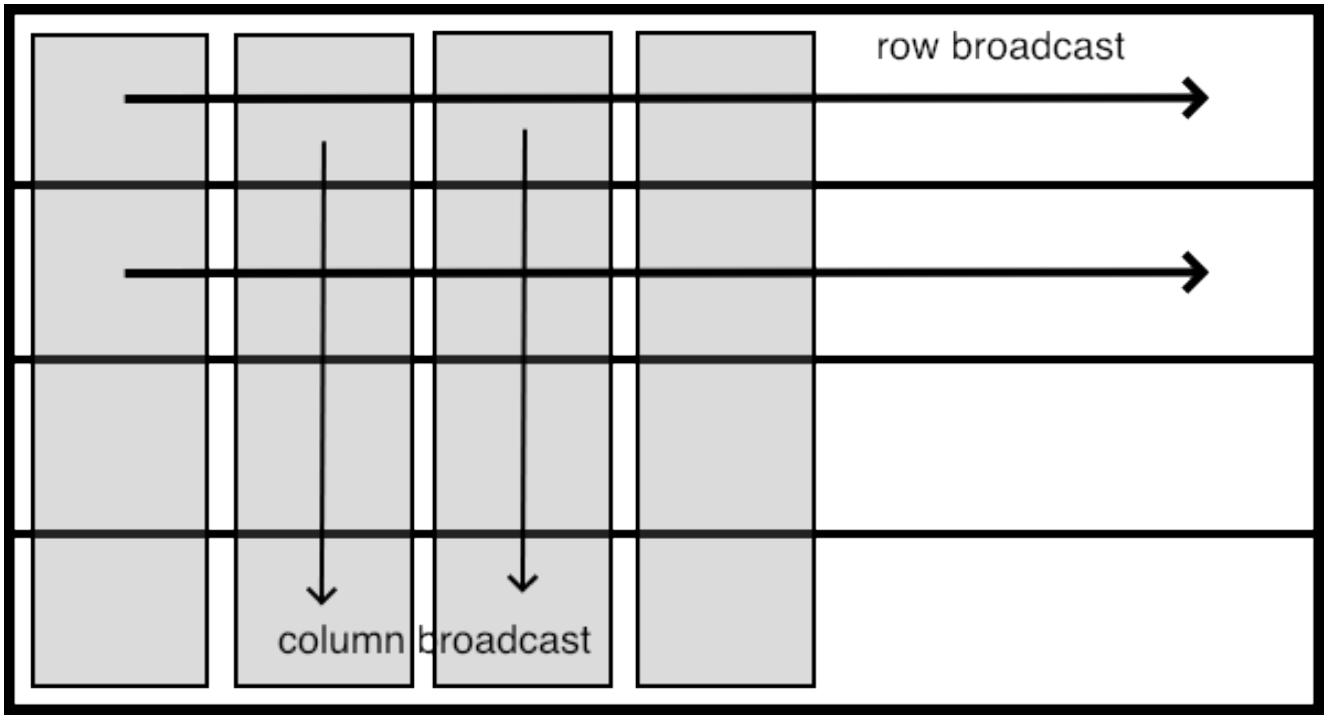
Row/column example

Simulate a process grid
create subcommunicator per column (or row)
assume processes numbered by rows

```
1 MPI_Comm_rank( MPI_COMM_WORLD, &procno );
2 proc_i = procno % proc_column_length;
3 proc_j = procno / proc_column_length;
4
5 MPI_Comm column_comm;
6 MPI_Comm_split( MPI_COMM_WORLD, proc_j, procno, &column_comm );
7
8 MPI_Bcast( data, ... column_comm );
```

Food for thought: there are many columns, but only one `column_comm` variable. Why?

Row and column communicators



Row and column broadcasts in subcommunicators

Exercise 35 `procgrid`

Organize your processes in a grid, and make subcommunicators for the rows and columns. For this compute the row and column number of each process.

In the row and column communicator, compute the rank. For instance, on a 2×3 processor grid you should find:

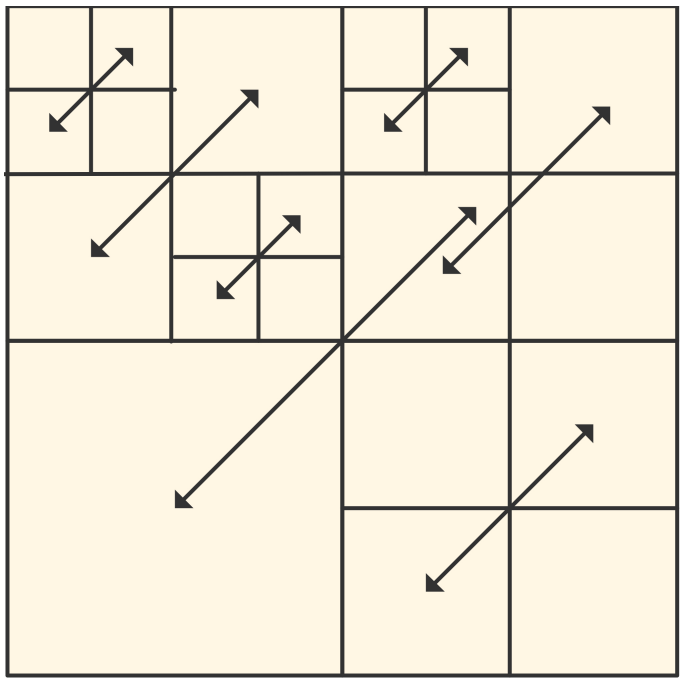
```
1 Global ranks:  Ranks in row:  Ranks in colum:
2   0  1  2      0  1  2        0  0  0
3   3  4  5      0  1  2        1  1  1
```

Check that the rank in the row communicator is the column number, and the other way around.

Run your code on different number of processes, for instance a number of rows and columns that is a power of 2, or that is a prime number. This is one occasion where you could use `ibrun -np 9`; normally you would *never* put a processor count on `ibrun`.

Exercise 36

Implement a recursive algorithm for matrix transposition, where each process stores one matrix element initially.



- Swap blocks $(1, 2)$ and $(2, 1)$; then
- Divide the processes into four subcommunicators, and apply this algorithm recursively on each;
- If the communicator has only one process, transpose the matrix in place.

Implement a gather to print out the final result.

Splitting by shared memory

- `MPI_Comm_split_type` splits into communicators of same type.
- MPI-3: only `MPI_COMM_TYPE_SHARED` splitting by shared memory.
- MPI-4: `MPI_COMM_TYPE_HW_GUIDED` split using an `info` value from `MPI_Get_hw_resource_types`.

```
1  // commsplittype.c
2  MPI_Info info;
3  MPI_Comm_split_type
4    (MPI_COMM_WORLD,
5     MPI_COMM_TYPE_SHARED,
6     procno,info,&sharedcomm);
7  MPI_Comm_size
8    (sharedcomm,&new_nprocs);
9  MPI_Comm_rank
10   (sharedcomm,&new_procno);
```

More

- Non-disjoint subcommunicators through process groups

  `MPI_Comm_group`, `MPI_Comm_create`
- Process topologies:
  cartesian, section 2,
  and graph, section 4.

Cartesian topologies

Multi-D process ordering

Processes have a linear number;
can we have a multi-dimensional number?

- 2D is easy (how?)
- More generally we need a function: `MPI_Dims_create`

Cartesian decomposition

Code:

```
1 // cartdims.c
2 int *dimensions = (int*) malloc(dim*
      sizeof(int));
3 for (int idim=0; idim<dim; idim++)
4   dimensions[idim] = 0;
5 MPI_Dims_create(nprocs,dim,dimensions);
```

Output:

```
1 mpicc -o cartdims
      ↪cartdims.o
2 Cartesian grid size: 3
      ↪dim: 1
3  3
4 Cartesian grid size: 3
      ↪dim: 2
5  3 x  1
6 Cartesian grid size: 4
      ↪dim: 1
7  4
8 Cartesian grid size: 4
      ↪dim: 2
9  2 x  2
10 Cartesian grid size: 4
      ↪dim: 3
11  2 x  2 x  1
12 Cartesian grid size: 12
      ↪dim: 1
13 12
14 Cartesian grid size: 12
      ↪dim: 2
15  4 x  3
```


TACC
THE UNIVERSITY OF TEXAS AT AUSTIN

Create/test Cartesian topology

```
MPI_Comm cart_comm;
int *periods = (int*) malloc(dim*sizeof(int));
for ( int id=0; id<dim; id++ ) periods[id] = 0;
MPI_Cart_create
  ( comm,dim,dimensions,periods,
    0,&cart_comm );
```

```
int dim;
MPI_Cartdim_get( cart_comm,&dim );
int *dimensions = (int*) malloc(dim*sizeof(int));
int *periods    = (int*) malloc(dim*sizeof(int));
int *coords     = (int*) malloc(dim*sizeof(int));
MPI_Cart_get( cart_comm,dim,dimensions,periods,coords );
```

You don't need to use all procs
Example: 2×2 grid for 5 procs,
last one will get `MPI_COMM_NULL` as cartesian communicator output

Rank translation

```
1  // cart.c
2  MPI_Comm comm2d;
3  int periodic[ndim]; periodic[0] = periodic[1] = 0;
4  MPI_Cart_create(comm,ndim,dimensions,periodic,1,&comm2d);
5  if (comm2d==MPI_COMM_NULL) {
6    printf("Process %d not included\n",procno);
7  } else {
8    MPI_Cart_coords(comm2d,procno,ndim,coord_2d);
9    MPI_Cart_rank(comm2d,coord_2d,&rank_2d);
10   printf("I am %d: (%d,%d); originally %d\n",
11            rank_2d,coord_2d[0],coord_2d[1],procno);
```

Cartesian communication

```
// cartcoord.c
for ( int id=0; id<dim; id++)
  periods[id] = id==0 ? 1 : 0;
MPI_Cart_create
  ( comm,dim,dimensions,periods,
    0,&period_comm );
```

Code:

```
int pred,succ;
MPI_Cart_shift
  (period_comm,/* dim: */ 0,/* up: */ 1,
   &pred,&succ);
printf
  ("periodic dimension 0:\n  src=%d, tgt
     =%d\n",
   pred,succ);
MPI_Cart_shift
  (period_comm,/* dim: */ 1,/* up: */ 1,
   &pred,&succ);
printf
  ("non-periodic dimension 1:\n  src=%d,
     tgt=%d\n",
   pred,succ);
```

Output:

```
Grid of size 6 in 3
    ↪dimensions:
   3 x  2 x  1
Shifting process 0.
periodic dimension 0:
  src=4, tgt=2
non-periodic dimension
    ↪1:
  src=-1, tgt=1
```

Subgrids

Code:

```
1  MPI_Cart_sub( period_comm,remain,&
       hyperplane );
2  if (procno==0) {
3    MPI_Topo_test( hyperplane,&topo_type );
4    MPI_Cartdim_get( hyperplane,&hyperdim )
       ;
5    printf("hyperplane has dimension %d,
       type %d\n",
6           hyperdim,topo_type);
7    MPI_Cart_get( hyperplane,dim,dims,
       period,coords );
8    printf("  periodic: ");
9    for (int id=0; id<2; id++)
10     printf("%d,",period[id]);
11   printf("\n");
```

Output:

```
1  Grid of size 6 in 3
       ↪dimensions:
2     3 x  2 x  1
3  hyperplane has
       ↪dimension 2,
       ↪type 2
4    periodic: 1,0,
```

Exercise 37 `isendirecvcart`

Use Cartesian topology routines to extend exercise 28 to two dimensions.

Inter-communicators

Inter-communicators

- Communicators so far are of *intra-communicator* type.
- Bridge between two communicators: *inter-communicator*.
- Example: communicator with newly spawned processes

In a picture

Illustration of ranks in an inter-communicator setup

```
1 // intercomm.c
2 MPI_Comm intercomm;
3 MPI_Intercomm_create
4   (/* local_comm:      */ split_half_comm,
5    /* local_leader:    */ local_leader_in_inter_comm,
6    /* peer_comm:       */ MPI_COMM_WORLD,
7    /* remote_peer_rank: */ global_rank_of_other_leader,
8    /* tag:             */ inter_tag,
9    /* newintercomm:    */ &intercomm );
```

Concepts

- Two local communicators
- The ‘peer’ communicator that contains them
- Leaders in each of them
- An inter-communicator over the leaders.

Routines

- `MPI_Intercomm_create`: create
- `MPI_Comm_get_parent`: the other leader (see process management)
- `MPI_Comm_remote_size`, `MPI_Comm_remote_group`: query the other communicator
- `MPI_Comm_test_inter`: is this an inter or intra?

2.3. MPI File I/O

Overview

This section discusses parallel I/O. What is the problem with regular I/O in parallel?

Commands learned:

- `MPI_File_open`/*write*/*close* and variants
- parallel file pointer routines: `MPI_File_set_view`/*write_at*

The trouble with parallel I/O

- Multiple process reads from one file: no problem.
- Multiple writes to one file: big problem.
- Everyone writes to separate file: stress on the file system, and requires post-processing.

MPI I/O

- Part of MPI since MPI-2
- Joint creation of one file from bunch of processes.
- You could also use `hdf5`, `netcdf`, `silo` ...

The usual bits

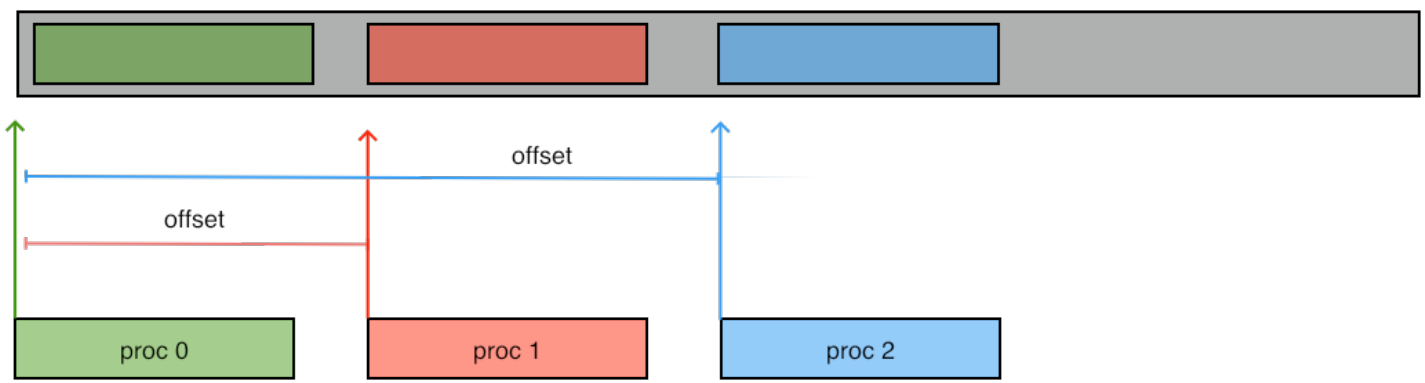
```
MPI_File mpifile;
MPI_File_open(comm,"blockwrite.dat",
        MPI_MODE_CREATE | MPI_MODE_WRONLY,MPI_INFO_NULL,
        &mpifile);
if (procno==0) {
  MPI_File_write
    (mpifile,output_data,nwords,MPI_INT,MPI_STATUS_IGNORE);
}
MPI_File_close(&mpifile);
```

```
type(MPI_File) :: mpifile ! F08
integer        :: mpifile ! F90
```

Prevent write conflicts 1: offset

Each process uses unique offset:

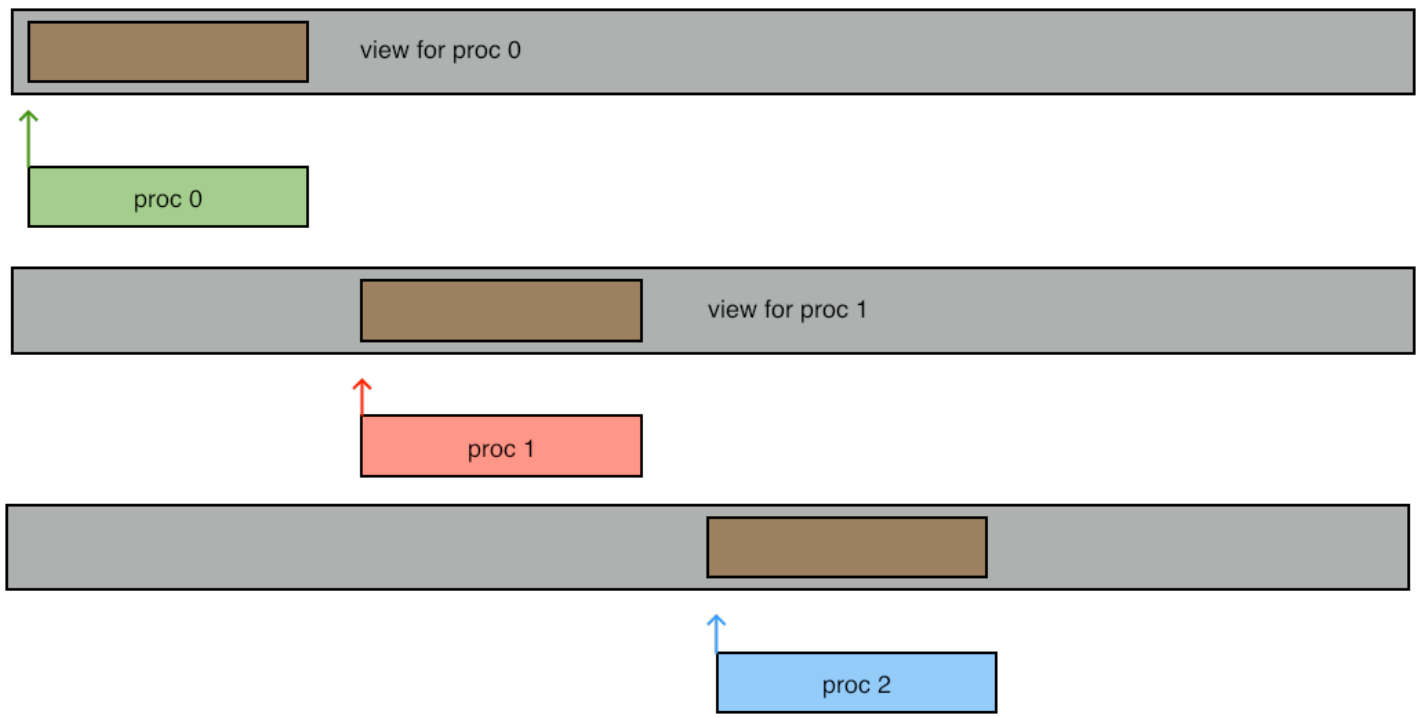
```
1 MPI_File_write_at
2    (mpifile,offset,output_data,nwords,
3     MPI_INT,MPI_STATUS_IGNORE);
```



Prevent write conflicts 2: view

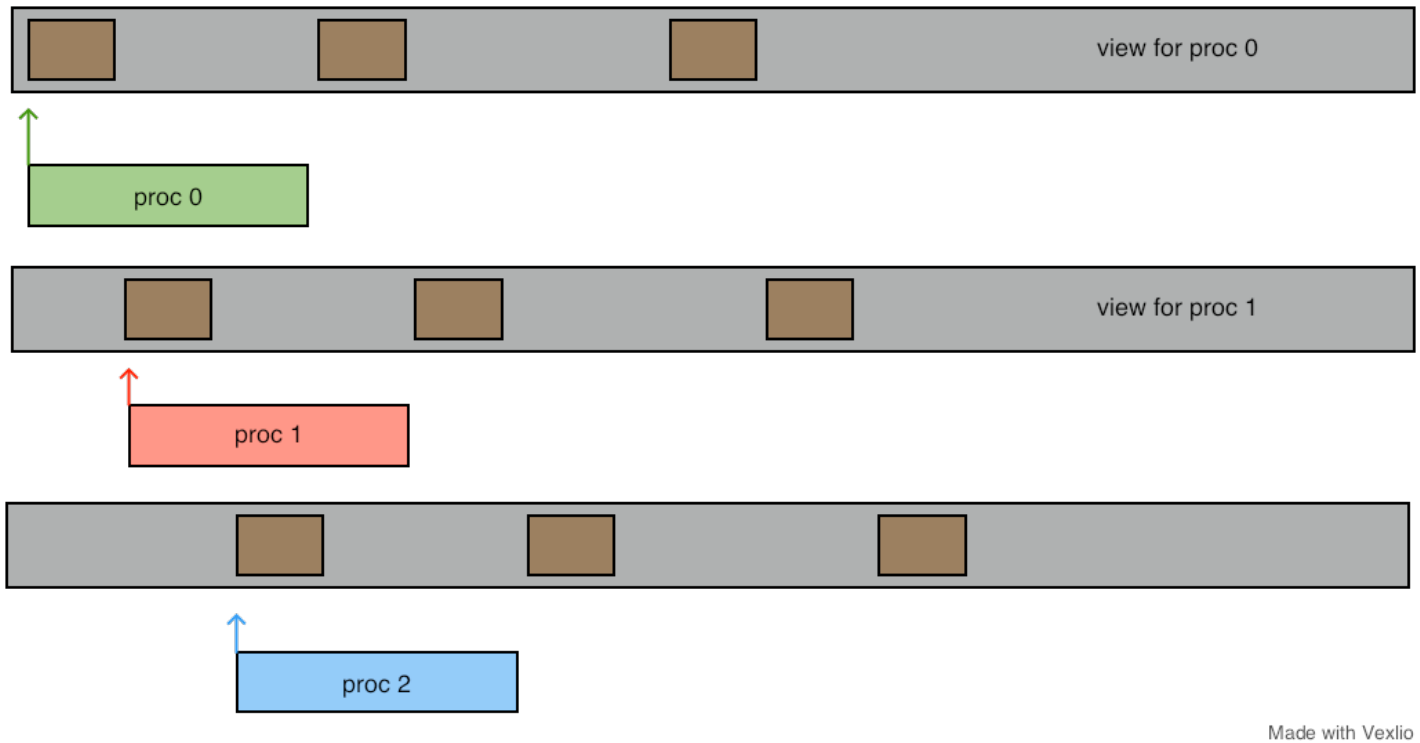
Each process gets unique 'view' into the file:

```
1 MPI_File_set_view
2   (mpifile,
3    offset,datatype,
4    MPI_INT,"native",MPI_INFO_NULL);
5 MPI_File_write // no offset, we have a view
6   (mpifile,output_data,nwords,MPI_INT,MPI_STATUS_IGNORE);
```



Write to a view, derived type

Complicated effects can be achieved with derived types:



Exercise 38 `blockwrite`

The given code works for one writing process. Compute a unique offset for each process (in bytes!) so that all the local arrays are placed in the output file in sequence.

Exercise 39 `viewwrite`

Solve the previous exercise by using `MPI_File_write` (that is, without offset), but by using `MPI_File_set_view` to specify the location.

Exercise 40 `scatterwrite`

Now write the local arrays cyclically to the file: with 5 processes and 3 elements per process the file should contain

```
1 4 7 10 13 | 2 5 8 11 14 | 3 6 9 12 15
```

Do this by defining a vector derived type and setting that as the file view.

2.4. One-sided communication

Overview

This section concernes one-sided operations, which allows 'shared memory' type programming. (Actual shared memory later.)

Commands learned:

- `MPI_Put`, `MPI_Get`, `MPI_Accumulate`
- Window commands: `MPI_Win_create`, `MPI_Win_allocate`
- Active target synchronization `MPI_Win_fence`
- `MPI_Win_post`/`wait`/*`start`*/*`complete`*
- Passive target synchronization `MPI_Win_lock` / `MPI_Win_lock`
- Atomic operations: `MPI_Fetch_and_op`

Basic mechanisms

Motivation

With two-sided messaging, you can not just put data on a different process: the other has to expect it and receive it.

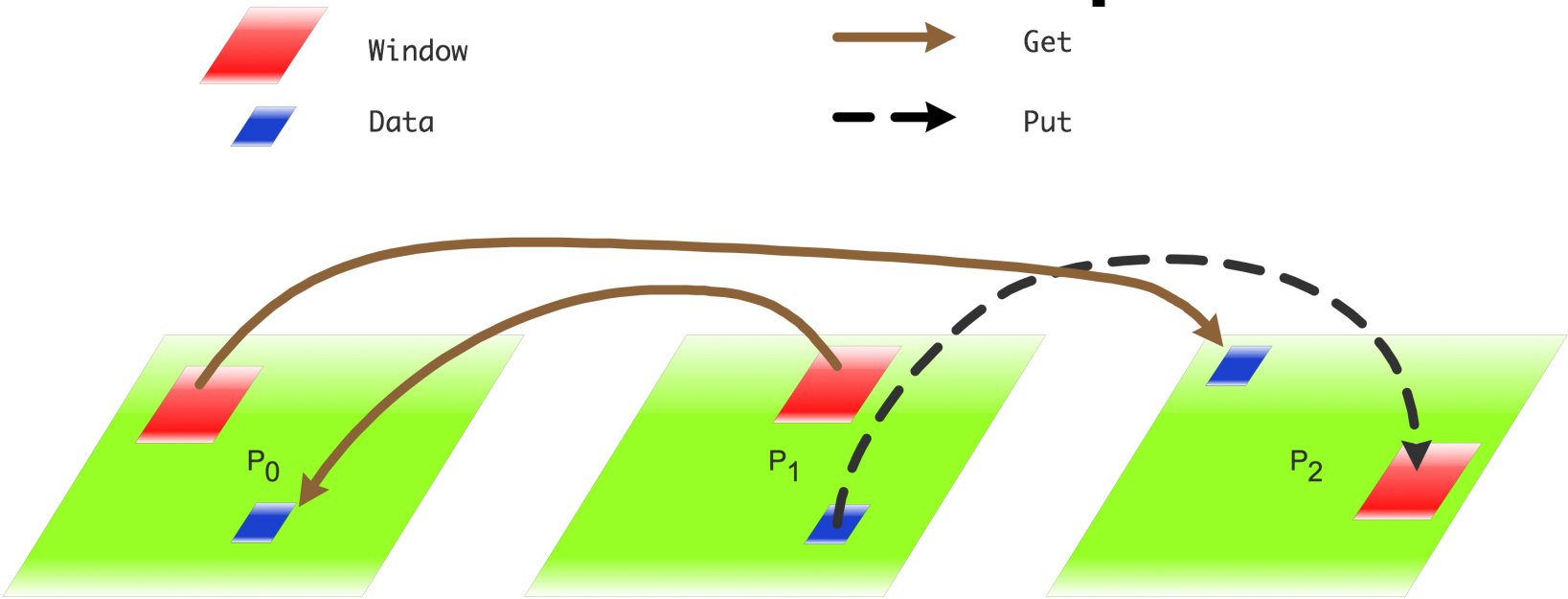
- Sparse matrix: it is easy to know what you are receiving, not what you need to send. Usually solved with complicated preprocessing step.
- Neuron simulation: spiking neuron propagates information to neighbors. Uncertain when this happens.
- Other irregular data structures: distributed hash tables.

Dynamic data

```
1 x = f();
2 p = hash(x);
3 MPI_Send( x, /* to: */ p );
```

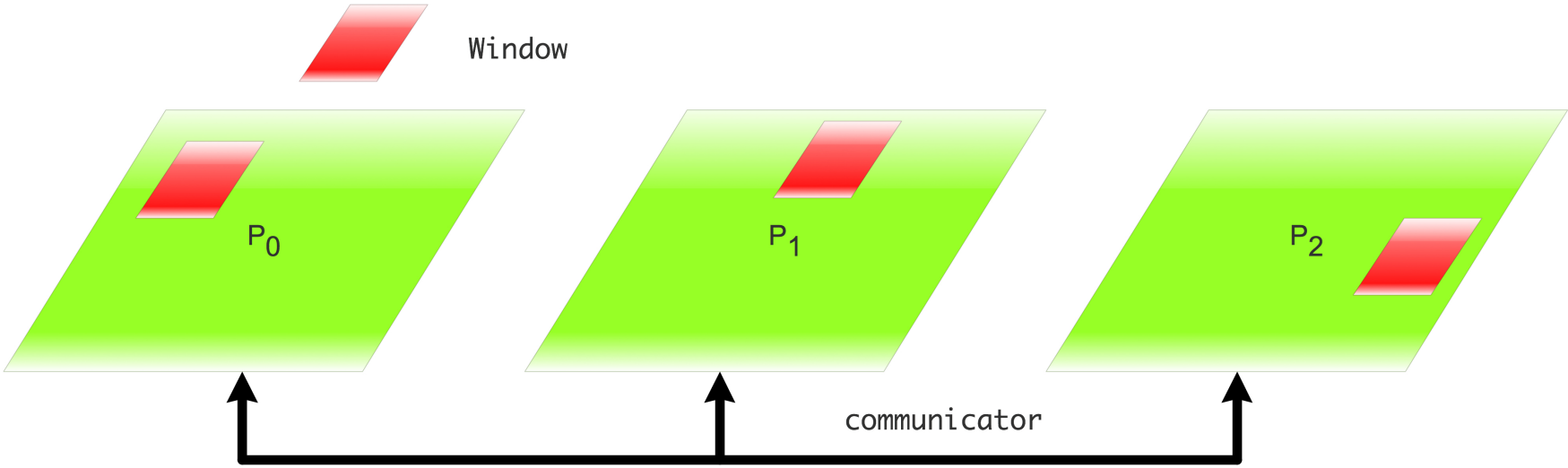
Problem: how does p know to post a receive, and how does everyone else know not to?

One-sided concepts



- A process has a *window* that other processes can access.
- *origin*: process doing a one-sided call
  target: process being accessed.
- One-sided calls: `MPI_Put`, `MPI_Get`, `MPI_Accumulate`.
- Various synchronization mechanisms.

Window creation




```
1 MPI_Win_create (void *base, MPI_Aint size,
2   int disp_unit, MPI_Info info, MPI_Comm comm, MPI_Win *win)
```

- `size`: in bytes
- *disp_unit*: `sizeof(`*type*`)`

Also call `MPI_Win_free` when done. This is important!

Window allocation

Instead of passing buffer, let MPI allocate with `MPI_Win_allocate` and return the buffer pointer:

```
1 int MPI_Win_allocate
2    (MPI_Aint size, int disp_unit, MPI_Info info,
3     MPI_Comm comm, void *baseptr, MPI_Win *win)
```

can use dedicated fast memory.

Active target synchronization

All processes call `MPI_Win_fence`. Epoch is between fences:

```
1 MPI_Win_fence(MPI_MODE_NOPRECEDE, win);
2 if (procno==producer)
3   MPI_Put( /* operands */, win);
4 MPI_Win_fence(MPI_MODE_NOSUCCEED, win);
```

Second fence indicates that one-sided communication is concluded: target knows that data has been put.

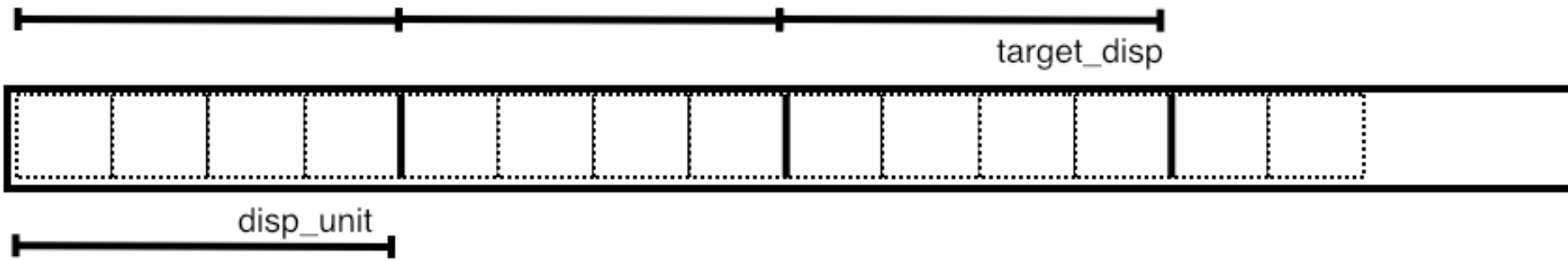
MPI_Put

Name	Param name	Explanation	C type	F type
MPI_Put (				
MPI_Put_c (				
	origin_addr	initial address of origin buffer	const void*	TYPE(*), DIMENSION(..)
	origin_count	number of entries in origin buffer	[int MPI_Count	INTEGER
	origin_datatype	datatype of each entry in origin buffer	MPI_Datatype	TYPE(MPI_Datatype)
	target_rank	rank of target	int	INTEGER
	target_disp	displacement from start of window to target buffer	MPI_Aint	INTEGER (KIND=MPI_ADDRESS_KIND
	target_count	number of entries in target buffer	[int MPI_Count	INTEGER
	target_datatype	datatype of each entry in target buffer	MPI_Datatype	TYPE(MPI_Datatype)
	win	window object used for communication	MPI_Win	TYPE(MPI_Win)
	)			

Location in the window

Location to write:

$$\texttt{window_base} + \texttt{target_disp} \times \texttt{disp_unit}.$$



Exercise 41 `rightput`

Revisit exercise 21 and solve it using `MPI_Put`.

Exercise 42 `randomput`

Write code where:

- process 0 computes a random number r
- if $r < .5$, zero writes in the window on 1;
- if $r \geq .5$, zero writes in the window on 2.

Exercise (optional) 43 (randomput)

Replace `MPI_Win_create` by `MPI_Win_allocate`.

Remaining simple routines: Get, Accumulate

- `MPI_Get` is converse of `MPI_Put`. Like Recv, but no status argument.
- `MPI_Accumulate` is a Put plus a reduction on the result: multiple accumulate calls in one epoch well-defined.
  Can use any predefined `MPI_Op` (not user-defined) or `MPI_REPLACE`.

MPI_Get

Name	Param name	Explanation	C type	F type
MPI_Get (				
MPI_Get_c (				
	origin_addr	initial address of origin buffer	void*	TYPE(*), DIMENSION(..)
	origin_count	number of entries in origin buffer	[int / MPI_Count	INTEGER
	origin_datatype	datatype of each entry in origin buffer	MPI_Datatype	TYPE(MPI_Datatype)
	target_rank	rank of target	int	INTEGER
	target_disp	displacement from window start to the beginning of the target buffer	MPI_Aint	INTEGER (KIND=MPI_ADDRESS_KIND
	target_count	number of entries in target buffer	[int / MPI_Count	INTEGER
	target_datatype	datatype of each entry in target buffer	MPI_Datatype	TYPE(MPI_Datatype)
	win	window object used for communication	MPI_Win	TYPE(MPI_Win)
	)			

MPI_Accumulate

Name	Param name	Explanation	C type	F type
MPI_Accumulate (				
MPI_Accumulate_c (				
	origin_addr	initial address of buffer	const void*	TYPE(*), DIMENSION(..)
	origin_count	number of entries in buffer	int MPI_Count	INTEGER
	origin_datatype	datatype of each entry	MPI_Datatype	TYPE(MPI_Datatype)
	target_rank	rank of target	int	INTEGER
	target_disp	displacement from start of window to beginning of target buffer	MPI_Aint	INTEGER (KIND=MPI_ADDRESS_KIND
	target_count	number of entries in target buffer	int MPI_Count	INTEGER
	target_datatype	datatype of each entry in target buffer	MPI_Datatype	TYPE(MPI_Datatype)
	op	reduce operation	MPI_Op	TYPE(MPI_Op)
	win	window object	MPI_Win	TYPE(MPI_Win)
	)			

Ordering and synchronization

Fence synchronization

Already mentioned active target synchronization:
the target indicates the start/end of an epoch.

Simplest mechanism: `MPI_Win_fence`, collective.

After the closing fence, buffers have been sent / windows have been updated.

Ordering of operations

Ordering is often undefined:

- No ordering of Get and Put/Accumulate operations
- No ordering of multiple Puts. Use Accumulate.

The following operations are well-defined inside one epoch:

- Instead of multiple Put operations, use Accumulate with `MPI_REPLACE`.
- `MPI_Get_accumulate` with `MPI_NO_OP` is safe.
- Multiple Accumulate operations from one origin are ordered by default.

Exercise (optional) 44 (`countdown`)

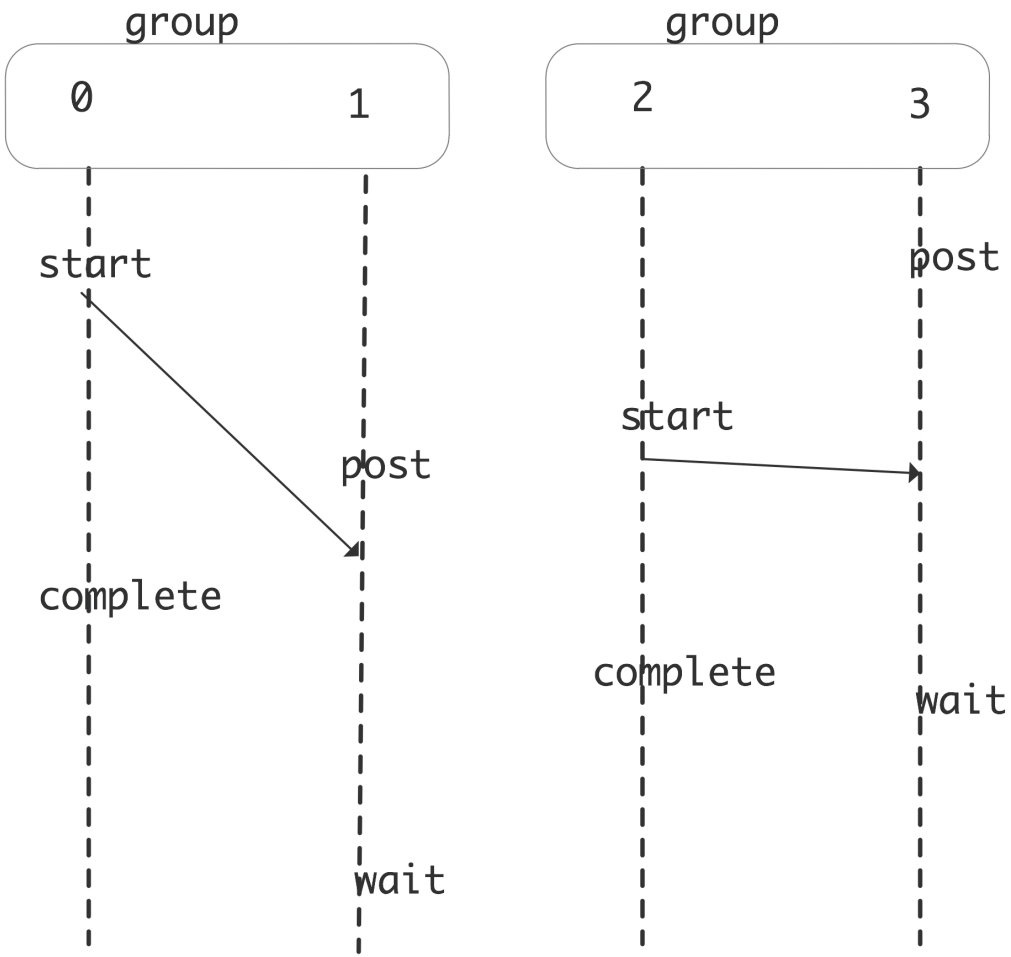
Implement a shared counter:

- One process maintains a counter;
- Iterate: all others at random moments update this counter.
- When the counter is no longer positive, everyone stops iterating.

The problem here is data synchronization: does everyone see the counter the same way?

A second active synchronization

Use `MPI_Win_post`, `MPI_Win_wait`, `MPI_Win_start`, `MPI_Win_complete` calls



More fine-grained than fences.

Passive target synchronization

Passive target synchronization

Lock a window on the target:

```
1 MPI_Win_lock
2    (int locktype, int rank, int assert, MPI_Win win)
3 MPI_Win_unlock
4    (int rank, MPI_Win win)
```

with types: `MPI_LOCK_SHARED` `MPI_LOCK_EXCLUSIVE`

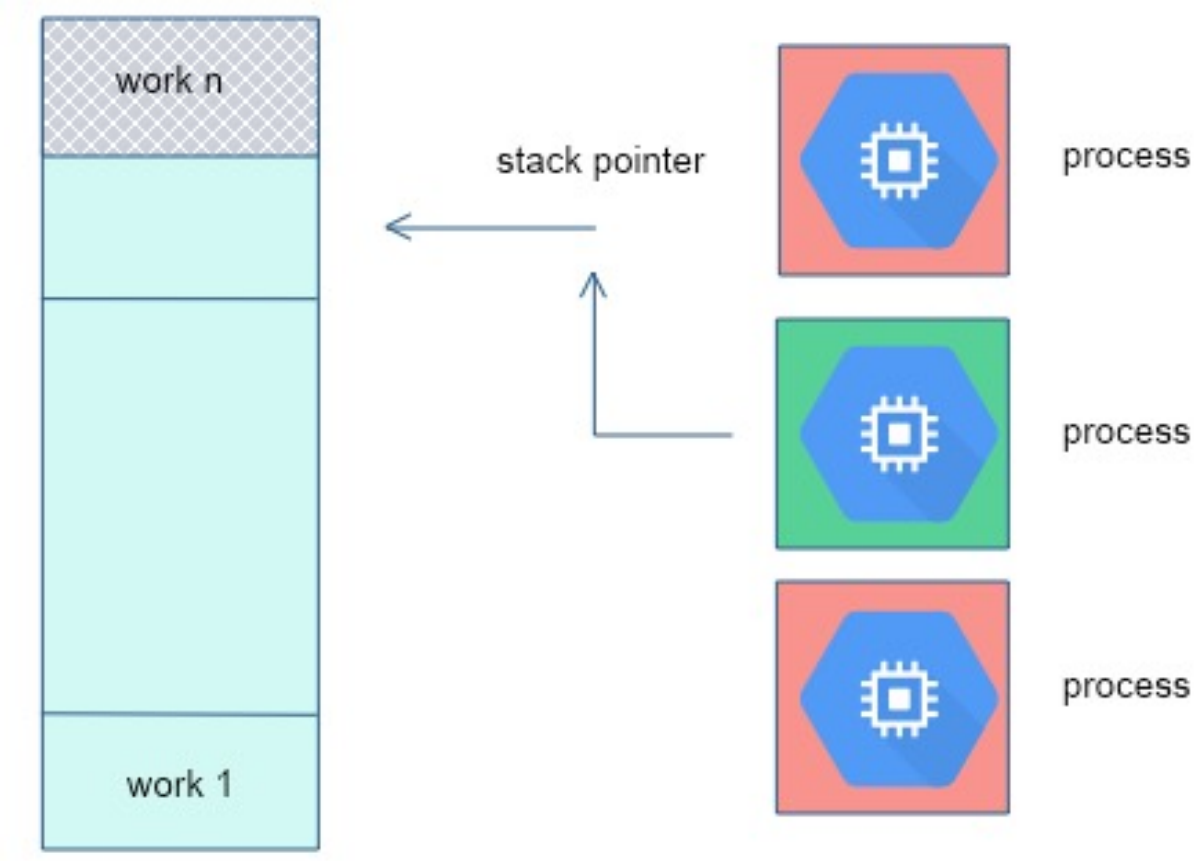
Justification

MPI-1/2 lacked tools for race condition-free one-sided communication.
These have been added in MPI-3.

Emulating shared memory with one-sided communication

- One process stores a table of work descriptors, and a 'stack pointer' stating how many there are.
- Each process reads the pointer, reads the corresponding descriptor, and decrements the pointer; and
- A process that has read a descriptor then executes the corresponding task.
- Non-collective behavior: processes only take a descriptor when they are available.

In a picture



Simplified model

- One process has a counter, which models the shared memory;
- Each process, if available, reads the counter; and
- . . . decrements the counter.
- No actual work: random decision if process is available.

Shared memory problems: what is a race condition?

Race condition: outward behavior depends on timing/synchronization of low-level events.
In shared memory associated with shared data.

Example:

Init: `I=0`
process 1: `I=I+2`
process 2: `I=I+3`

scenario 1.		scenario 2.		scenario 3.	
I = 0					
read I = 0	read I = 0	read I = 0	read I = 0	read I = 0	
local I = 2	local I = 3	local I = 2	local I = 3	local I = 2	
write I = 2			write I = 3	write I = 2	
	write I = 3	write I = 2			read I = 2
					local I = 5
					write I = 5
I = 3		I = 2		I = 5	

(In MPI, the read/write would be `MPI_Get` / `MPI_Put` calls)

Case study in shared memory: 1, wrong

```
1  // countdownput.c
2  MPI_Win_fence(0,the_window);
3  int counter_value;
4  MPI_Get( &counter_value,1,MPI_INT,
5           counter_process,0,1,MPI_INT,
6           the_window);
7  MPI_Win_fence(0,the_window);
8  if (i_am_available) {
9    int decrement = -1;
10   counter_value += decrement;
11   MPI_Put
12     ( &counter_value,   1,MPI_INT,
13       counter_process,0,1,MPI_INT,
14       the_window);
15 }
16 MPI_Win_fence(0,the_window);
```

Discussion

- The multiple `MPI_Put` calls conflict.
- Code is correct if in each iteration there is only one writer.
- Question: In that case, can we take out the middle fence?
- Question: what is wrong with

```
1 MPI_Win_fence(0,the_window);
2 if (i_am_available) {
3   MPI_Get( &counter_value, ... )
4   MPI_Win_fence(0,the_window);
5   MPI_Put( ... )
6 }
7 MPI_Win_fence(0,the_window);
```

?

Case study in shared memory: 2, hm

```
1  // countdownacc.c
2  MPI_Win_fence(0,the_window);
3  int counter_value;
4  MPI_Get( &counter_value,1,MPI_INT,
5           counter_process,0,1,MPI_INT,
6           the_window);
7  MPI_Win_fence(0,the_window);
8  if (i_am_available) {
9    int decrement = -1;
10   MPI_Accumulate
11     ( &decrement,       1,MPI_INT,
12       counter_process,0,1,MPI_INT,
13       MPI_SUM,
14       the_window);
15 }
16 MPI_Win_fence(0,the_window);
```

Discussion: need for atomics

- `MPI_Accumulate` is atomic, so no conflicting writes.
- What is the problem?
- Answer: Processes are not reading unique *`counter_value`* values.
- Conclusion: Read and update need to come together:
  read unique value and immediately update.

Atomic 'get-and-set-with-no-one-coming-in-between':
`MPI_Fetch_and_op` / `MPI_Get_accumulate`.
Former is simple version: scalar only.

MPI_Fetch_and_op

Name	Param name	Explanation	C type	F type
MPI_Fetch_and_op (				
	origin_addr	initial address of buffer	const void*	TYPE(*), DIMENSION(..)
	result_addr	initial address of result buffer	void*	TYPE(*), DIMENSION(..)
	datatype	datatype of the entry in origin, result, and target buffers	MPI_Datatype	TYPE(MPI_Datatype)
	target_rank	rank of target	int	INTEGER
	target_disp	displacement from start of window to beginning of target buffer	MPI_Aint	INTEGER (KIND=MPI_ADDRESS_KIND)
	op	reduce operation	MPI_Op	TYPE(MPI_Op)
	win	window object	MPI_Win	TYPE(MPI_Win)
	)			

Case study in shared memory: 3, good

```
1  MPI_Win_fence(0,the_window);
2  int
3    counter_value;
4  if (i_am_available) {
5    int
6      decrement = -1;
7    total_decrement++;
8    MPI_Fetch_and_op
9      ( /* operate with data from origin: */   &decrement,
10       /* retrieve data from target:    */   &counter_value,
11       MPI_INT, counter_process, 0, MPI_SUM,
12       the_window);
13 }
14 MPI_Win_fence(0,the_window);
15 if (i_am_available) {
16   my_counter_values[n_my_counter_values++] = counter_value;
17 }
```

Allowable operators. (Hint!)

MPI type	Python	meaning	applies to
	MPI.Op		
MPI_MAX	*MPI.MAX*	maximum	integer, floating point
MPI_MIN	*MPI.MIN*	minimum	
MPI_SUM	*MPI.SUM*	sum	integer, floating point, com
		multilanguage types	
MPI_PROD	*MPI.PROC*	product	
MPI_REPLACE	*MPI.REPLACE*	overwrite	
MPI_NO_OP	*MPI.OP_NULL*	no change	
MPI_LAND	*MPI.LAND*	logical and	C integer, logical
MPI_LOR	*MPI.LOR*	logical or	
MPI_LXOR	*MPI.LXOR*	logical xor	
MPI_BAND	*MPI.BAND*	bitwise and	integer, byte, multilanguage
MPI_BOR	*MPI.BOR*	bitwise or	
MPI_BXOR	*MPI.BXOR*	bitwise xor	
MPI_MAXLOC	*MPI.MAXLOC*	max value and location	MPI_DOUBLE_INT and such
MPI_MINLOC	*MPI.MINLOC*	min value and location	

No user-defined operators.

TACC
THE UNIVERSITY OF TEXAS AT AUSTIN

Problem

We are using fences, which are collective.
What if a process is still operating on its local work?

Better (but more tricky) solution:
use passive target synchronization and locks.

Passive target epoch

```
1 if (rank == 0) {
2   MPI_Win_lock (MPI_LOCK_EXCLUSIVE, 1, 0, win);
3   MPI_Put (outbuf, n, MPI_INT, 1, 0, n, MPI_INT, win);
4   MPI_Win_unlock (1, win);
5 }
```

No action on the target required!

Exercise 45 `lockfetch`

Investigate atomic updates using passive target synchronization. Use `MPI_Win_lock` with an exclusive lock, which means that each process only acquires the lock when it absolutely has to.

- All processs but one update a window:

```
1 int one=1;
2 MPI_Fetch_and_op(&one, &readout,
3     MPI_INT, repo, zero_disp, MPI_SUM,
4     the_win);
```

- while the remaining process spins until the others have performed their update.

Use an atomic operation for the latter process to read out the shared value.
Can you replace the exclusive lock with a shared one?

Exercise 46 `lockfetchshared`

As exercise 45, but now use a shared lock: all processes acquire the lock simultaneously and keep it as long as is needed.

The problem here is that coherence between window buffers and local variables is now not forced by a fence or releasing a lock. Use `MPI_Win_flush_local` to force coherence of a window (on another process) and the local variable from `MPI_Fetch_and_op`.

3. Advanced (MPI-3/4) topics

Justification

Recent additions to the MPI standard allow your code to deal with unusual scenarios or very large scale runs.

3.1. Advanced collectives

Non-blocking collectives

- Collectives are blocking.
- Compare blocking/non-blocking sends:

  `MPI_Send` $\longrightarrow$ `MPI_Isend`

  immediate return of control, produce request object.
- Non-blocking collectives:

  `MPI_Bcast` $\longrightarrow$ `MPI_Ibcast`

  Same:

```
1 MPI_Isomething( <usual arguments>, MPI_Request *req);
```

- Considerations:
  - Calls return immediately;
  - the usual story about buffer reuse
  - Requires `MPI_Wait...` for completion.
  - Multiple collectives can complete in any order
- Why?
  - Use for overlap communication/computation
  - Imbalance resilience
  - Allows pipelining

MPI_Ibcast

Name	Param name	Explanation	C type	F type
MPI_Ibcast (				
MPI_Ibcast_c (				
	buffer	starting address of buffer	void*	TYPE(*), DIMENSION(..)
	count	number of entries in buffer	int MPI_Count	INTEGER
	datatype	datatype of buffer	MPI_Datatype	TYPE(MPI_Datatype)
	root	rank of broadcast root	int	INTEGER
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	request	communication request	MPI_Request*	TYPE(MPI_Request)
	)			

Overlapping collectives and computation

Independent collective and local operations:

$$y \leftarrow Ax + (x^t x)y$$

```
1 MPI_Iallreduce( .... x ..., &request);
2 // compute the matrix vector product
3 MPI_Wait(request);
4 // do the addition
```

Simultaneous reductions

Do two reductions (on the same communicator) with different operators simultaneously:

$$\alpha \leftarrow x^t y$$
$$\beta \leftarrow \|z\|_\infty$$

which translates to:

```
1 MPI_Request reqs[2];
2 MPI_Iallreduce
3    ( &local_xy,  &global_xy, 1,MPI_DOUBLE,MPI_SUM,comm,
4      &(reqs[0]) );
5 MPI_Iallreduce
6    ( &local_xinf,&global_xin,1,MPI_DOUBLE,MPI_MAX,comm,
7      &(reqs[1]) );
8 MPI_Waitall(2,reqs,MPI_STATUSES_IGNORE);
```

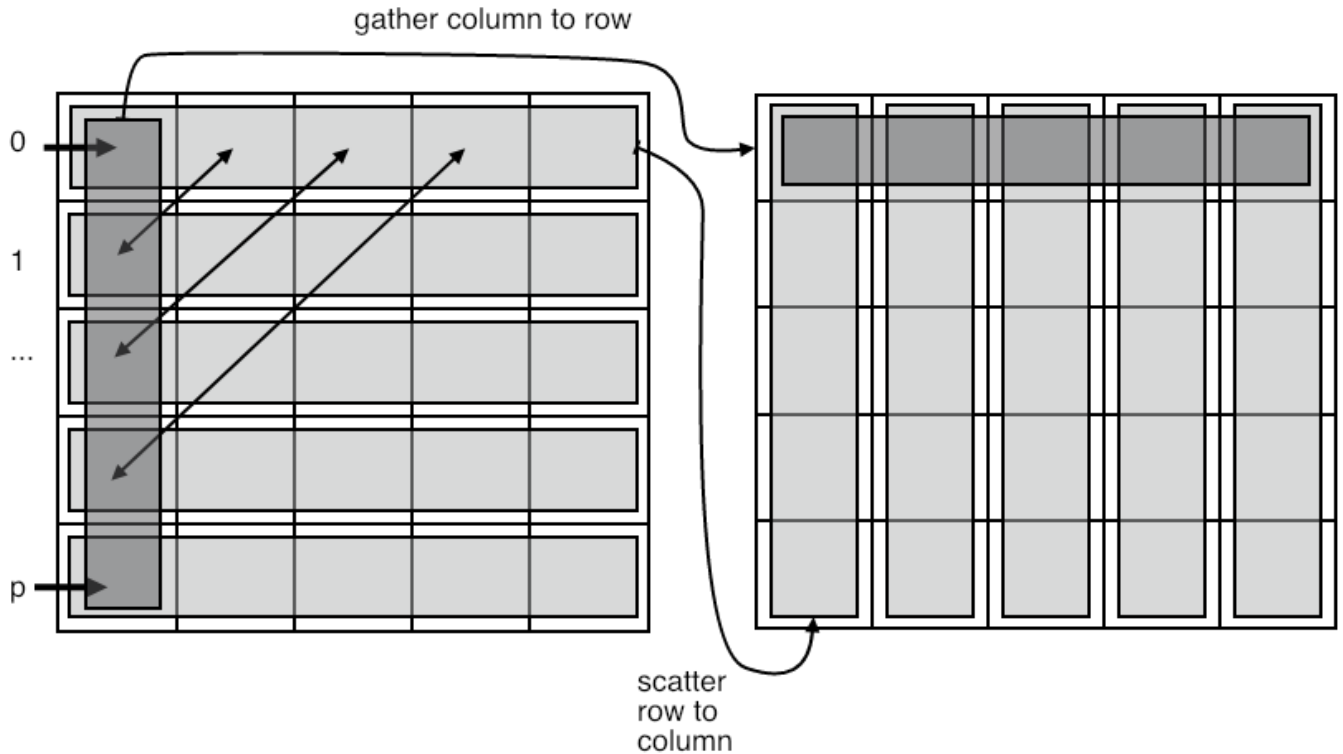
Matching collectives

Blocking and non-blocking collectives don't match:
either all processes call the non-blocking or all call the blocking one.
Thus the following code is incorrect:

```
1 if (rank==root)
2   MPI_Reduce( &x /* ... */ root,comm );
3 else
4   MPI_Ireduce( &x /* ... */ root,comm,&req);
```

This is unlike the point-to-point behavior of non-blocking calls: you can catch a message with `MPI_Irecv` that was sent with `MPI_Send`.

Transpose as gather/scatter



Every process needs to do a scatter or gather.

Simultaneous collectives

Transpose matrix by scattering all rows simultaneously.
Each scatter involves all processes, but with a different spanning tree.

```
1 MPI_Request scatter_requests[nprocs];
2 for (int iproc=0; iproc<nprocs; iproc++) {
3   MPI_Iscatter( regular,1,MPI_DOUBLE,
4                 &(transpose[iproc]),1,MPI_DOUBLE,
5                 iproc,comm,scatter_requests+iproc);
6 }
7 MPI_Waitall(nprocs,scatter_requests,MPI_STATUSES_IGNORE);
```

Persistent collectives

Persistent collectives (MPI-4)

Similar to persistent send/recv:

```
1 double *buffer;
2 MPI_Allreduce_init( buffer ...., &request );
3 for ( ... ) {
4   // fill buffer
5   MPI_Start( request );
6   // possibly other activities
7   MPI_Wait( &request );
8 }
9 MPI_Request_free( &request );
```

Available for all collectives and neighborhood collectives.

Example

```
1  // powerpersist1.c
2  double localnorm,globalnorm=1.;
3  MPI_Request reduce_request;
4  MPI_Allreduce_init
5    ( &localnorm,&globalnorm,1,MPI_DOUBLE,MPI_SUM,
6      comm,MPI_INFO_NULL,&reduce_request);
7  for (int it=0; ; it++) {
8    /*
9    * Matrix vector product
10   */
11   matmult(indata,outdata,buffersize);
12
13 // start computing norm of output vector
14   localnorm = local_l2_norm(outdata,buffersize);
15   double old_globalnorm = globalnorm;
16   MPI_Start( &reduce_request );
17
18 // end computing norm of output vector
19   MPI_Wait( &reduce_request,MPI_STATUS_IGNORE );
20   globalnorm = sqrt(globalnorm);
21 // now 'globalnorm' is the L2 norm of 'outdata'
22   scale(outdata,indata,buffersize,1./globalnorm);
23 }
24 MPI_Request_free( &reduce_request );
```

Note also the `MPI_Info` parameter.

Persistent vs non-blocking

Both request-based.

- Non-blocking is 'ad hoc': buffer info not known before the collective call.
- Persistent allows 'planning ahead': management of internal buffers and such.

Request handling:

- Non-blocking: wait deallocates the request
- Persistent: wait deactivates the request, still requires `MPI_Request_free`.

Non-blocking barrier

Just what is a barrier?

- Barrier is not *time* synchronization but *state* synchronization.
- Test on non-blocking barrier: 'has everyone reached some state'

Use case: adaptive refinement

- Some processes decide locally to alter their structure
- . . . need to communicate that to neighbors
- Problem: neighbors don't know whether to expect update calls, if at all.
- Solution:
  - send update msgs, if any;
  - then post barrier.
  - Everyone probe for updates, test for barrier.

Use case: distributed termination detection

- Distributed termination detection (Matocha and Kamp, 1998): draw a global conclusion with local operations
- Everyone posts the barrier when done;
- keeps doing local computation while testing for the barrier to complete

MPI_Ibarrier

Name	Param name	Explanation	C type	F type
MPI_Ibarrier (				
	comm	communicator	MPI_Comm	TYPE(MPI_Comm)
	request	communication request	MPI_Request*	TYPE(MPI_Request)
	)			

Step 1

Do sends, post barrier.

```
1  // ibarrierprobe.c
2  if (i_do_send) {
3    /*
4    * Pick a random process to send to,
5    * not yourself.
6    */
7    int receiver = rand()%nprocs;
8    MPI_Ssend(&data,1,MPI_FLOAT,receiver,0,comm);
9  }
10 /*
11  * Everyone posts the non-blocking barrier
12  * and gets a request to test/wait for
13  */
14 MPI_Request barrier_request;
15 MPI_Ibarrier(comm,&barrier_request);
```

Step 2

Poll for barrier and messages

```
1  for ( ; ; step++) {
2    int barrier_done_flag=0;
3    MPI_Test(&barrier_request,&barrier_done_flag,
4             MPI_STATUS_IGNORE);
5  //stop if you're done!
6    if (barrier_done_flag) {
7      break;
8    } else {
9  // if you're not done with the barrier:
10     int flag; MPI_Status status;
11     MPI_Iprobe
12       ( MPI_ANY_SOURCE,MPI_ANY_TAG,
13         comm, &flag, &status );
14     if (flag) {
15 // absorb message!
```

3.2. Shared memory

Shared memory myths

Myth:

MPI processes use network calls, whereas OpenMP threads access memory directly, therefore OpenMP is more efficient for shared memory.

Truth:

MPI implementations use copy operations when possible, whereas OpenMP has thread overhead, and affinity/coherence problems.

Main problem with MPI on shared memory: data duplication.

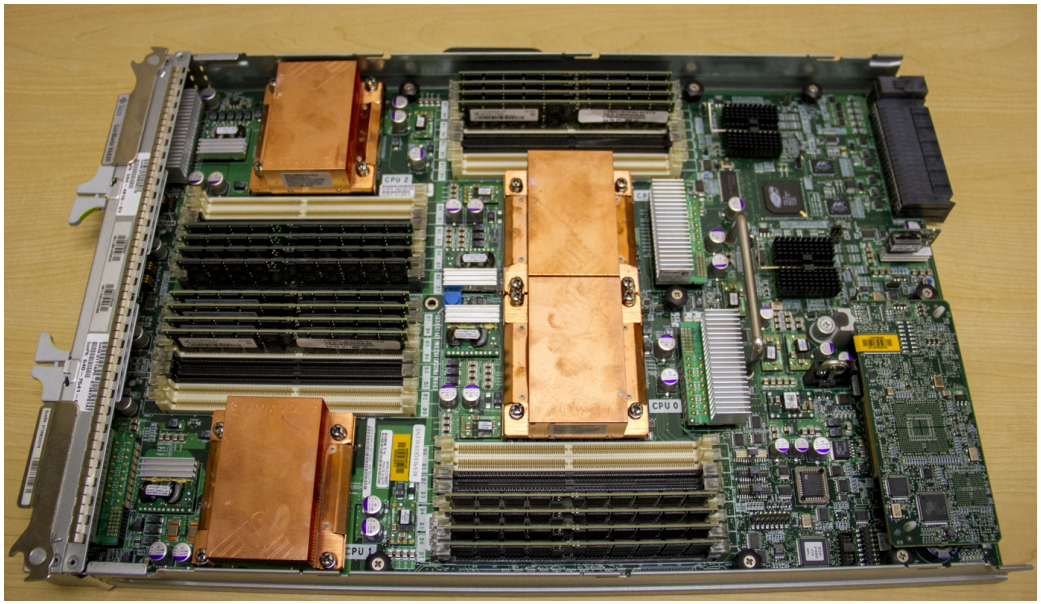
MPI shared memory

- Shared memory access: two processes can access each other's memory through `double*` (and such) pointers, if they are on the same shared memory.
- Limitation: only window memory.
- Non-use case: remote update. This has all the problems of traditional shared memory (race conditions, consistency).
- Good use case: every process needs access to large read-only dataset Example: ray tracing.

Shared memory threatments in MPI

- MPI uses optimizations for shared memory: copy instead of socket call
- One-sided offers 'fake shared memory': you can access another process' data, but only through function calls.
- MPI-3 shared memory gives you a pointer to another process' memory,
  if that process is on the same shared memory.

Shared memory per cluster node



- Cluster node has shared memory
- Memory is attached to specific socket
- beware Non-Uniform Memory Access (NUMA) effects

Shared memory interface

Here is the high level overview; details next.

- Use `MPI_Comm_split_type` to find processes on the same shared memory
- Use `MPI_Win_allocate_shared` to create a window between processes on the same shared memory
  (MPI-4.1: other window creation calls allowed, but the burden is on you!)
- Use `MPI_Win_shared_query` to get pointer to another process' window data.
- You can now use `memcpy` instead of `MPI_Put`.

Discover shared memory

- `MPI_Comm_split_type` splits into communicators of same type.
- Use type: `MPI_COMM_TYPE_SHARED` splitting by shared memory.

Code:

```
1  // commsplittype.c
2  MPI_Info info;
3  MPI_Comm_split_type
4    (MPI_COMM_WORLD,
5     MPI_COMM_TYPE_SHARED,
6     procno,info,&sharedcomm)
       ;
7  MPI_Comm_size
8    (sharedcomm,&new_nprocs);
9  MPI_Comm_rank
10   (sharedcomm,&new_procno);
```

Output:

```
1 There are 10 ranks total
2 [0] is processor 0 in a shared
      ↪group of 5, running on
      ↪c209-010.frontera.tacc.utexas.edu
3 [5] is processor 0 in a shared
      ↪group of 5, running on
      ↪c209-011.frontera.tacc.utexas.edu
```

Exercise 47 `shareddata`

Write a program that uses `MPI_Comm_split_type` to analyze for a run

1. How many nodes there are;
2. How many processes there are on each node.

If you run this program on an unequal distribution, say 10 processes on 3 nodes, what distribution do you find?

```
1 Nodes: 3; processes: 10
2 TACC:  Starting up job 4210429
3 TACC:  Starting parallel tasks...
4 There are 3 nodes
5 Node sizes: 4 3 3
6 TACC:  Shutdown complete. Exiting.
```

Allocate shared window

Use `MPI_Win_allocate_shared` to create a window that can be shared;

- Has to be on a communicator on shared memory
- Example: window is one double.

```
1  // sharedbulk.c
2  MPI_Win node_window;
3  MPI_Aint window_size; double *window_data;
4  if (onnode_procid==0)
5    window_size = sizeof(double);
6  else window_size = 0;
7  MPI_Win_allocate_shared
8    ( window_size,sizeof(double),MPI_INFO_NULL,
9      nodecomm,
10     &window_data,&node_window);
```

Get pointer to other windows

Use `MPI_Win_shared_query`:

```
1 MPI_Aint window_size0; int window_unit; double *win0_addr;
2 MPI_Win_shared_query
3   ( node_window,0,
4     &window_size0,&window_unit, &win0_addr );
```

MPI_Win_shared_query

Name	Param name	Explanation	C type	F type
MPI_Win_shared_query (				
MPI_Win_shared_query_c (				
	win	shared memory window object	MPI_Win	TYPE(MPI_Win)
	rank	rank in the group of window win or MPI_PROC_NULL	int	INTEGER
	size	size of the window segment	MPI_Aint*	INTEGER (KIND=MPI_ADDRESS_KIND)
	disp_unit	local unit size for displacements, in bytes	int* / MPI_Aint*	INTEGER
	baseptr	address for load/store access to window segment	void*	TYPE(C_PTR)
	)			

Allocated memory

Memory will be allocated contiguously
convenient for address arithmetic,
not for NUMA: set *alloc_shared_noncontig* true in `MPI_Info` object.

Example: each window stores one double. Measure distance in bytes:

Strategy: default behavior of shared window allocation

```
Distance 1 to zero: 8
Distance 2 to zero: 16
```

Strategy: allow non-contiguous shared window allocation

```
Distance 1 to zero: 4096
Distance 2 to zero: 8192
```

Question: what is going on here?

Exciting example: bulk data

- Application: ray tracing:
  large read-only data strcture describing the scene
- traditional MPI would duplicate:
  excessive memory demands
- Better: allocate shared data on process 0 of the shared communicator
- Everyone else points to this object.

Split by other hardware types

MPI-4: split by other hardware features through `MPI_COMM_TYPE_HW_GUIDED` and `MPI_Get_hw_resource_types`)

3.3. Process management

Overview

This section discusses processes management; intra communicators.

Commands learned:

- `MPI_Comm_spawn`, `MPI_UNIVERSE_SIZE`
- `MPI_Comm_get_parent`, `MPI_Comm_remote_size`

Process management

- Use `MPI_Attr_get` to retrieve `MPI_UNIVERSE_SIZE` attribute indicating space for creating more processes outside `MPI_COMM_WORLD`.
- New processes have their own `MPI_COMM_WORLD`.
- Communication between the two communicators: 'inter communicator'
  (the old type is 'intra communicator')

Space for processes

Probably a machine dependent component.

Suggested standard:

```
mpiexec -n 4 -usize 8 spawn_manager
```

Intel MPI at TACC:

```
MY_MPIRUN_OPTIONS="-usize 8" ibrun -np 4 spawn_manager
```

Discover size of the universe:

```
1 MPI_Attr_get(MPI_COMM_WORLD, MPI_UNIVERSE_SIZE,
2   (void*)&universe_sizep, &flag);
```

Manager program

```
1  int universe_size, *universe_size_attr,uflag;
2  MPI_Comm_get_attr
3    (comm_world,MPI_UNIVERSE_SIZE,
4     &universe_size_attr,&uflag);
5  if (uflag) {
6    universe_size = *universe_size_attr;
7  } else {
8    printf("This MPI does not support UNIVERSE_SIZE.\nUsing world size");
9    universe_size = world_n;
10 }
11 int work_n = universe_size - world_n;
12 if (world_p==0) {
13   printf("A universe of size %d leaves room for %d workers\n",
14          universe_size,work_n);
15   printf(".. spawning from %s\n",procname);
16 }
```

Manager program (cont'd)

```
1 const char *workerprogram = "./spawnapp";
2 MPI_Comm_spawn(workerprogram,MPI_ARGV_NULL,
3                work_n,MPI_INFO_NULL,
4                0,comm_world,&comm_inter,NULL);
```

Worker program

```
1 // spawnworker.c
2 MPI_Comm_size(MPI_COMM_WORLD,&nworkers);
3 MPI_Comm_rank(MPI_COMM_WORLD,&workerno);
4 MPI_Comm_get_parent(&parent);
```

Were you spawned?

```
1  // spawnapp.c
2  MPI_Comm comm_parent;
3  MPI_Comm_get_parent(&comm_parent);
4  int is_child = (comm_parent!=MPI_COMM_NULL);
5  if (is_child) {
6    int nworkers,workerno;
7    MPI_Comm_size(MPI_COMM_WORLD,&nworkers);
8    MPI_Comm_rank(MPI_COMM_WORLD,&workerno);
9    printf("I detect I am worker %d/%d running on %s\n",
10          workerno,nworkers,procname);
```

3.4. Process topologies

Overview

This section discusses topologies:

- Cartesian topology
- MPI-1 Graph topology (not to be used)
- MPI-3 Graph topology

Commands learned:

- `MPI_Dist_graph_create`, `MPI_DIST_GRAPH`, `MPI_Dist_graph_neighbors_count`
- `MPI_Neighbor_allgather` and such

Process topologies

- Processes communication has a structure
- Example: Cartesian grid, each process 4 (or so) neighbors
  see section 2
  order of communication irrelevant
- Express operations in terms of topology
- Elegance of expression
- MPI can optimize

Process reordering

- Consecutive process numbering often the best:
  divide array by chunks
- Not optimal for grids or general graphs:
- MPI is allowed to renumber ranks
- Graph topology gives information from which MPI can deduce renumbering

MPI-1 topology

- Cartesian topology
  see 2
- Graph topology, globally specified.
  Not scalable, do not use!

MPI-3 topology

- Graph topologies locally specified: scalable!
  Limit cases:
  - each process specifies its own connectivity
  - one process specifies whole graph.
- Neighborhood collectives:
  expression close to the algorithm.

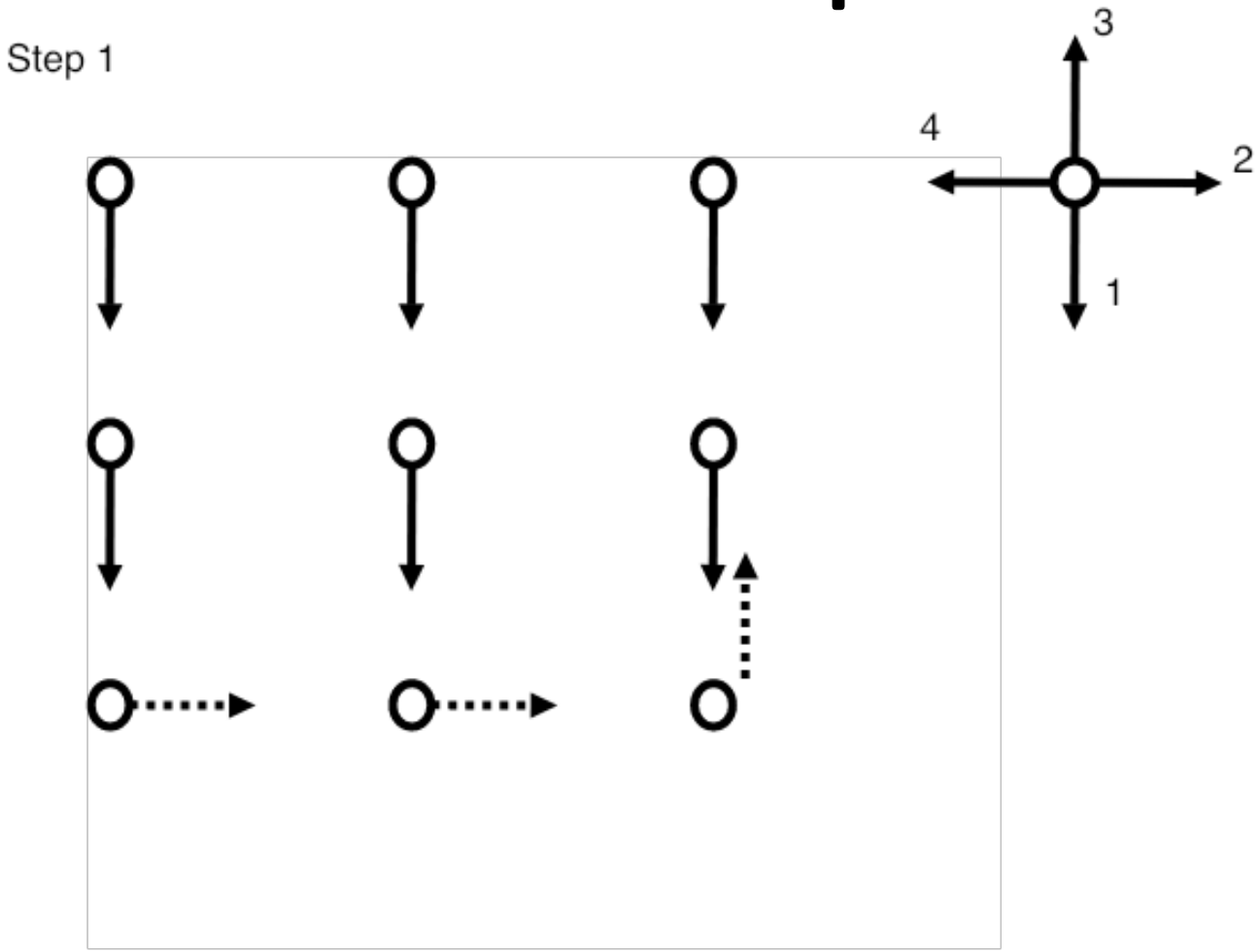
Graph topologies

Example: 5-point stencil

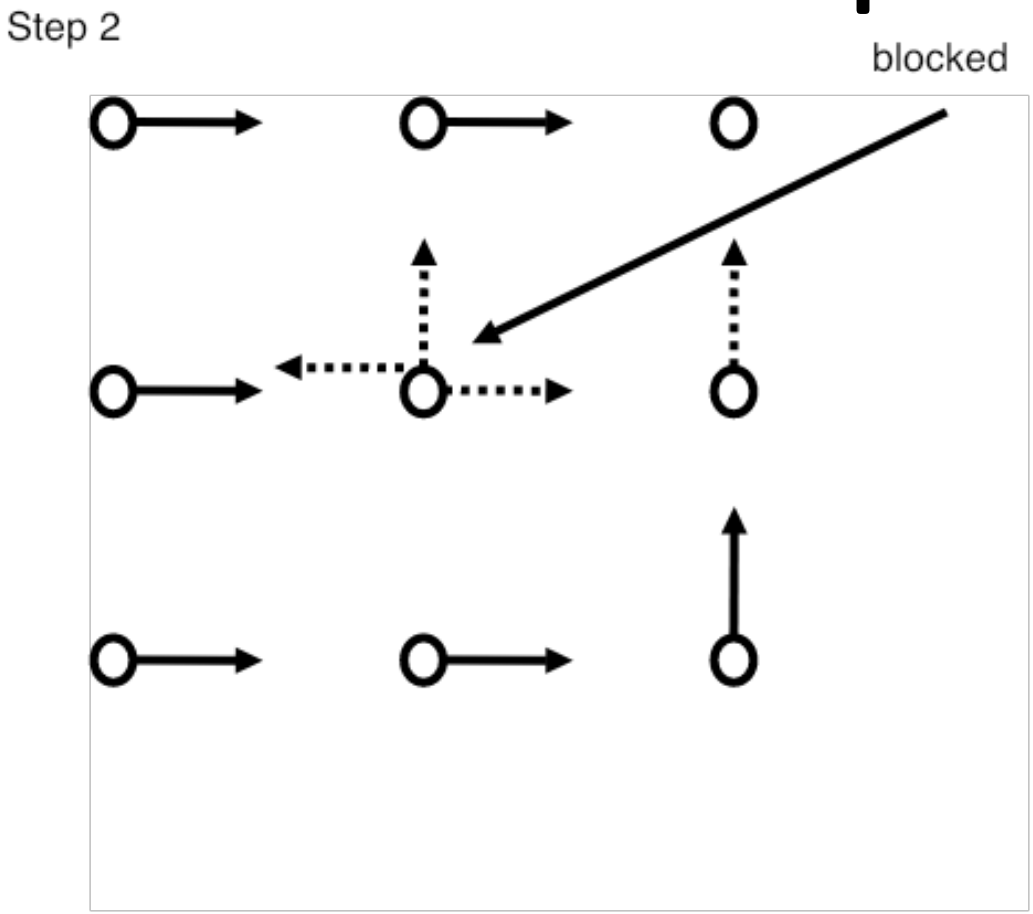
Neighbor exchange, spelled out:

- Each process communicates down/right/up/left
- Send and receive at the same time.
- Can optimally be done in four steps

Step 1



Step 2

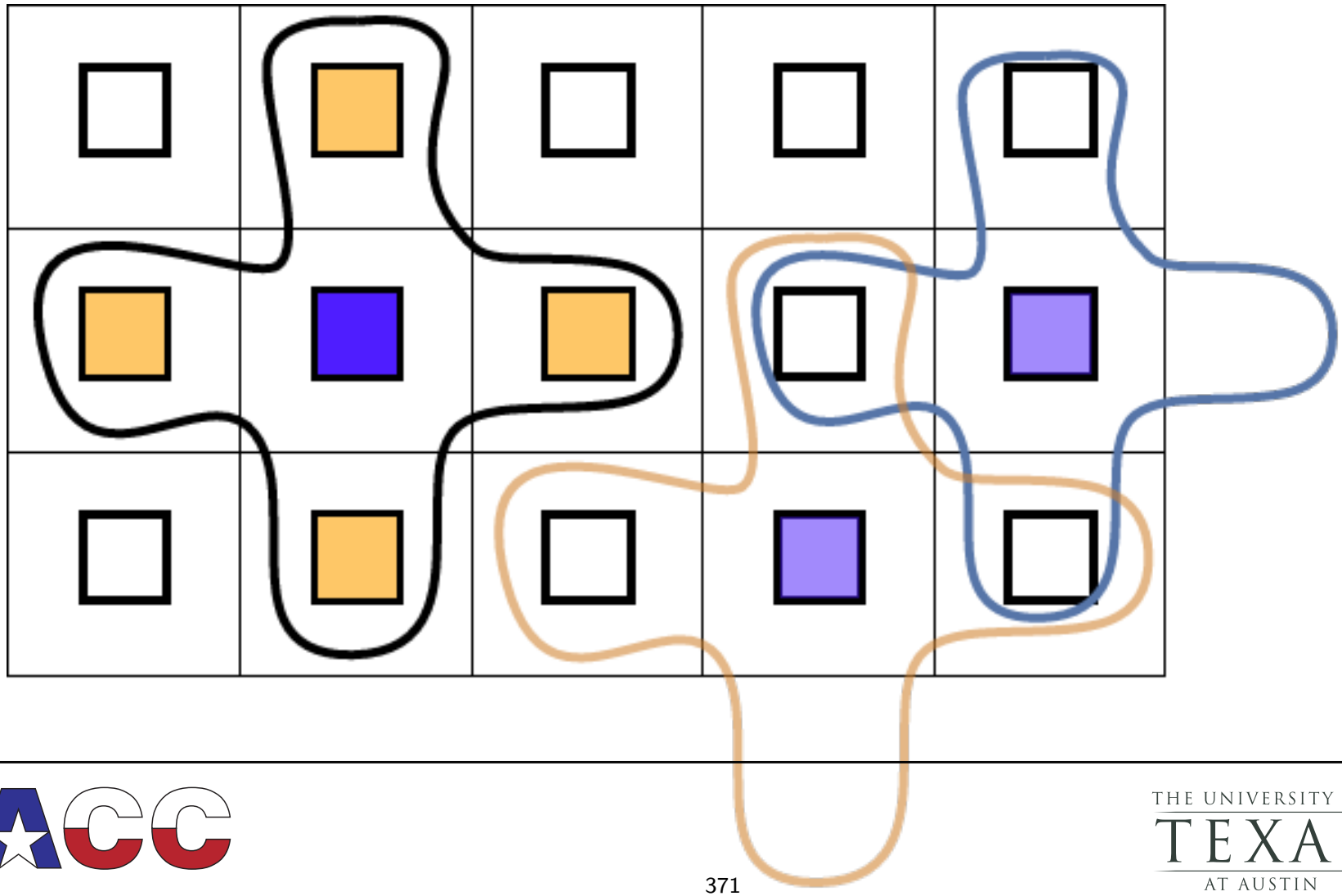


The middle node is blocked because all its targets are already receiving or a channel is occupied:
one missed turn

Neighborhood collective

This is really a 'local gather':
each node does a gather from its neighbors in whatever order.

`MPI_Neighbor_allgather`



Why neighborhood collectives?

- Using `MPI_Isend` / `MPI_Irecv` is like spelling out a collective, imposes order;
- Collectives can use pipelining as opposed to sending a whole buffer;
- Collectives can use spanning trees as opposed to direct connections.

Create graph topology

```
1 int MPI_Dist_graph_create
2    (MPI_Comm comm_old, int nsources, const int sources[],
3     const int degrees[], const int destinations[],
4     const int weights[], MPI_Info info, int reorder,
5     MPI_Comm *comm_dist_graph)
```

- *nsources* how many source nodes described? (Usually 1)
- *sources* the processes being described (Usually *MPI_Comm_rank* value)
- *degrees* how many processes to send to
- *destinations* their ranks
- *weights*: usually set to *MPI_UNWEIGHTED*.
- *info*: *MPI_INFO_NULL* will do
- *reorder*: 1 if dynamically reorder processes

Neighborhood collectives

```
1 int MPI_Neighbor_allgather
2    (const void *sendbuf, int sendcount,MPI_Datatype sendtype,
3     void *recvbuf, int recvcount, MPI_Datatype recvtype,
4     MPI_Comm comm)
```

Like an ordinary `MPI_Allgather`, but
the receive buffer has a length enough for *degree* messages
(instead of comm size).

Neighbor querying

After `MPI_Neighbor_allgather` data in the buffer is *not* in normal rank order.

- `MPI_Dist_graph_neighbors_count` gives actual number of neighbors. (Why do you need this?)
- `MPI_Dist_graph_neighbors` lists neighbor numbers.

MPI_Dist_graph_neighbors_count

Name	Param name	Explanation	C type	F type
MPI_Dist_graph_neighbors_count (				
	comm	communicator with distributed graph topology	MPI_Comm	TYPE(MPI_Comm)
	indegree	number of edges into this process	int*	INTEGER
	outdegree	number of edges out of this process	int*	INTEGER
	weighted	false if MPI_UNWEIGHTED was supplied during creation, true otherwise	int*	LOGICAL
	)			

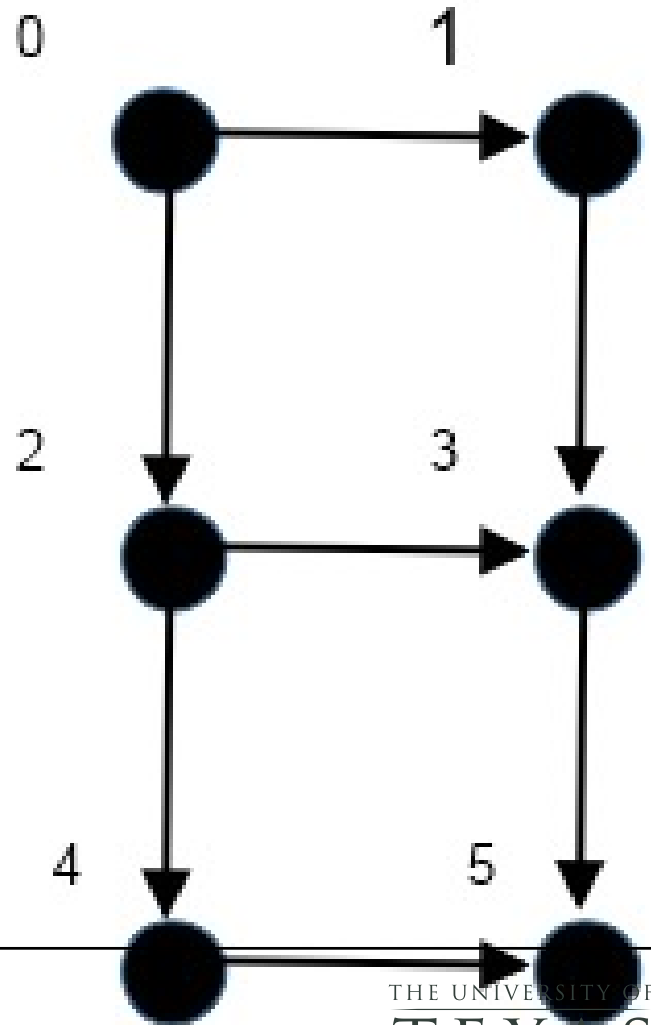
MPI_Dist_graph_neighbors

Name	Param name	Explanation	C type	F type
MPI_Dist_graph_neighbors (				
	comm	communicator with distributed graph topology	MPI_Comm	TYPE(MPI_Comm)
	maxindegree	size of sources and sourceweights arrays	int	INTEGER
	sources	processes for which the calling process is a destination	int[]	INTEGER(maxindegree)
	sourceweights	weights of the edges into the calling process	int[]	INTEGER(*)
	maxoutdegree	size of destinations and destweights arrays	int	INTEGER
	destinations	processes for which the calling process is a source	int[]	INTEGER(maxoutdegree)
	destweights	weights of the edges out of the calling process	int[]	INTEGER(*)
	)			

Example: Systolic graph

Code:

```
// graph.c
for ( int i=0; i<=1; i++ ) {
  int neighb_i = proci+i;
  if (neighb_i<0 || neighb_i>=idim)
    continue;
  int j = 1-i;
  int neighb_j = procj+j;
  if (neighb_j<0 || neighb_j>=jdim)
        continue;
  destinations[ degree++ ] =
        PROC(neighb_i,neighb_j,idim,jdim);
}
MPI_Dist_graph_create
  (comm,
   /* I specify just one proc: me */ 1,
   &procno,&degree,destinations,weights,
   MPI_INFO_NULL,0,
   &comm2d
   );
```



Output

Code:

```
1  int indegree,outdegree,
2    weighted;
3  MPI_Dist_graph_neighbors_count

4    (comm2d,
5     &indegree,&outdegree,
6     &weighted);
7  int
8    my_ij[2] = {proci,procj},
9    other_ij[4][2];
10 MPI_Neighbor_allgather
11   ( my_ij,2,MPI_INT,
12     other_ij,2,MPI_INT,
13     comm2d );
```

Output:

```
1  [ 0 = (0,0)] has  2
       ↪outbound: 1, 2,
2     0 inbound:
3  [ 1 = (0,1)] has  1
       ↪outbound: 3,
4     1 inbound: (0,0)=0
5  [ 2 = (1,0)] has  2
       ↪outbound: 3, 4,
6     1 inbound: (0,0)=0
7  [ 3 = (1,1)] has  1
       ↪outbound: 5,
8     2 inbound: (0,1)=1
       ↪(1,0)=2
9  [ 4 = (2,0)] has  1
       ↪outbound: 5,
10    1 inbound: (1,0)=2
11 [ 5 = (2,1)] has  0
       ↪outbound:
12    2 inbound: (1,1)=3
       ↪(2,0)=4
```

Query

Explicit query of neighbor process ranks.

Code:

```
int indegree,outdegree,
  weighted;
MPI_Dist_graph_neighbors_count
  (comm2d,
   &indegree,&outdegree,
   &weighted);
int
  my_ij[2] = {proci,procj},
  other_ij[4][2];
MPI_Neighbor_allgather
  ( my_ij,2,MPI_INT,
    other_ij,2,MPI_INT,
    comm2d );
```

Output:

```
    0 inbound:
    1 inbound: 0
    1 inbound: 0
    2 inbound: 1 2
    1 inbound: 2
    2 inbound: 4 3
```

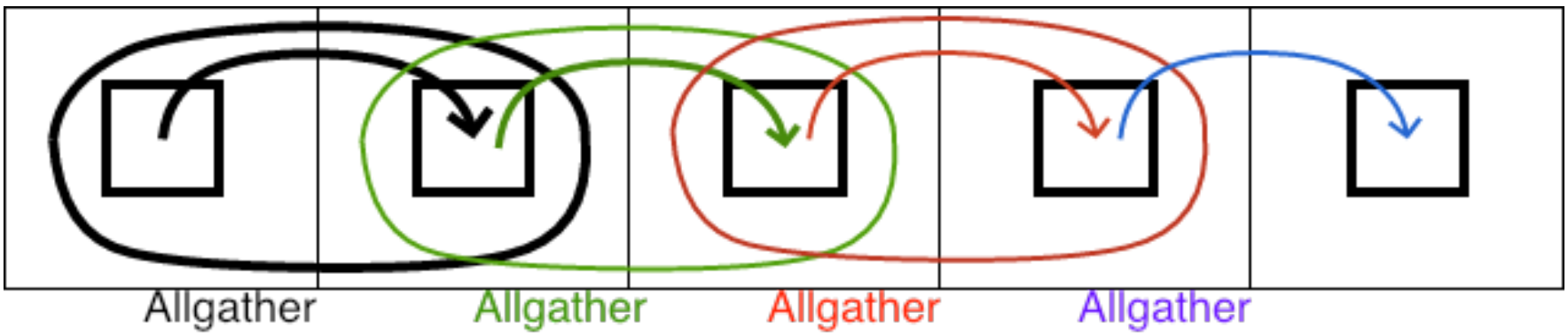
Exercise 48 `rightgraph`

▸ Earlier rightsend exercise

Revisit exercise 21 and solve it using `MPI_Dist_graph_create`. Use figure 382 for inspiration.

Use a degree value of 1.

Inspiring picture for the previous exercise



Solving the right-send exercise with neighborhood collectives

Hints for the previous exercise

Two approaches:

1. Declare just one source: the previous process. Do this! Or:
2. Declare two sources: the previous and yourself. In that case bear in mind slide 375.

More graph collectives

- Heterogeneous: `MPI_Neighbor_alltoallw`.
- Non-blocking: `MPI_Ineighbor_allgather` and such
- Persistent: `MPI_Neighbor_allgather_init`, `MPI_Neighbor_allgatherv_init`.

4. Other

4.1. Tracing, performance, and such

Overview

We briefly touch on peripheral issues issues to MPI.

CMake

CMake

. MPI is discoverable by CMake.

CMake for C

```
1  cmake_minimum_required( VERSION 3.12 )
2  project( ${PROJECTNAME} VERSION 1.0 )
3
4  # https://cmake.org/cmake/help/latest/module/FindMPI.html
5  find_package( MPI )
6
7  add_executable( ${PROJECTNAME} ${PROJECTNAME}.c )
8  target_include_directories(
9          ${PROJECTNAME} PUBLIC
10         ${MPI_C_INCLUDE_DIRS} ${CMAKE_CURRENT_SOURCE_DIR} )
11 target_link_libraries(
12         ${PROJECTNAME} PUBLIC
13         ${MPI_C_LIBRARIES} )
14
15 install( TARGETS ${PROJECTNAME} DESTINATION . )
```

Errors

Built-in handlers

Default: global termination.

```
1 MPI_Comm_set_errhandler(MPI_COMM_WORLD,MPI_ERRORS_ARE_FATAL);
```

MPI-4: *Only terminate on communicator:* `MPI_ERRORS_ABORT`.

Local handling: `MPI_ERRORS_RETURN`:

Handlers on specific classes

Associate error handler with communicator:

`MPI_Comm_set_errhandler` `MPI_Comm_get_errhandler`

Other:

- `MPI_File_set_errhandler`, `MPI_File_call_errhandler`,
  MPI-4: *`MPI_Session_set_errhandler`*, *`MPI_Session_call_errhandler`*,
  `MPI_Win_set_errhandler`, `MPI_Win_call_errhandler`.

Handling errors

```
1 char errtxt[MPI_MAX_ERROR_STRING];
2 int err = status.MPI_ERROR;
3 int len=MPI_MAX_ERROR_STRING;
4 MPI_Error_string(err,errtxt,&len);
5 printf("Waitall error: %d %s\n",err,errtxt);
```

Define new errors

```
1 int nonzero_code;
2 MPI_Add_error_code(nonzero_class,&nonzero_code);
3 MPI_Add_error_string(nonzero_code,"Attempting to send zero buffer");
```

Performance measurement

Timers

MPI has a *wall clock* timer: `MPI_Wtime` which gives the number of seconds from a certain point in the past.

The timer has a resolution of `MPI_Wtick`

Timers can be global

```
1 int *v,flag;
2 MPI_Attr_get( comm, MPI_WTIME_IS_GLOBAL, &v, &flag );
3 if (mytid==0) printf("Time synchronized? %d->%d\n",flag,*v);
```

but probably aren't.

Example

```
1  // pingpong.c
2  if (procno==processA) {
3    t = MPI_Wtime();
4    for (int n=0; n<NEXPERIMENTS; n++) {
5      MPI_Send(send_buf,1,MPI_DOUBLE,
6      MPI_Recv(recv_buf,1,MPI_DOUBLE,
7    }
8    /* total time */          t = MPI_Wtime()-t;
9    /* per experiment */   t /= NEXPERIMENTS;
10   /* single message: */ t /= 2.;
```

Global timing

Processes don't start/end simultaneously. What does a timing result mean overall? Take average or maximum?

Alternative:

```
1    MPI_Barrier(comm)
2    t = MPI_Wtime();
3    // something happens here
4    MPI_Barrier(comm)
5    t = MPI_Wtime()-t;
```

Profiling

See other lecture: MPIP, TAU, et cetera.

Your own profiling interface

Every routine `MPI_Something` calls a routine `PMPI_Something` that does the actual work. You can now write your `MPI_...` routine which calls *`PMPI_`*`...`, and inserting your own profiling calls.

```
main() {
  MPI_Send ( buffer,ct,tp, ... );
}
```

call

```
int MPI_Send ( buffer,ct,tp, ... ) {
  PMPI_Send( buffer,ct,tp, ... ) ;
}
```

Programming for performance

Eager limit

- Optimization for small messages: bypass rendez-vous protocol (slide 144)
- Cross-over point: 'Eager limit'.
- Force efficient messages by increasing the eager limit.
- Beware: decreasing payoff for large messages, and
- Beware: buffers for eager send eat into your available memory.

Eager limit setting

- For Intel MPI: `I_MPI_EAGER_THRESHOLD`
- mvapich2: `MV2_IBA_EAGER_THRESHOLD`
- OpenMPI: `OpenMPI` the `--mca` options *`btl_openib_eager_limit`* and *`btl_openib_rndv_eager_limit`*.

Blocking versus non-blocking

- Non-blocking sends `MPI_Isend` / `MPI_Irecv` can be more efficient than blocking
- Also: allow overlap computation/communication (latency hiding)
- However: can usually not be considered a replacement.

Progress

MPI is not magically active in the background, so latency hiding is not automatic. Same for passive target synchronization and non-blocking barrier completion.

- Dedicated communications processor or thread.
  This is implementation dependent; for instance, Intel MPI:
  `I_MPI_ASYNC_PROGRESS_...` variables.
- Force progress by occasional calls to a polling routine such as `MPI_Iprobe`.

Persistent sends

If a communication between the same pair of processes, involving the same buffer, happens regularly, it is possible to set up a *persistent communication*.

- `MPI_Send_init`
- `MPI_Recv_init`
- `MPI_Start`

Buffering

- MPI has internal buffers: copying costs performance
- Use your own buffer:
  - `MPI_Buffer_attach`
  - `MPI_Bsend`
- Copying is also a problem for derived datatypes.

Graph topology and neighborhood collectives

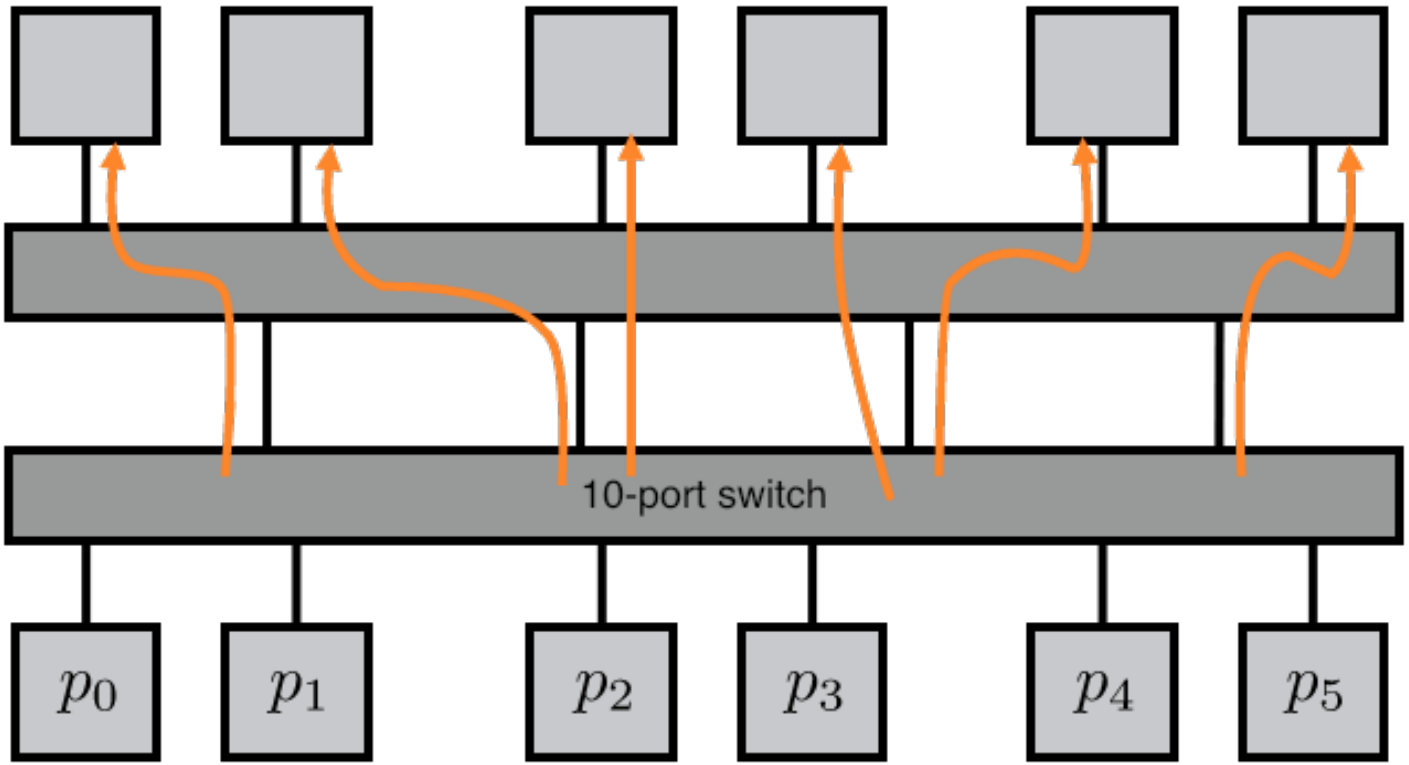
- Mapping problem to architecture sometimes not trivial
- Load balancers: *ParMetis*, *Zoltan*
- Graph topologies: *MPI_Dist_graph_adjacent*:
  allowed to reorder ranks for proximity
- Neighborhood collectives allow MPI to schedule optimally.
  - `MPI_Neighbor_allgather` (and *MPI_Neighbor_allgather_v*)
  - `MPI_Neighbor_alltoall`

Network issues

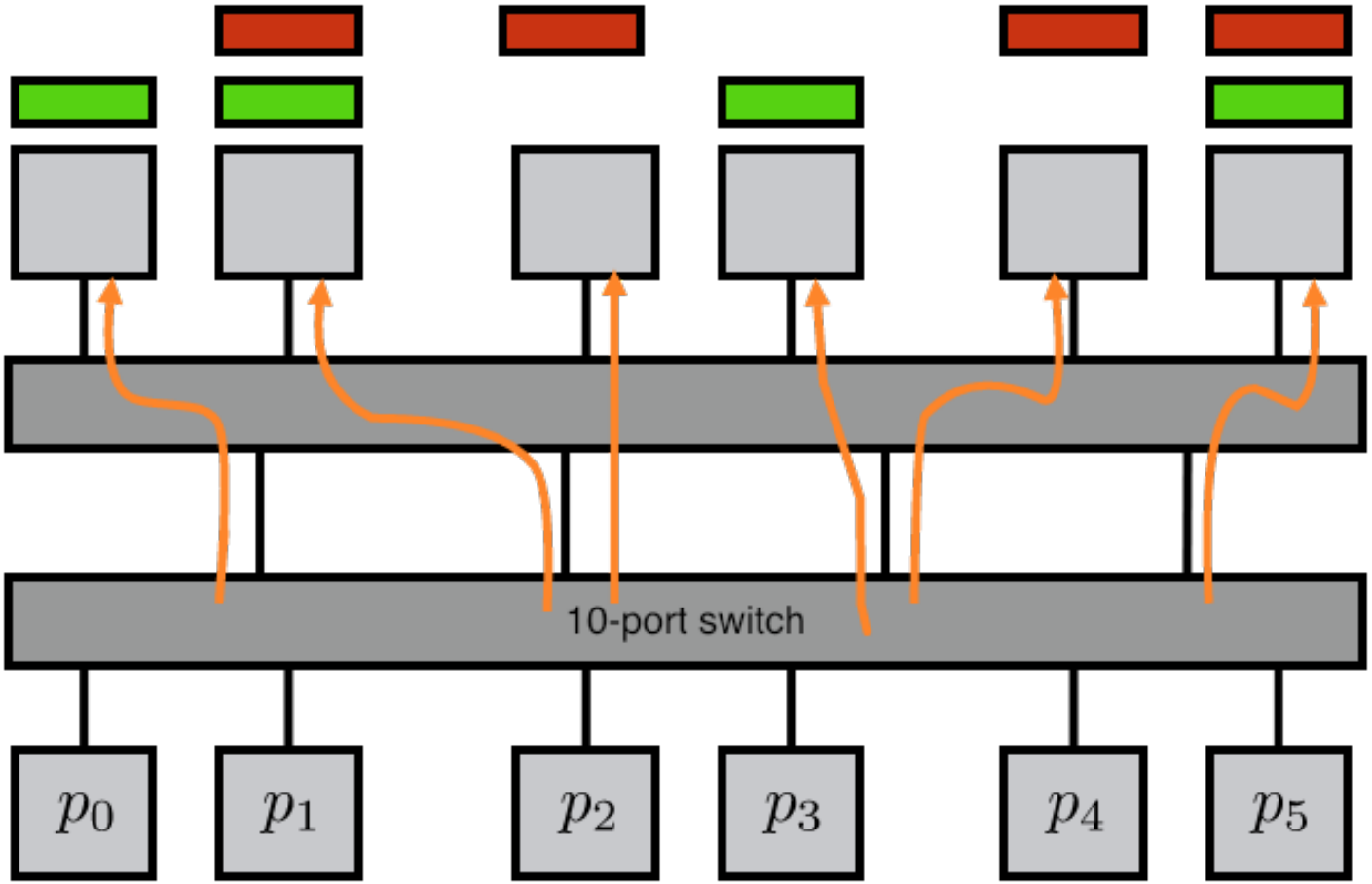
Network contention means that

- Your messages can collide with other jobs
- messages within your job can collide

Output routing



Contention



Offloading and onloading

- Network cards can offer assistance
- Mellanox: off-loading
  limited repertoire of scenarios where it helps
- Intel disagrees: on-loading
- Either way, investigate the capabilities of your network.